

MST EXAM · ONE-STOP REVISION

# COMPUTER ARCHITECTURE

## MST Survival Notes

Units 1 – 6 · full theory in plain language · every diagram

all 46 question-bank questions answered · 2-page cheat sheet

**Prepared for:** Maher Bhatt · B.Tech CSE (Core), ITM (SLS) Baroda University

Built from your Unit 2 – 6 slides and the MST question bank. Unit 1 (RTL, bus, micro-operations, ALU) had no slide uploaded, so it is written from the standard textbook (Mano, *Computer System Architecture*) and matched to the questions.

# Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Read this first</b>                                            | <b>4</b>  |
| Where the marks are (question bank: 218 marks total)              | 4         |
| Four things I found in your material (read these)                 | 4         |
| Rules printed at the bottom of your question bank (do not ignore) | 4         |
| 7-day plan (adjust if your exam date is different)                | 4         |
| How this PDF is organised                                         | 5         |
| <b>The big picture in one page</b>                                | <b>6</b>  |
| <b>UNIT 1 - Register Transfer and Micro-operations</b>            | <b>7</b>  |
| 1.1 Register Transfer Language (RTL)                              | 7         |
| 1.2 Common bus system                                             | 7         |
| 1.3 Types of micro-operations                                     | 8         |
| 1.4 Arithmetic micro-operations                                   | 8         |
| 1.5 Logic micro-operations                                        | 8         |
| 1.6 Shift micro-operations                                        | 9         |
| 1.7 Arithmetic Logic Unit (ALU)                                   | 9         |
| <b>UNIT 2 - Basic Computer Organization and Design</b>            | <b>11</b> |
| 2.1 Instruction codes and the stored-program organization         | 11        |
| 2.2 Computer registers                                            | 12        |
| 2.3 Computer instructions – the three formats                     | 13        |
| 2.4 Timing and control                                            | 15        |
| 2.5 Instruction cycle – fetch, decode, indirect, execute          | 16        |
| 2.6 Memory-reference instructions in detail                       | 17        |
| 2.7 Input-output and interrupt                                    | 17        |
| 2.8 Design of the accumulator unit                                | 18        |
| <b>UNIT 3 - Programming the Basic Computer</b>                    | <b>20</b> |
| 3.1 Machine language vs assembly language                         | 20        |
| 3.2 Assembly language format and pseudo-instructions              | 20        |
| 3.3 The assembler – two-pass translation                          | 20        |
| 3.4 Program loops                                                 | 21        |
| 3.5 Arithmetic programs – double-precision addition               | 22        |
| 3.6 Subroutines                                                   | 22        |
| 3.7 Input-output programming                                      | 23        |
| <b>UNIT 4 - Microprogrammed Control</b>                           | <b>25</b> |
| 4.1 Control memory and the microprogrammed control unit           | 25        |

|                                            |    |
|--------------------------------------------|----|
| 4.2 Address sequencing . . . . .           | 26 |
| 4.3 Microinstruction code format . . . . . | 26 |

## **UNIT 5 - Central Processing Unit** **28**

|                                                               |    |
|---------------------------------------------------------------|----|
| 5.1 General register organization . . . . .                   | 28 |
| 5.2 Stack organization . . . . .                              | 29 |
| 5.3 Instruction formats . . . . .                             | 31 |
| 5.4 Addressing modes . . . . .                                | 32 |
| 5.5 Data transfer, manipulation and program control . . . . . | 33 |
| 5.6 RISC and CISC . . . . .                                   | 34 |

## **UNIT 6 - Pipeline and Vector Processing** **36**

|                                                              |    |
|--------------------------------------------------------------|----|
| 6.1 Flynn's classification and parallel processing . . . . . | 36 |
| 6.2 Pipelining . . . . .                                     | 36 |
| 6.3 Arithmetic pipeline (floating-point addition) . . . . .  | 36 |
| 6.4 Instruction pipeline . . . . .                           | 37 |
| 6.5 RISC pipeline . . . . .                                  | 37 |
| 6.6 Vector processing and array processors . . . . .         | 37 |

## **PART 2 - Model answers to all 46 questions** **38**

|                                                     |    |
|-----------------------------------------------------|----|
| Section A – 3-mark questions (Q1 to Q18) . . . . .  | 38 |
| Section B – 5-mark questions (Q19 to Q34) . . . . . | 42 |
| Section C – 7-mark questions (Q35 to Q46) . . . . . | 52 |

## **PART 3 - Cheat sheet** **63**

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Opcodes of the basic computer (memory-reference, $I = 0 / I = 1$ ) . . . . .   | 63 |
| Fetch, decode, indirect – memorise verbatim . . . . .                          | 63 |
| Interrupt cycle . . . . .                                                      | 63 |
| Addressing modes – one-line effective address (EA) . . . . .                   | 63 |
| 2's complement subtraction (5 steps) . . . . .                                 | 64 |
| ALU select lines (S3 S2 pick the unit, S1 S0 pick Y, Cin = extra +1) . . . . . | 64 |
| General register organization – control word . . . . .                         | 64 |
| Register windows formula . . . . .                                             | 64 |
| Microinstruction fields . . . . .                                              | 64 |
| Pipeline speedup . . . . .                                                     | 64 |
| RPN quick reference . . . . .                                                  | 64 |
| One-paragraph answers for the two most awkward Qs . . . . .                    | 64 |
| Numbers to double-check on the day (from Q4, Q21, Q35) . . . . .               | 64 |

# Read this first

the 3-minute briefing before you touch anything else

Bro, here is the deal: this subject **looks** huge, but the exam is not. The question bank has only **46 questions** and they keep repeating the same handful of topics in 3-, 5- and 7-mark disguises. Learn each topic once properly and you have covered five or six questions with it. That is what this PDF is built around.

## Where the marks are (question bank: 218 marks total)

| Topic                                                                                        | Unit  | Questions                                  | Marks | Share |
|----------------------------------------------------------------------------------------------|-------|--------------------------------------------|-------|-------|
| CPU organization: general registers, stack, addressing modes, instruction formats, RISC/CISC | 5     | Q14 15 16 17 18 30 31 32 33 34 42 43 44 45 | 68    | 31%   |
| Micro-operations, bus, ALU, binary numericals                                                | 1     | Q1 2 3 4 19 20 21 22 35 36                 | 46    | 21%   |
| Basic computer: registers, instruction cycle, MRI, formats, stored program                   | 2     | Q5 6 7 23 24 25 37 38 46                   | 45    | 21%   |
| Microprogrammed control                                                                      | 4     | Q11 12 13 28 29 41                         | 26    | 12%   |
| Assembly, subroutines, programs                                                              | 3     | Q9 10 27 40                                | 18    | 8%    |
| Interrupts                                                                                   | 2 / 5 | Q8 26 39                                   | 15    | 7%    |
| Unit 6 – pipelining and vector processing                                                    | 6     | no question in the bank                    | 0     | 0%    |

**EXAM TIP** Unit 5 alone is almost a third of the bank and it is the easiest unit to score in (tables, lists, short numericals). Unit 6 has zero questions in the bank, so treat it as a bonus read at the very end.

## Four things I found in your material (read these)

1. **The bank says "20 questions" per section, but it actually has  $18 + 16 + 12 = 46$ .** Nothing is missing, the headings are just off.
2. **Q25 asks about "NBR". That is not an instruction.** In the slides, NBR is a *label* for the memory word that holds the constant  $-100$  used to start the loop counter in the "add 100 numbers" program. It is almost certainly a typo for **ISZ**. I explain BSA, LDA, STA, BUN **and** ISZ, and I also explain what NBR is, so you are safe either way.
3. **The slides skip things the bank asks.** The slides have no fetch/decode timing (Q23), no execution RTL of memory-reference instructions (Q25, Q38) and no Unit 1 at all. I filled these from the standard text so every question has full material.
4. **One slide bug:** the "clear locations 500 to 5FF" program loads  $-255$ , but 500 to 5FF is **256** words, so it should be  $-256$  (otherwise location 5FF is left untouched). If you are asked this program, write DEC  $-256$ .

## Rules printed at the bottom of your question bank (do not ignore)

- **Draw a diagram wherever it fits, even if the question does not say so** – it fetches extra marks. Every diagram in this PDF is small enough to redraw in 2 minutes.
- In MCQs do not overwrite; an overwritten answer counts as **0**.
- D2D students: write your class at the top of the answer sheet.

## 7-day plan (adjust if your exam date is different)

| Day | Do this                                                                                               | Goal                                     |
|-----|-------------------------------------------------------------------------------------------------------|------------------------------------------|
| 1   | Unit 1: RTL, bus, micro-op types, shifts, ALU. Then solve Q3, Q4, Q21, Q35 with a pen, no peeking.    | Numericals become automatic              |
| 2   | Unit 2 part A: stored program, registers, common bus, instruction formats, control unit.              | Draw Fig 2.1 – 2.6 from memory           |
| 3   | Unit 2 part B: instruction cycle, fetch/decode RTL, MRI RTL table, interrupts, I/O.                   | Recite T0 T1 T2 and D0T4...D6T6          |
| 4   | Unit 3 (assembly, assembler, subroutine, add-N-numbers program) and Unit 4 (microprogrammed control). | Write the add-N program cold             |
| 5   | Unit 5: general register organization, stack, addressing modes, formats, RISC vs CISC.                | Biggest mark block – be slow and careful |

| Day | Do this                                                                      | Goal                 |
|-----|------------------------------------------------------------------------------|----------------------|
| 6   | Answer section: attempt every 5- and 7-mark question on paper, then compare. | Find your weak spots |
| 7   | Cheat sheet (last pages) twice. Redraw 8 key diagrams. Sleep.                | Walk in calm         |

## How this PDF is organised

- **Part 1 - Notes** (Units 1 to 5, Unit 6 as a short appendix). Plain-language, exam-ready, with coloured boxes: **EXAM TIP**, **MEMORIZE**, **DEV BRAIN** (you code, so I map hardware to things you already know), **TRAP**.
- **Part 2 - Model answers** to all 46 bank questions in the order of the bank, written the way you should write them in the paper. Where a diagram is needed the answer says **DRAW Fig x.y** and the figure is in Part 1.
- **Part 3 - Cheat sheet**: opcodes, RTL, addressing modes, formulas on two pages for the last night.

# The big picture in one page

read this and the rest of the subject will click

A computer is a machine that repeats one loop forever: **fetch an instruction from memory, decode it, execute it**. Everything in the syllabus is just a closer look at one piece of that loop.

| Concept in the syllabus               | What it really is                                                        | You already know it as                                                                    |
|---------------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Stored program (Unit 2)               | Instructions and data live in the same memory, as numbers                | Your code and your variables both sit in RAM                                              |
| PC (program counter)                  | Register holding the address of the next instruction                     | Instruction pointer / "which line runs next"                                              |
| Registers: AR, DR, AC, IR, TR         | Tiny ultra-fast storage inside the CPU                                   | CPU-local variables                                                                       |
| Instruction cycle                     | Fetch, decode, (find operand), execute, repeat                           | <pre>while(true){ op = mem[pc++]; switch(op){...} }</pre>                                 |
| Micro-operation / RTL (Unit 1)        | One tiny register-level step, written as $R2 \leftarrow R1$              | One machine step inside a single line of your code                                        |
| Control unit                          | Circuit that fires the right micro-operations at the right clock tick    | The interpreter loop that decides what to do                                              |
| Hardwired vs microprogrammed (Unit 4) | Control built from gates vs control read from a ROM of microinstructions | Hard-coded if/else vs a lookup table / firmware                                           |
| Addressing modes (Unit 5)             | Different ways to say where the operand is                               | literal, variable, pointer <code>*p</code> , <code>arr[i]</code> , <code>obj.field</code> |
| Stack                                 | LIFO memory with a stack pointer                                         | The call stack, undo history, browser back button                                         |
| Subroutine (BSA / return)             | Reusable code block you branch to and come back from                     | A function call and return                                                                |
| Interrupt                             | Hardware signal that pauses the program to run a handler                 | Event listener / webhook (instead of polling in a loop)                                   |
| Pipeline (Unit 6)                     | Overlap different stages of different instructions                       | CI/CD pipeline or assembly line: many jobs in flight                                      |
| RISC vs CISC                          | Few simple instructions vs many complex ones                             | Small composable primitives vs a big built-in standard library                            |

**DEV BRAIN** Whenever a topic feels abstract, ask: "*what would this be in code?*" Half of computer architecture is the same ideas you already use (pointers, stacks, function calls, events), just implemented with wires.

# UNIT 1 - Register Transfer and Micro-operations

RTL, common bus, arithmetic / logic / shift micro-operations, ALU

No slides were uploaded for this unit. Written from the standard textbook and matched to Q1-4, Q19-22, Q35-36 (46 marks).

## 1.1 Register Transfer Language (RTL)

**Micro-operation:** an elementary operation performed on the data stored in registers during one clock pulse (shift, load, clear, add, increment...). **RTL** is the symbolic notation used to describe such micro-operations precisely and briefly. RTL says *what* happens; the control unit is the circuit that makes it happen.

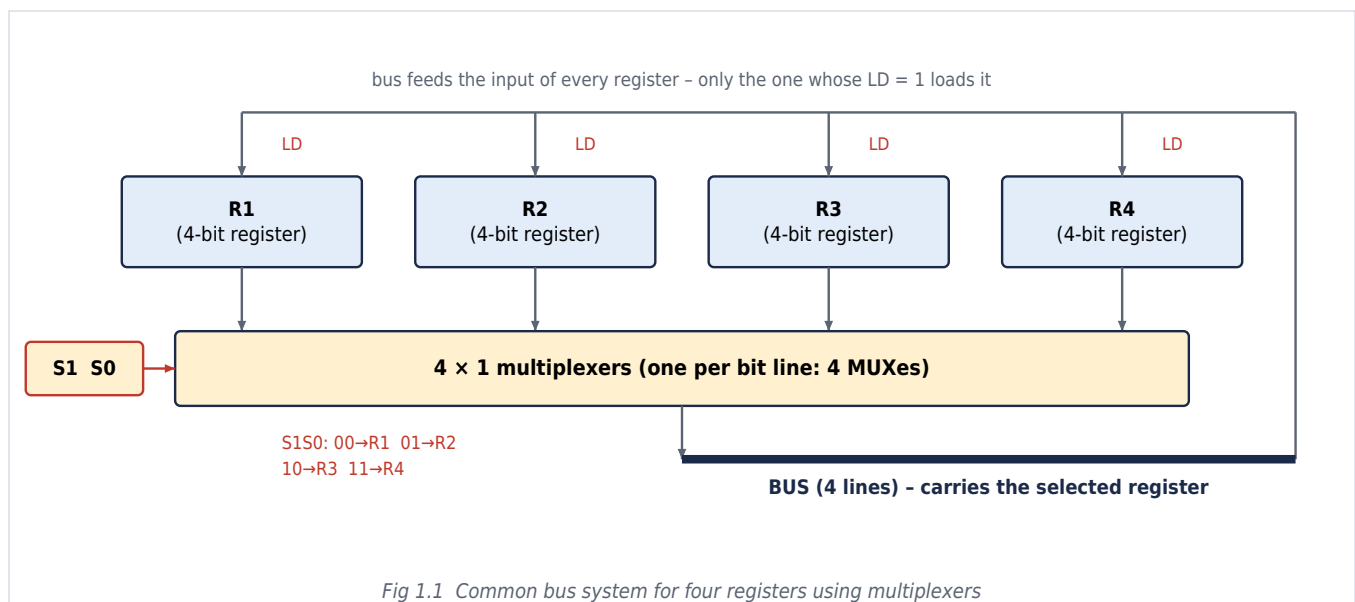
| Symbol                     | Meaning                                                         | Example                              |
|----------------------------|-----------------------------------------------------------------|--------------------------------------|
| Capital letters (+ digits) | Name of a register                                              | AR, PC, IR, R1, R2                   |
| Arrow $\leftarrow$         | Transfer of data (source on the right, destination on the left) | $R2 \leftarrow R1$                   |
| Colon :                    | Separates a control condition from the operation                | P: $R2 \leftarrow R1$                |
| Comma ,                    | Two operations that happen at the same clock edge               | $R2 \leftarrow R1, R1 \leftarrow R2$ |
| Parentheses ( )            | A part (bits) of a register                                     | $R2(0-7), PC(H)$                     |
| Square brackets [ ]        | Memory word selected by an address register                     | $DR \leftarrow M[AR]$                |

### Two RTL statements explained (needed for Q1)

- Simple transfer**  $R2 \leftarrow R1$  - the content of R1 is copied into R2. R1 keeps its value (a copy, not a move).
  - Conditional transfer** P:  $R2 \leftarrow R1$  - the transfer happens only if the control function P = 1, and it happens at the next clock edge. In hardware: P is connected to the LOAD input of R2 and the outputs of R1 to the inputs of R2.
- More forms: simultaneous transfer T:  $R2 \leftarrow R1, R1 \leftarrow R2$  (swap; works because both registers copy on the same edge), memory read  $DR \leftarrow M[AR]$  and memory write  $M[AR] \leftarrow R1$ .

## 1.2 Common bus system

Instead of wiring every register to every other register, all registers share one set of wires called the **bus**. Only **one register may drive the bus at a time**. A multiplexer (or three-state buffers) chooses who gets to drive it.



- Structure (4 registers, 4 bits each):** four 4x1 multiplexers, one per bit line. Select lines **S1 S0** choose which register feeds the bus.
- Two-step transfer:** (1) **read** = MUX places the source register on the bus; (2) **write** = the destination register's **LD** input is set to 1 and it loads at the clock edge.
- For  $n$  registers you need  $\log_2 n$  select lines. For  $n$  registers of  $k$  bits you need  **$k$  multiplexers**, each  $n \times 1$ .
- Three-state alternative:** each register output goes through a tri-state buffer; a decoder enables exactly one buffer, all others stay in high-impedance (disconnected).

| Transfer           | S1 S0 (source on bus) | Load enabled | Meaning                                |
|--------------------|-----------------------|--------------|----------------------------------------|
| $R1 \leftarrow R2$ | 01                    | LD of R1     | $BUS \leftarrow R2, R1 \leftarrow BUS$ |
| $R3 \leftarrow R1$ | 00                    | LD of R3     | $BUS \leftarrow R1, R3 \leftarrow BUS$ |
| $R4 \leftarrow R3$ | 10                    | LD of R4     | $BUS \leftarrow R3, R4 \leftarrow BUS$ |
| $R2 \leftarrow R4$ | 11                    | LD of R2     | $BUS \leftarrow R4, R2 \leftarrow BUS$ |

**DEV BRAIN** The bus is a **shared channel with one writer at a time**, exactly like a mutex around a shared variable. Two registers writing together = bus contention = garbage.

### 1.3 Types of micro-operations

| Type                     | What it does                                                                            | Hardware used                           | Example                        |
|--------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------|--------------------------------|
| <b>Register transfer</b> | Copies data from one register to another; nothing is calculated                         | Bus / MUX / load inputs                 | $R2 \leftarrow R1$             |
| <b>Arithmetic</b>        | Numeric calculation: add, subtract, increment, decrement, negate                        | Binary adder, adder-subtractor, counter | $R3 \leftarrow R1 + R2$        |
| <b>Logic</b>             | Bit-by-bit Boolean operation on each bit position independently (no carry between bits) | One gate per bit                        | $R3 \leftarrow R1 \wedge R2$   |
| <b>Shift</b>             | Moves the bits of a register left or right                                              | Shift register / combinational shifter  | $R1 \leftarrow \text{shl } R1$ |

### 1.4 Arithmetic micro-operations

| RTL                     | Operation               | How it is done                        |
|-------------------------|-------------------------|---------------------------------------|
| $R3 \leftarrow R1 + R2$ | Add                     | Binary adder (chain of full adders)   |
| $R3 \leftarrow R1 - R2$ | Subtract                | $R1 + R2' + 1$ (2's complement of R2) |
| $R2 \leftarrow R2'$     | Complement (1's)        | Invert every bit                      |
| $R2 \leftarrow R2' + 1$ | Negate (2's complement) | Invert, then add 1                    |
| $R1 \leftarrow R1 + 1$  | Increment               | Up-counter                            |
| $R1 \leftarrow R1 - 1$  | Decrement               | Down-counter                          |

- **Binary adder:**  $n$  full adders in a chain; the carry of each feeds the next (ripple carry).
- **Adder-subtractor:** XOR each bit of B with a mode bit **M** and feed M as the input carry.  $M = 0 \rightarrow A + B$ ;  $M = 1 \rightarrow A + B' + 1 = A - B$ .
- **Overflow** (signed numbers):  $V = C_n \oplus C_{n-1}$  (carry into the sign bit XOR carry out of it).

#### How to subtract using 2's complement (5 steps - used in Q21 and Q35)

1. Write both numbers with the same number of bits (8 here).
2. Invert every bit of the number being subtracted (1's complement).
3. Add 1 to get its 2's complement.
4. Add that to the first number.
5. **Discard the final carry.** If the final carry was 1 the result is positive/correct (no borrow). If it was 0 the result is negative and is shown in 2's complement form.

**TRAP** In subtraction the carry-out being 1 does **not** mean overflow - it means "no borrow", i.e.  $A \geq B$  for unsigned numbers. Do not write the carry into your 8-bit answer.

### 1.5 Logic micro-operations

Logic micro-operations treat every bit as an independent binary variable. Example with  $A = 1100$  and  $B = 1010$ :

| Operation | RTL                          | Result | Typical use                                   |
|-----------|------------------------------|--------|-----------------------------------------------|
| AND       | $R3 \leftarrow R1 \wedge R2$ | 1000   | Masking (clear chosen bits)                   |
| OR        | $R3 \leftarrow R1 \vee R2$   | 1110   | Selective set (force chosen bits to 1)        |
| XOR       | $R3 \leftarrow R1 \oplus R2$ | 0110   | Selective complement; XOR with itself = clear |

| Operation | RTL                 | Result | Typical use         |
|-----------|---------------------|--------|---------------------|
| NOT       | $R1 \leftarrow R1'$ | 0011   | Complement all bits |

## 1.6 Shift micro-operations

Shifts move bits sideways. Three kinds: **logical**, **circular (rotate)**, **arithmetic**. Worked on the 4-bit register **1011**:

| Shift                   | Rule                                               | 1011 becomes                   |
|-------------------------|----------------------------------------------------|--------------------------------|
| Logical left (shl)      | 0 enters the LSB; the MSB is lost                  | 0110                           |
| Logical right (shr)     | 0 enters the MSB; the LSB is lost                  | 0101                           |
| Circular left (cil)     | MSB re-enters at the LSB; nothing is lost          | 0111                           |
| Circular right (cir)    | LSB re-enters at the MSB; nothing is lost          | 1101                           |
| Arithmetic right (ashr) | Sign bit is kept and copied to the right; LSB lost | 1101                           |
| Arithmetic left (ashl)  | 0 enters LSB; overflow if the sign bit changes     | 0110 (sign changed → overflow) |

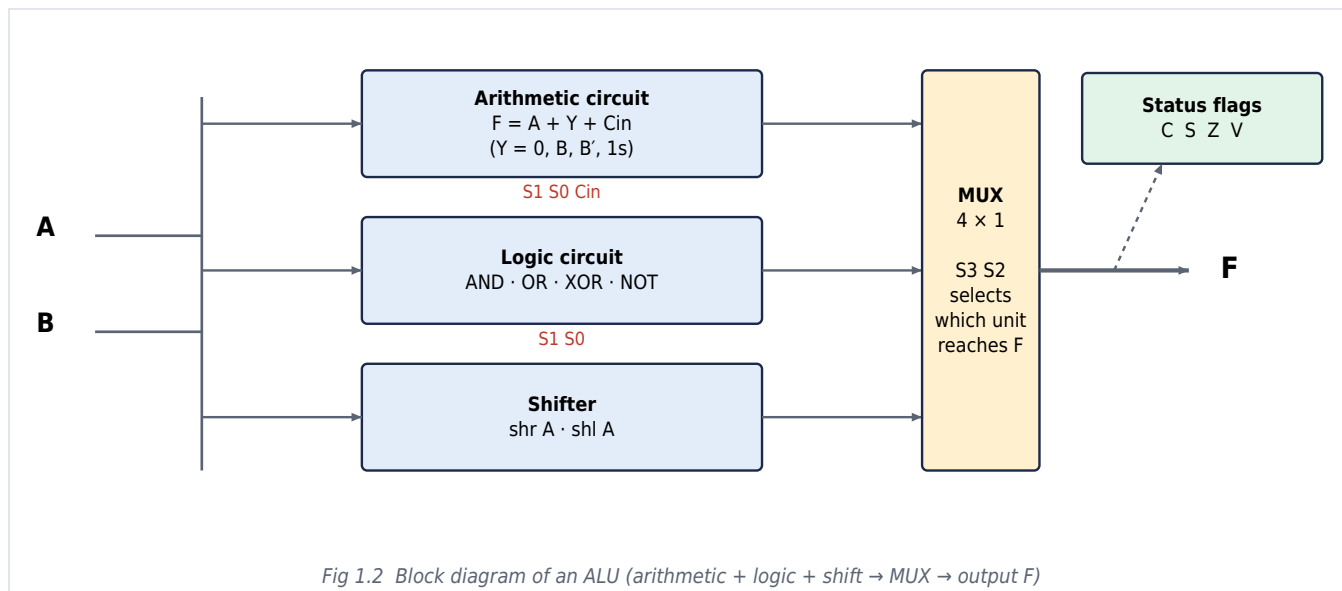
### Logical right shift vs arithmetic right shift (Q20)

|                             | Logical right shift              | Arithmetic right shift                    |
|-----------------------------|----------------------------------|-------------------------------------------|
| <b>Bit entering the MSB</b> | Always 0                         | A copy of the sign bit (MSB unchanged)    |
| <b>Meant for</b>            | Unsigned numbers / bit patterns  | Signed 2's complement numbers             |
| <b>Effect on value</b>      | Divides an unsigned number by 2  | Divides a signed number by 2 (keeps sign) |
| <b>Example (1011 → )</b>    | 0101 (negative became positive!) | 1101 (still negative)                     |

**MEMORIZE** Arithmetic shift **left** =  $\times 2$ , arithmetic shift **right** =  $\div 2$  for signed numbers. Logical shifts never preserve the sign.

## 1.7 Arithmetic Logic Unit (ALU)

The ALU is the combinational circuit that performs the arithmetic, logic and shift micro-operations on the operands and produces a result and status flags. The CPU takes operands from registers (typically AC and DR), the ALU computes, and the result goes back to a register – all inside one clock cycle.



- **Arithmetic circuit:** a 4-bit adder with a MUX on the B input that picks 0, B, B' or all 1s (call it Y). Output  $F = A + Y + C_{in}$ .
- **Logic circuit:** one AND, OR, XOR and NOT gate per bit, chosen by a 4×1 MUX.
- **Shifter:**  $shr A$  /  $shl A$ .
- A final **MUX (S3 S2)** selects which of the three units reaches the output F. **Status flags:** C (carry), S (sign), Z (zero), V (overflow).

| S3 S2 | S1 S0 | C <sub>in</sub> | Operation           | Function                   |
|-------|-------|-----------------|---------------------|----------------------------|
| 00    | 00    | 0               | $F = A$             | Transfer A                 |
| 00    | 00    | 1               | $F = A + 1$         | Increment A                |
| 00    | 01    | 0               | $F = A + B$         | Add                        |
| 00    | 01    | 1               | $F = A + B + 1$     | Add with carry-in          |
| 00    | 10    | 0               | $F = A + B'$        | A plus 1's complement of B |
| 00    | 10    | 1               | $F = A + B' + 1$    | <b>Subtract (A - B)</b>    |
| 00    | 11    | 0               | $F = A - 1$         | Decrement A                |
| 00    | 11    | 1               | $F = A$             | Transfer A                 |
| 01    | 00    | x               | $F = A \wedge B$    | AND                        |
| 01    | 01    | x               | $F = A \vee B$      | OR                         |
| 01    | 10    | x               | $F = A \oplus B$    | XOR                        |
| 01    | 11    | x               | $F = A'$            | Complement A               |
| 10    | x x   | x               | $F = \text{shr } A$ | Shift right                |
| 11    | x x   | x               | $F = \text{shl } A$ | Shift left                 |

**EXAM TIP**

You do not need to memorise all 14 rows. Remember the pattern: **S3S2 picks the unit** (00 arithmetic, 01 logic, 10 shr, 11 shl), **S1S0 picks Y** in the arithmetic unit (0, B, B', all-1s), and Cin adds the extra +1. Subtract = **S1S0 = 10, Cin = 1**.

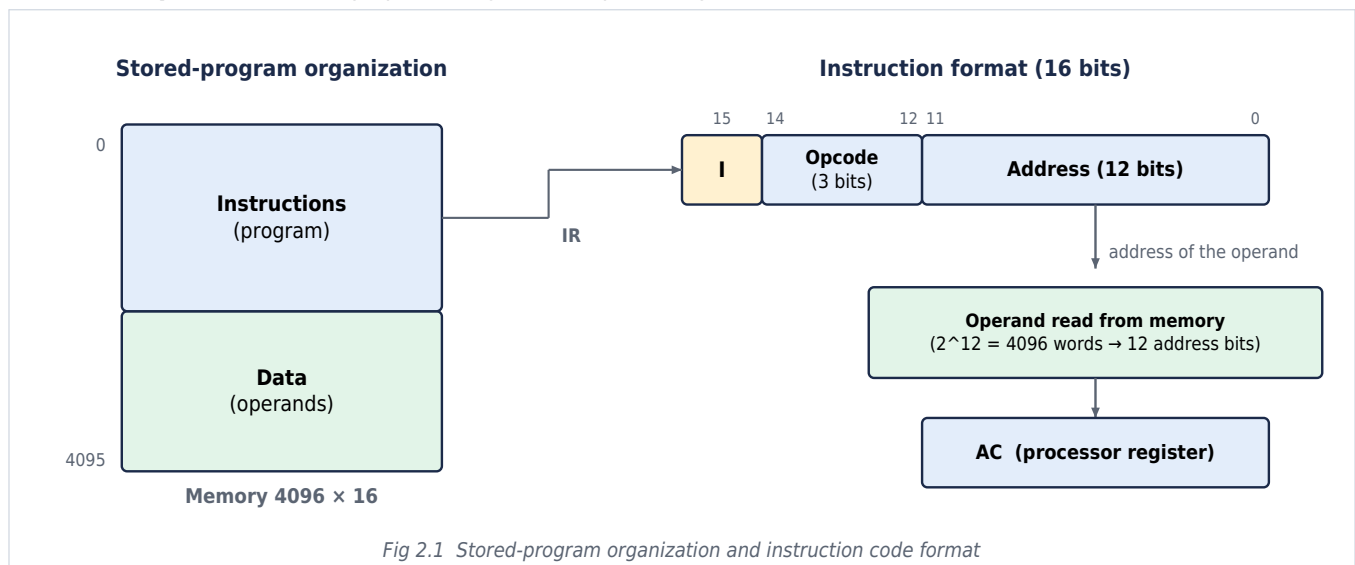
# UNIT 2 - Basic Computer Organization and Design

instruction codes, registers, formats, control, instruction cycle, interrupts

This is the heart of the subject: a tiny imaginary computer (16-bit word, 4096-word memory, one accumulator) that every later unit builds on. If you understand this unit, Units 3 and 4 become easy.

## 2.1 Instruction codes and the stored-program organization

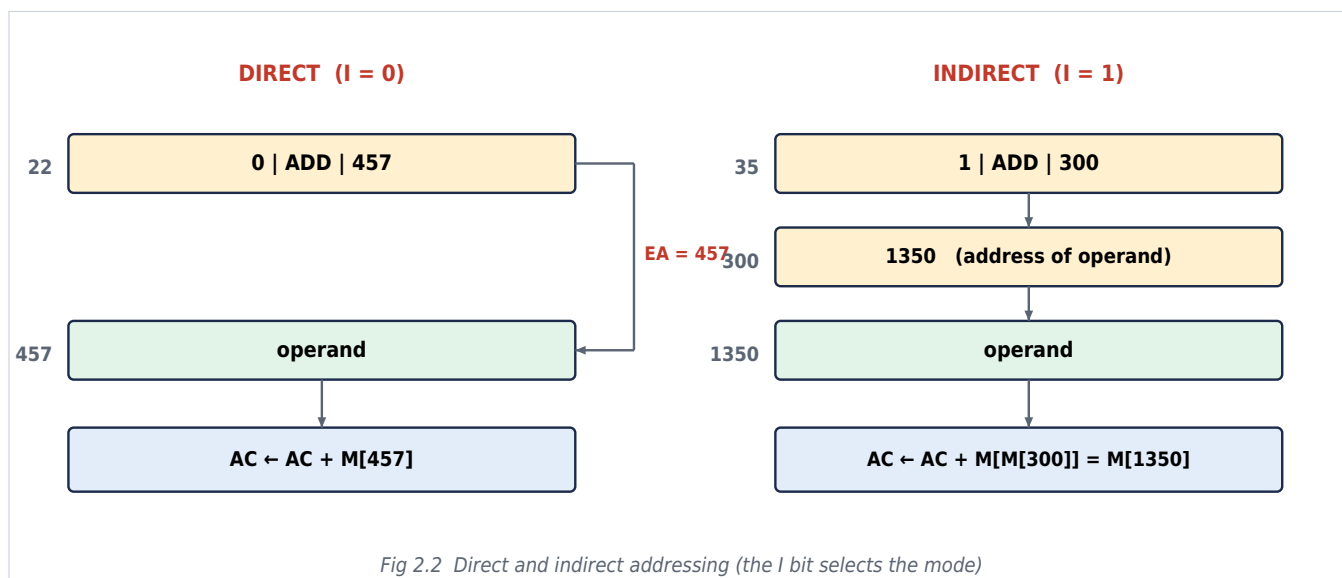
- **Program** = a set of instructions that specify the operations, the operands and the sequence in which processing occurs.
- **Computer instruction** = a binary code that specifies a sequence of micro-operations for the computer.
- **Instruction code** = a group of bits that tells the computer to perform a specific operation. It has two main parts: **opcode** (what to do) and **address / operand** (on what).
- **Opcode** = the bits that define the operation (add, subtract, shift, complement...). For  $2^n$  operations you need at least **n opcode bits**. Every operation gets a unique binary code.



- **Stored-program concept:** instructions (program) and data (operands) are both kept in the **same memory**, in different sections. The control reads instructions from the program section.
- Memory has 4096 words →  $2^{12} = 4096$  → **12 address bits**. Each word is 16 bits, so an instruction = **4 bits** (I + 3-bit opcode) + **12-bit address**.
- The simplest computer has one processor register, the **accumulator (AC)**. The operand read from memory is combined with AC.

### Addressing: immediate, direct, indirect (the I bit)

| Mode                    | What the address part holds                                    | Operand is found...                             |
|-------------------------|----------------------------------------------------------------|-------------------------------------------------|
| <b>Immediate</b>        | The operand itself                                             | inside the instruction (no memory access)       |
| <b>Direct (I = 0)</b>   | The address of the operand                                     | at that address in memory (1 memory access)     |
| <b>Indirect (I = 1)</b> | The address of a word that contains the address of the operand | at the address stored there (2 memory accesses) |

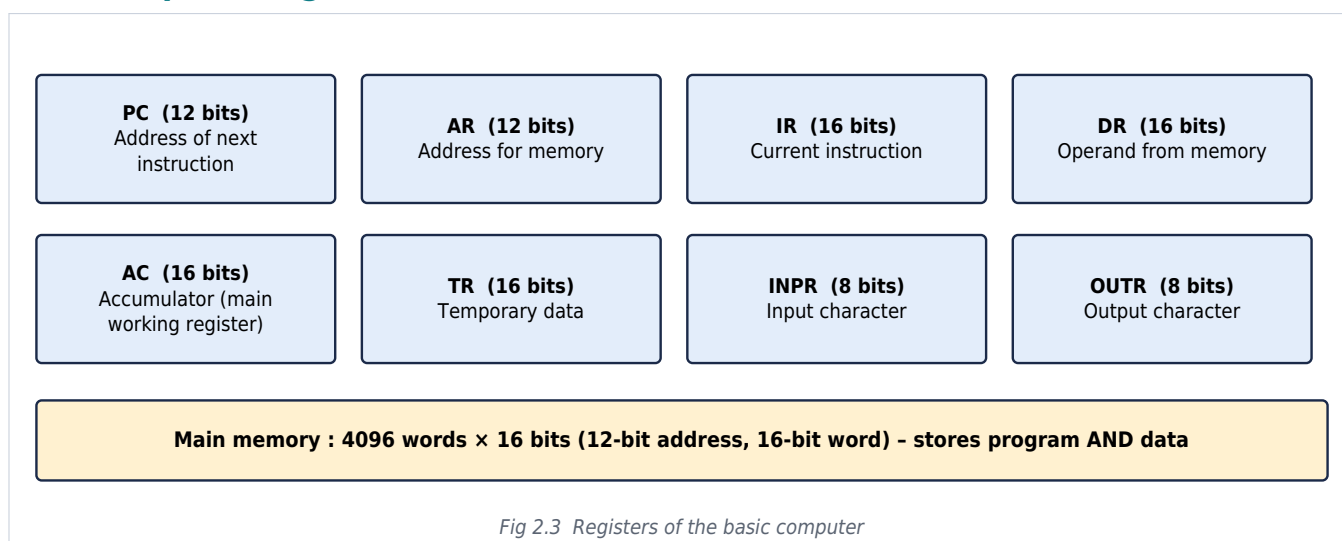


**Effective address (EA)** = the address where the operand really lives. Direct: EA = address part. Indirect: EA = the content of the address part. In Fig 2.2 the instruction at 22 is direct so EA = 457. The instruction at 35 is indirect: location 300 holds 1350, so EA = 1350.

In binary the format of **ADD 457** (address written in hex as in the slide) is: I = 0, opcode = 001, address = 0100 0101 0111 → 0001 0100 0101 0111 = 1457h.

**DEV BRAIN** Immediate = a literal (x = 5). Direct = a variable. Indirect = a pointer dereference (x = \*p): one extra memory hop.

## 2.2 Computer registers

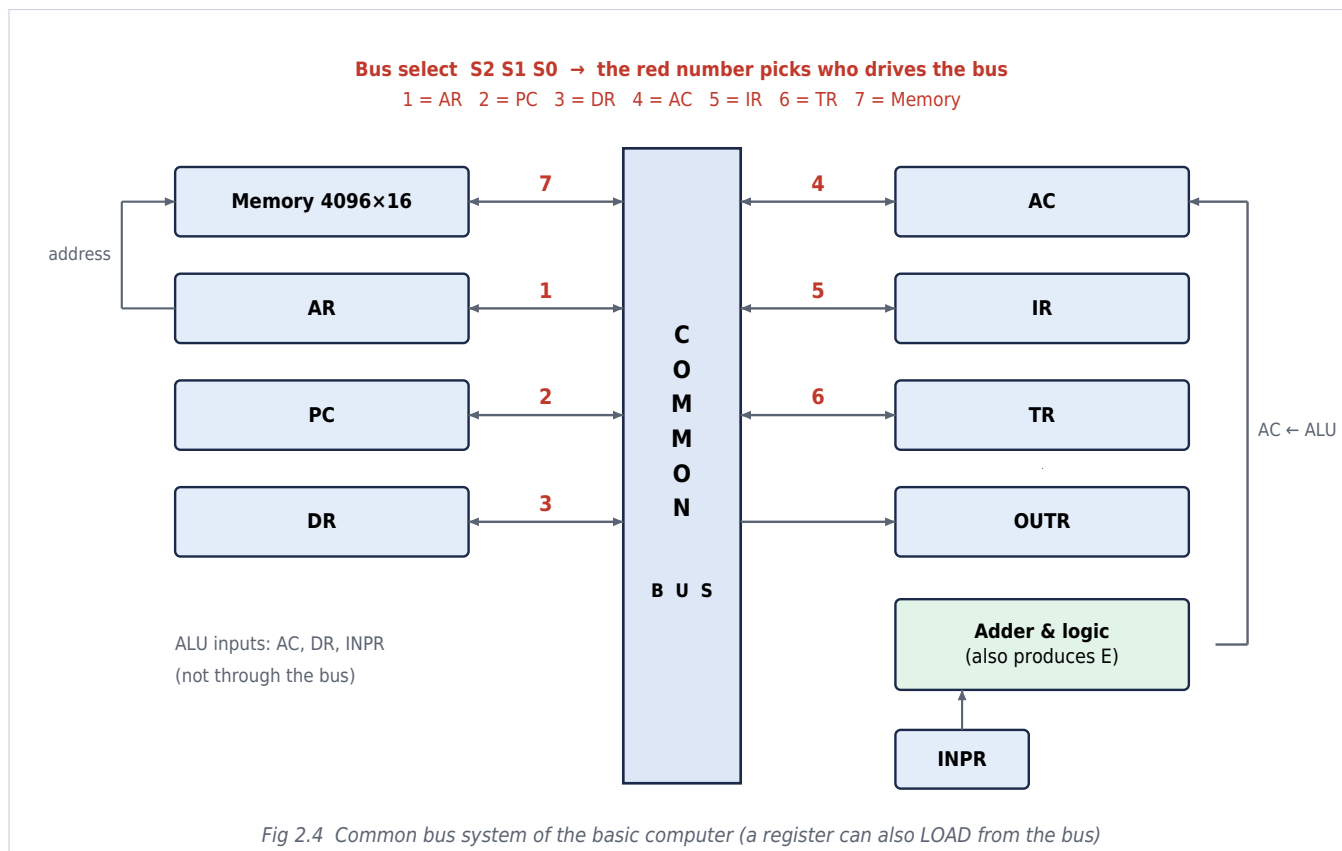


| Register                         | Bits | Function (exam-ready)                                                                          |
|----------------------------------|------|------------------------------------------------------------------------------------------------|
| <b>PC - Program Counter</b>      | 12   | Holds the <b>address of the next instruction</b> to be fetched. Incremented after every fetch. |
| <b>AR - Address Register</b>     | 12   | Holds the <b>address for memory</b> . Every memory read/write uses AR as the address.          |
| <b>IR - Instruction Register</b> | 16   | Holds the <b>instruction code</b> just read from memory (opcode goes to the control unit).     |
| <b>DR - Data Register</b>        | 16   | Holds the <b>operand read from memory</b> (or to be written); one input of the ALU.            |
| <b>AC - Accumulator</b>          | 16   | The general-purpose <b>processing register</b> ; ALU result is stored here.                    |
| <b>TR - Temporary Register</b>   | 16   | Holds <b>temporary data</b> during processing (e.g. the PC during an interrupt).               |
| <b>INPR - Input Register</b>     | 8    | Holds one <b>input character</b> from the keyboard.                                            |
| <b>OUTR - Output Register</b>    | 8    | Holds one <b>output character</b> for the printer/display.                                     |

**Flip-flops you must recognise:** **I** (indirect/mode bit), **E** (extended AC bit, holds carry), **S** (start/stop), **R** (interrupt), **IEN** (interrupt enable), **FGI** / **FGO** (input / output flags), and the 4-bit **sequence counter SC**.

### Common bus system of the basic computer

Memory and the registers are connected through **one common bus** (16 lines). Three select lines **S2 S1 S0** decide which unit puts its data on the bus. Any register can also load from the bus when its **LD** control is 1.



| S2 S1 S0       | 001 | 010 | 011 | 100 | 101 | 110 | 111    |
|----------------|-----|-----|-----|-----|-----|-----|--------|
| Drives the bus | AR  | PC  | DR  | AC  | IR  | TR  | Memory |

- Each register has control inputs: **LD** (load from bus), **INR** (increment), **CLR** (clear).
- Memory address always comes straight from AR** (not via the bus). Memory has Read and Write control lines.
- AC is special:** its input comes from the **Adder & Logic** circuit whose inputs are AC, DR and INPR – not from the bus. AC output goes to the bus and back to the adder.
- Examples:  $AR \leftarrow PC = S2S1S0 = 010, LD(AR)$ .  $IR \leftarrow M[AR] = S = 111, \text{Read}, LD(IR)$ .  $PC \leftarrow PC + 1 = INR(PC)$ .  $M[AR] \leftarrow AC = S = 100, \text{Write}$ .

## 2.3 Computer instructions - the three formats

The basic computer has three 16-bit instruction formats. The control tells them apart using **bit 15 (I)** and the **opcode (bits 14-12)**.

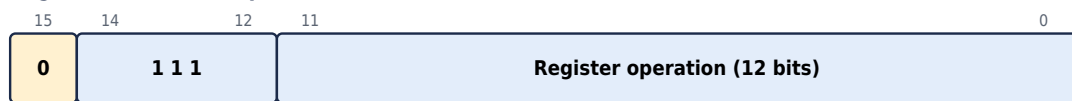
**Memory-reference (opcode 000 - 110)****Register-reference (opcode 111, I = 0)****Input-output (opcode 111, I = 1)**

Fig 2.5 Three 16-bit instruction formats (bit 15 = I / mode bit, bits 14-12 = opcode, bits 11-0 = address or operation)

| Format                    | Recognised by                  | Uses memory?              | Remaining bits                   |
|---------------------------|--------------------------------|---------------------------|----------------------------------|
| <b>Memory-reference</b>   | Opcode $\neq$ 111 (000 to 110) | Yes                       | I = mode bit, 12-bit address     |
| <b>Register-reference</b> | Opcode = 111 and I = 0         | No (works on AC / E)      | 12 bits select the operation     |
| <b>Input-output</b>       | Opcode = 111 and I = 1         | No (works on INPR / OUTR) | 12 bits select the I/O operation |

**TRAP**

For opcode 111 bit 15 is **not** a mode bit any more; it just separates register-reference (0) from input-output (1). That is why hex codes starting with 7 are register-reference and F are input-output.

**Memory-reference instructions (MRI) - 7 instructions**

| Symbol     | Hex (I=0) | Hex (I=1) | Description                          | RTL (execute phase)                                                                                                                                                  |
|------------|-----------|-----------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AND</b> | 0xxx      | 8xxx      | AND memory word to AC                | D0T4: $DR \leftarrow M[AR]$<br>D0T5: $AC \leftarrow AC \wedge DR, SC \leftarrow 0$                                                                                   |
| <b>ADD</b> | 1xxx      | 9xxx      | Add memory word to AC                | D1T4: $DR \leftarrow M[AR]$<br>D1T5: $AC \leftarrow AC + DR, E \leftarrow Cout, SC \leftarrow 0$                                                                     |
| <b>LDA</b> | 2xxx      | Axxx      | Load memory word into AC             | D2T4: $DR \leftarrow M[AR]$<br>D2T5: $AC \leftarrow DR, SC \leftarrow 0$                                                                                             |
| <b>STA</b> | 3xxx      | Bxxx      | Store AC in memory                   | D3T4: $M[AR] \leftarrow AC, SC \leftarrow 0$                                                                                                                         |
| <b>BUN</b> | 4xxx      | Cxxx      | Branch unconditionally               | D4T4: $PC \leftarrow AR, SC \leftarrow 0$                                                                                                                            |
| <b>BSA</b> | 5xxx      | Dxxx      | Branch and save return address       | D5T4: $M[AR] \leftarrow PC, AR \leftarrow AR + 1$<br>D5T5: $PC \leftarrow AR, SC \leftarrow 0$                                                                       |
| <b>ISZ</b> | 6xxx      | Exxx      | Increment word and skip next if zero | D6T4: $DR \leftarrow M[AR]$<br>D6T5: $DR \leftarrow DR + 1$<br>D6T6: $M[AR] \leftarrow DR, \text{ if } (DR = 0) \text{ then } PC \leftarrow PC + 1, SC \leftarrow 0$ |

Note: the slides list the codes but stop before the execution RTL. The RTL column is the standard Mano version that Q25 and Q38 ask for.

**Register-reference instructions (7xxx) - operate on AC and E**

Common control function  $r = D7 \cdot I \cdot T3$ , and  $B_i = IR(i)$  picks the operation. Only one of the bits 0-11 is 1.

| Symbol | Hex  | RTL                                                                  | Meaning                  |
|--------|------|----------------------------------------------------------------------|--------------------------|
| CLA    | 7800 | $rB11: AC \leftarrow 0$                                              | Clear AC                 |
| CLE    | 7400 | $rB10: E \leftarrow 0$                                               | Clear E                  |
| CMA    | 7200 | $rB9: AC \leftarrow AC'$                                             | Complement AC            |
| CME    | 7100 | $rB8: E \leftarrow E'$                                               | Complement E             |
| CIR    | 7080 | $rB7: AC \leftarrow shr AC, AC(15) \leftarrow E, E \leftarrow AC(0)$ | Circulate right AC and E |

| Symbol | Hex  | RTL                                                                          | Meaning                  |
|--------|------|------------------------------------------------------------------------------|--------------------------|
| CIL    | 7040 | rB6: $AC \leftarrow \text{shl } AC, AC(0) \leftarrow E, E \leftarrow AC(15)$ | Circulate left AC and E  |
| INC    | 7020 | rB5: $AC \leftarrow AC + 1$                                                  | Increment AC             |
| SPA    | 7010 | rB4: if $(AC(15) = 0)$ then $PC \leftarrow PC + 1$                           | Skip next if AC positive |
| SNA    | 7008 | rB3: if $(AC(15) = 1)$ then $PC \leftarrow PC + 1$                           | Skip next if AC negative |
| SZA    | 7004 | rB2: if $(AC = 0)$ then $PC \leftarrow PC + 1$                               | Skip next if AC zero     |
| SZE    | 7002 | rB1: if $(E = 0)$ then $PC \leftarrow PC + 1$                                | Skip next if E zero      |
| HLT    | 7001 | rB0: $S \leftarrow 0$                                                        | Halt the computer        |

### Input-output instructions (Fxxx)

Common control function  $p = D7 \cdot I \cdot T3$ , with  $B_i = IR(i)$ .

| Symbol | Hex  | RTL                                                      | Meaning                  |
|--------|------|----------------------------------------------------------|--------------------------|
| INP    | F800 | pB11: $AC(0-7) \leftarrow \text{INPR}, FGI \leftarrow 0$ | Input character to AC    |
| OUT    | F400 | pB10: $\text{OUTR} \leftarrow AC(0-7), FGO \leftarrow 0$ | Output character from AC |
| SKI    | F200 | pB9: if $(FGI = 1)$ then $PC \leftarrow PC + 1$          | Skip on input flag       |
| SKO    | F100 | pB8: if $(FGO = 1)$ then $PC \leftarrow PC + 1$          | Skip on output flag      |
| ION    | F080 | pB7: $IEN \leftarrow 1$                                  | Interrupt enable on      |
| IOF    | F040 | pB6: $IEN \leftarrow 0$                                  | Interrupt enable off     |

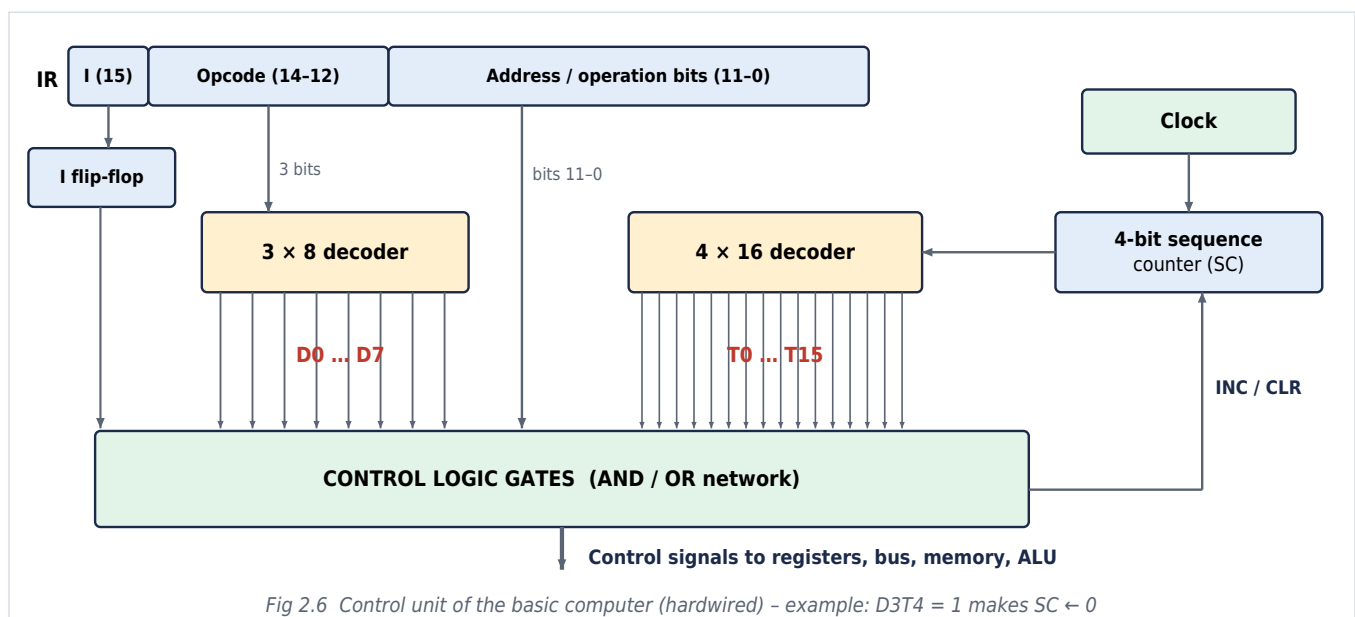
**MEMORIZE** **Instruction set completeness:** a computer's instruction set is complete if it has enough instructions of four kinds: **arithmetic, logic and shift** (ADD, CMA, INC, CIR, CIL, AND, CLA); **data transfer** (LDA, STA); **control** (BUN, BSA, ISZ); **input/output** (INP, OUT).

## 2.4 Timing and control

Two ways to build the control unit:

| Hardwired control                                                                      | Microprogrammed control                               |
|----------------------------------------------------------------------------------------|-------------------------------------------------------|
| Control logic built from <b>gates, flip-flops, decoders</b> and other digital circuits | Control information stored in a <b>control memory</b> |
| Very fast (optimised wiring)                                                           | Slower (a memory read per microinstruction)           |
| Any change means <b>re-wiring</b> the hardware                                         | Change by <b>updating the microprogram</b>            |

The basic computer uses hardwired control. Its control unit has: **two decoders, a sequence counter and control logic gates**.



- The instruction in IR is split: bit 15 goes to the **I flip-flop**; bits 14-12 (opcode) go to a **3×8 decoder** giving **D0 ... D7**; bits 11-0 go straight to the control gates.
- A **4-bit sequence counter (SC)** counts 0 to 15 and a **4×16 decoder** turns it into timing signals **T0 ... T15**.
- SC is **incremented** every clock pulse, or **cleared** to 0 when an instruction finishes, so the next timing signal is T0 again.
- Example D3T4**:  $SC \leftarrow 0$  - for STA (D3), at time T4 the memory write is done and SC is cleared, so T5 never happens and control jumps back to T0 (start of the next fetch). Each control function is the AND of one D signal and one T signal.

**DEV BRAIN** Think of **D0-D7** as the case of the switch (which instruction) and **T0-T15** as the step counter inside that case (which micro-step). D2T5 = "instruction 2, step 5".

## 2.5 Instruction cycle - fetch, decode, indirect, execute

Every instruction goes through these phases and then control goes back to fetch the next one, until HLT:

- Fetch** the instruction from memory.
- Decode** the instruction.
- Read the effective address** from memory if the instruction is indirect.
- Execute** the instruction.

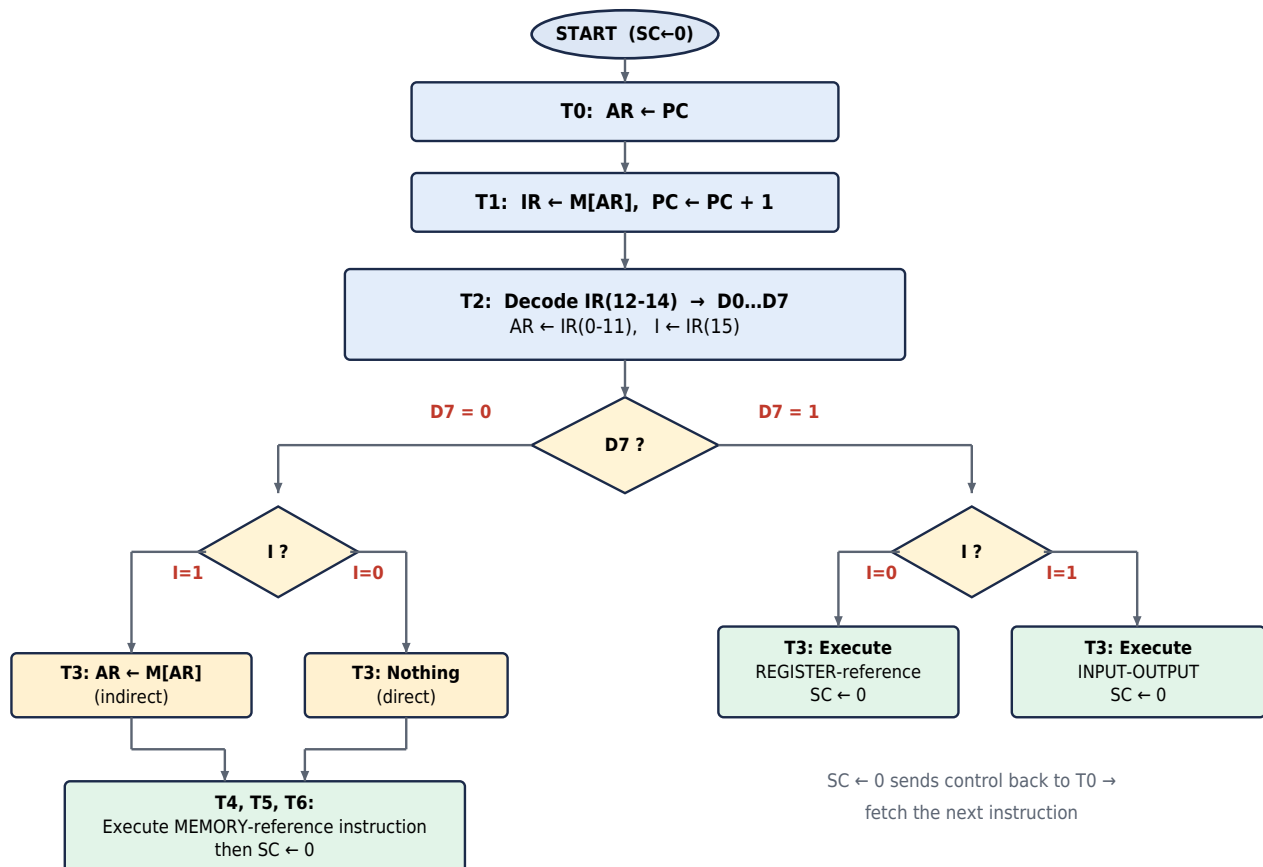


Fig 2.7 Flowchart of the instruction cycle (fetch → decode → indirect → execute)

### Fetch and decode in RTL (Q6, Q23) - memorise these three lines

| Timing    | RTL                                                                                             | What actually happens                                                                                                                    |
|-----------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>T0</b> | $AR \leftarrow PC$                                                                              | PC is put on the bus ( $S = 010$ ) and loaded into AR (LD of AR). AR now holds the address of the instruction.                           |
| <b>T1</b> | $IR \leftarrow M[AR], PC \leftarrow PC + 1$                                                     | Memory is read ( $S = 111$ ) and the instruction goes into IR. In the same pulse PC is incremented so it points to the next instruction. |
| <b>T2</b> | $D0...D7 \leftarrow \text{decode } IR(12-14),$<br>$AR \leftarrow IR(0-11), I \leftarrow IR(15)$ | The 3×8 decoder decodes the opcode. The address bits are copied to AR and the mode bit to the I flip-flop. Decode is finished.           |

## After decode - what happens at T3

| Condition                            | RTL at T3             | Meaning                                             |
|--------------------------------------|-----------------------|-----------------------------------------------------|
| D7' · I (memory-reference, indirect) | $AR \leftarrow M[AR]$ | Read the effective address from memory              |
| D7' · I' (memory-reference, direct)  | nothing               | Address in AR is already the effective address      |
| D7 · I' (register-reference)         | execute at T3         | Executed immediately (rB_i), then $SC \leftarrow 0$ |
| D7 · I (input-output)                | execute at T3         | Executed immediately (pB_i), then $SC \leftarrow 0$ |

**DEV BRAIN** T0-T2 is  $op = mem[pc++]$ ;  $switch(op \gg 12)$ . T3 with the indirect case is  $addr = *addr$ . Everything after is the case body.

## 2.6 Memory-reference instructions in detail

After T3 every memory-reference instruction has the effective address in AR. The execution RTL is in the MRI table of section 2.3. In words:

- **AND** - read the word into DR, then  $AC \leftarrow AC \wedge DR$  (bitwise AND).
- **ADD** - read the word into DR, then  $AC \leftarrow AC + DR$ ; the carry-out goes to E.
- **LDA** - read the word into DR, then copy DR to AC. (Memory  $\rightarrow$  AC.)
- **STA** - write AC to the memory word at AR. (AC  $\rightarrow$  memory.) Only one step, so it clears SC at T4.
- **BUN** -  $PC \leftarrow AR$ . The effective address becomes the next instruction address (a jump).
- **BSA** - Branch and Save return Address: store PC (the address of the *next* instruction) in  $M[AR]$ , then start the subroutine at  $AR + 1$ . This is how a subroutine is **called**.
- **ISZ** - Increment and Skip if Zero: read the word, add 1, write it back, and if the result is 0 skip the next instruction. Used as a **loop counter**.

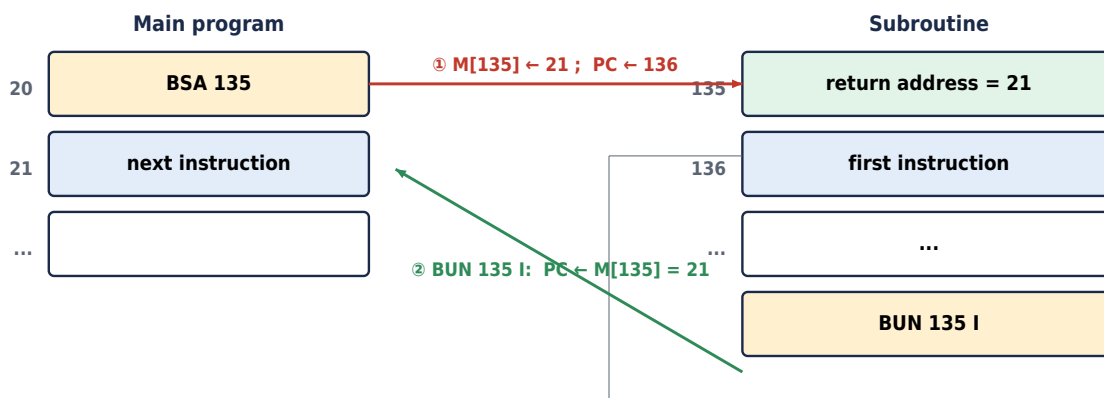


Fig 3.1 BSA (call) stores the return address; BUN ... I (return) reads it back

**EXAM TIP** Q25 mentions **NBR**. NBR is not an instruction: in the "add 100 numbers" program it is a **label** for the word that holds the constant  $-100$  (DEC  $-100$ ) used to initialise the loop counter CTR. Explain BSA, LDA, STA, BUN (and ISZ) and add one line about NBR.

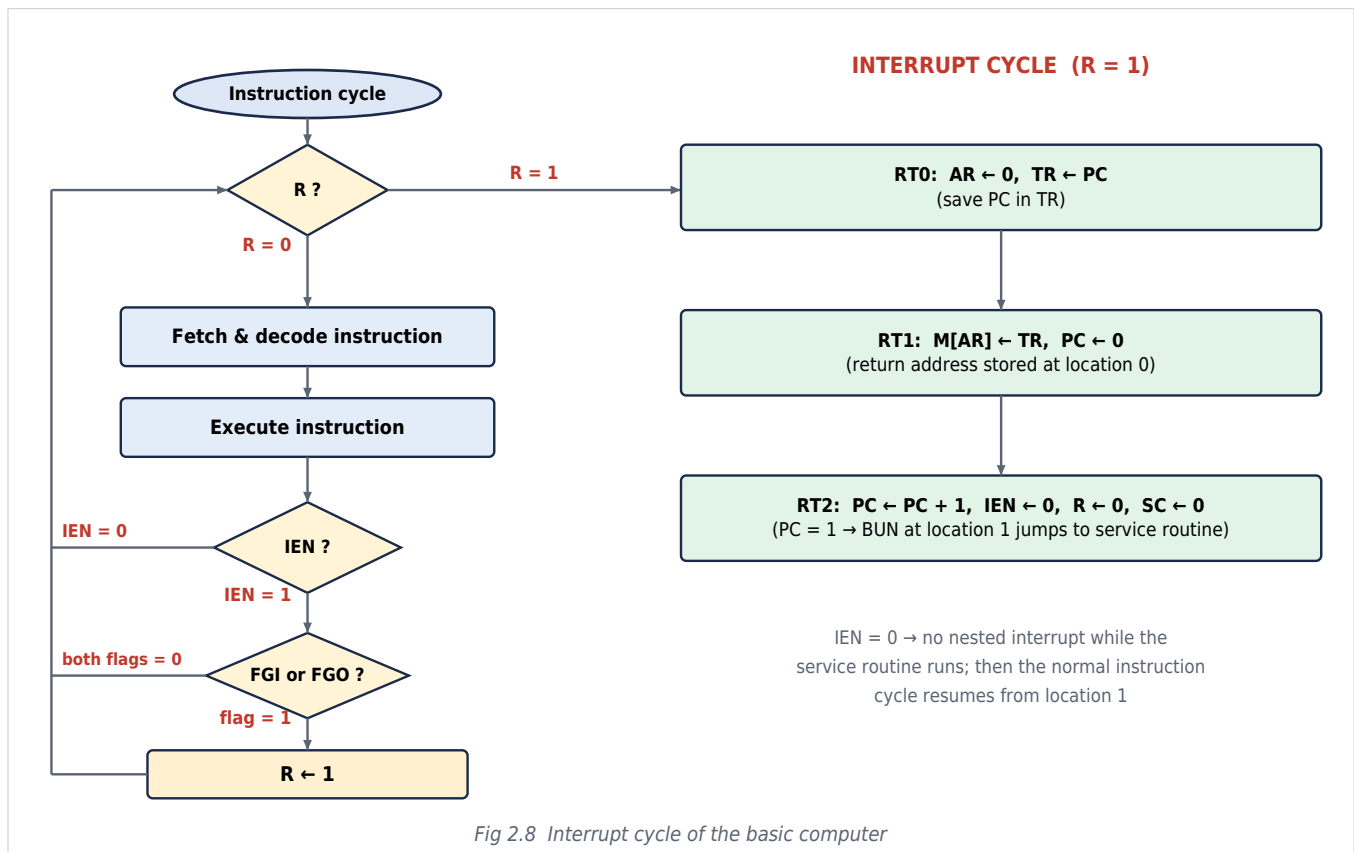
## 2.7 Input-output and interrupt

### Input-output of the basic computer

- A terminal (keyboard + printer) sends/receives 8-bit alphanumeric characters serially. **INPR** holds the input character, **OUTR** the output character. Both talk to **AC in parallel**.
- **FGI** (input flag) is 0 initially. A key press shifts the character into INPR and sets  $FGI = 1$ . The computer sees  $FGI = 1$ , moves INPR to AC (INP) and clears  $FGI$  to 0.
- **FGO** (output flag) is 1 initially (printer ready). The computer checks  $FGO = 1$ , moves AC to OUTR (OUT) and clears  $FGO = 0$ . The printer sets  $FGO = 1$  again when it finishes printing.
- The computer must never load a new character into OUTR while  $FGO = 0$  (the printer is busy).

## Interrupt cycle (Q8, Q26, Q39)

**Why an interrupt is needed:** input/output devices are very slow compared with the CPU. Without interrupts the CPU has to sit in a loop testing the flag (**programmed I/O / polling**) and wastes time. With an interrupt the CPU runs the main program and the device **tells** the CPU when it is ready.



- A flip-flop **R** (interrupt flip-flop) is added. **R = 0** → normal instruction cycle. **R = 1** → interrupt cycle.
- IEN** (interrupt enable) is set by ION and cleared by IOF. If IEN = 0 the programmer does not want interrupts and the CPU continues normally.
- During the execute phase the control checks IEN. If IEN = 1 it checks **FGI** and **FGO**. If either flag is 1, R is set to 1 (this is allowed at any time except when T0, T1, T2 are active).
- Condition to set R:  $T_0' T_1' T_2' (IEN)(FGI + FGO): R \leftarrow 1$ . Fetch/decode use **R'T0, R'T1, R'T2** so they are skipped when R = 1.

| Timing     | RTL                                                                       | Effect                                                    |
|------------|---------------------------------------------------------------------------|-----------------------------------------------------------|
| <b>RT0</b> | $AR \leftarrow 0, TR \leftarrow PC$                                       | AR cleared to 0 (address of location 0); PC saved in TR   |
| <b>RT1</b> | $M[AR] \leftarrow TR, PC \leftarrow 0$                                    | Return address stored in memory location 0; PC cleared    |
| <b>RT2</b> | $PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$ | PC = 1; further interrupts disabled; interrupt cycle ends |

**What happens to the current program:** its execution is **paused, not lost**. The address of the next instruction is stored in memory location 0 and control branches to location 1, where the programmer places a BUN to the **interrupt service routine**. That routine must first save AC and E, check which device raised the interrupt, service it, restore AC and E, turn interrupts on again (ION) and return with an **indirect branch to location 0** (**BUN 0 I**) - the main program resumes exactly where it stopped.

**DEV BRAIN** Polling = `while(!ready){}` (busy-waiting). Interrupt = **event listener / callback / webhook**: register a handler, do useful work, get called when the event fires. Location 0 is where the "stack frame" (just the PC) is saved.

## 2.8 Design of the accumulator unit

To design the logic around AC we list every statement that changes AC; each one becomes a term in the control gates for its LD, INR and CLR inputs:

| Control function | Micro-operation                              | Meaning            |
|------------------|----------------------------------------------|--------------------|
| D0T5             | $AC \leftarrow AC \wedge DR$                 | AND with DR        |
| D1T5             | $AC \leftarrow AC + DR$                      | ADD with DR        |
| D2T5             | $AC \leftarrow DR$                           | Transfer from DR   |
| pB11             | $AC(0-7) \leftarrow INPR$                    | Transfer from INPR |
| rB9              | $AC \leftarrow AC'$                          | Complement         |
| rB7              | $AC \leftarrow shr\ AC, AC(15) \leftarrow E$ | Shift right        |
| rB6              | $AC \leftarrow shl\ AC, AC(0) \leftarrow E$  | Shift left         |
| rB11             | $AC \leftarrow 0$                            | Clear              |
| rB5              | $AC \leftarrow AC + 1$                       | Increment          |

- **LD(AC)** = D0T5 + D1T5 + D2T5 + pB11 + rB9 + rB7 + rB6 (every case that loads a new value)
- **INR(AC)** = rB5 **CLR(AC)** = rB11
- The 16-bit **adder & logic circuit** takes AC, DR and INPR as inputs and feeds AC; its output also goes to the bus.

# UNIT 3 - Programming the Basic Computer

machine and assembly language, assembler, loops, subroutines, I/O programs

## 3.1 Machine language vs assembly language

A program can be written at several levels. Same job, "C = A + B":

| Level                        | What it looks like                | Notes                                                                                                                                     |
|------------------------------|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| High-level language          | C = A + B                         | Fortran, C++, Java. Written the way people think; a <b>compiler</b> translates it to machine code.                                        |
| Assembly language (symbolic) | LDA 004 / ADD 005 / STA 006 / HLT | Symbols for opcodes and addresses. One symbolic instruction $\leftrightarrow$ one binary instruction. Translated by an <b>assembler</b> . |
| Machine language (hex)       | 2004 1005 3006 7001               | Same binary code written in octal or hex.                                                                                                 |
| Machine language (binary)    | 0010 0000 0000 0100 ...           | The exact bits stored in memory; the only thing the CPU understands.                                                                      |

| Loc / Label | Instruction | What it does                           |
|-------------|-------------|----------------------------------------|
| 000         | LDA 004     | Load first operand into AC             |
| 001         | ADD 005     | Add second operand to AC               |
| 002         | STA 006     | Store sum in location 006              |
| 003         | HLT         | Halt computer                          |
| 004         | 0053        | First operand (83 decimal)             |
| 005         | FFE9        | Second operand (-23 in 2's complement) |
| 006         | 0000        | Sum goes here $\rightarrow$ 003C = 60  |

### Machine language vs assembly language (Q9) - exam table

| Point            | Machine language                     | Assembly language                                     |
|------------------|--------------------------------------|-------------------------------------------------------|
| Form             | Binary / hex numbers                 | Mnemonics and symbols (LDA, ADD, labels)              |
| Readability      | Very hard for humans                 | Easier to read, write and debug                       |
| Translator       | None - executed directly             | Needs an <b>assembler</b>                             |
| Portability      | Specific to one CPU                  | Specific to one CPU (but easier to port by rewriting) |
| Address handling | Programmer writes absolute addresses | Labels / symbolic addresses; assembler computes them  |
| Error chance     | High                                 | Lower                                                 |

## 3.2 Assembly language format and pseudo-instructions

A line has three optional parts: **label** (ends with a comma, e.g. LOP, ), **instruction**, and **comment** (after a slash). A **pseudo-instruction** is not a machine instruction; it is an order to the assembler about the translation.

| Symbol | Information for the assembler                                                     |
|--------|-----------------------------------------------------------------------------------|
| ORG N  | Hexadecimal N is the memory location of the next line (sets the location counter) |
| END    | End of the symbolic program                                                       |
| DEC N  | Signed decimal number N to be converted to binary                                 |
| HEX N  | Hexadecimal number N to be converted to binary                                    |

## 3.3 The assembler - two-pass translation

An **assembler** accepts a symbolic (**source**) program and produces its binary machine-language equivalent (**object** program). It needs **two passes** because a label can be used before it is defined.

|      | First pass                            | Second pass              |
|------|---------------------------------------|--------------------------|
| Goal | Build the <b>address-symbol table</b> | Generate the binary code |

|                                          | First pass                                                | Second pass                                                                                         |
|------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Location counter (LC)</b>             | $LC \leftarrow 0$ ; ORG sets LC; $LC + 1$ after each line | Restarted at 0; ORG sets LC again                                                                   |
| <b>On a label</b>                        | Store the label and current LC in the table               | -                                                                                                   |
| <b>On ORG / END</b>                      | ORG → set LC. END → go to second pass                     | ORG → set LC. END → done                                                                            |
| <b>On DEC / HEX</b>                      | Just increment LC                                         | Convert the operand to binary and store it at LC                                                    |
| <b>On a memory-reference instruction</b> | Just increment LC                                         | Get the opcode, look up the address in the table, set the I bit (0 direct, 1 indirect), store at LC |
| <b>On a non-MRI instruction</b>          | Just increment LC                                         | Get the opcode from the table and store it at LC                                                    |
| <b>Errors</b>                            | -                                                         | Invalid mnemonic or undefined symbol → error message                                                |

### Example: subtract two numbers (DIF = MIN – SUB)

| Loc / Label | Instruction  | What it does                                        |
|-------------|--------------|-----------------------------------------------------|
|             | ORG 100      | Program starts at hex 100                           |
| 100         | LDA SUB      | $AC \leftarrow \text{subtrahend } (-23)$            |
| 101         | CMA          | Complement AC                                       |
| 102         | INC          | $AC \leftarrow -(-23) = 23$ (2's complement negate) |
| 103         | ADD MIN      | $AC \leftarrow 23 + 83 = 106$                       |
| 104         | STA DIF      | Store result                                        |
| 105         | HLT          | Halt                                                |
| 106         | MIN, DEC 83  | Minuend                                             |
| 107         | SUB, DEC -23 | Subtrahend                                          |
| 108         | DIF, HEX 0   | Result goes here                                    |
|             | END          |                                                     |

**Address-symbol table built by the first pass:** MIN = 106, SUB = 107, DIF = 108. The second pass replaces MIN, SUB, DIF by these addresses.

## 3.4 Program loops

A **program loop** is a sequence of instructions executed many times, each time on different data. The basic computer has no loop instruction, so a loop is built with **ISZ** (counter), **BUN** (jump back) and **indirect addressing** (walk through an array with a pointer).

### Program: add 100 numbers stored from address 150 (hex)

| Loc / Label | Instruction    | What it does                                   |
|-------------|----------------|------------------------------------------------|
|             | ORG 100        | Program origin is hex 100                      |
| 100         | LDA ADS        | Load first address of operands                 |
| 101         | STA PTR        | Store in pointer                               |
| 102         | LDA NBR        | Load -100                                      |
| 103         | STA CTR        | Store in counter                               |
| 104         | CLA            | Clear accumulator (sum = 0)                    |
| 105         | LOP, ADD PTR I | Add operand pointed to by PTR (indirect!)      |
| 106         | ISZ PTR        | Increment pointer                              |
| 107         | ISZ CTR        | Increment counter; skip next when it becomes 0 |
| 108         | BUN LOP        | Repeat the loop                                |
| 109         | STA SUM        | Store the sum                                  |
| 10A         | HLT            | Halt                                           |
| 10B         | ADS, HEX 150   | First address of operands                      |
| 10C         | PTR, HEX 0     | Reserved for the pointer                       |
| 10D         | NBR, DEC -100  | Constant used to initialise the counter        |
| 10E         | CTR, HEX 0     | Reserved for the counter                       |

| Loc / Label | Instruction | What it does                  |
|-------------|-------------|-------------------------------|
| 10F         | SUM, HEX 0  | Sum is stored here            |
|             | ORG 150     | Origin of operands is hex 150 |
| 150         | DEC 75      | First operand                 |
| ...         | ...         | ...                           |
| 1B3         | DEC 23      | Last (100th) operand          |
|             | END         |                               |

The slide numbers the lines 1-19; the locations above are the real hex addresses that follow from ORG 100 (slide shows the same program).

- **Initialisation:** PTR  $\leftarrow$  150 (start of array), CTR  $\leftarrow$  -100, AC  $\leftarrow$  0.
- **Loop body:** ADD PTR I (indirect: use the content of PTR as the address), ISZ PTR (move to the next number), ISZ CTR (count up towards 0).
- **Termination:** when CTR reaches 0, ISZ skips the BUN, so control falls out to STA SUM.

### Program: clear hex locations 500 to 5FF

Same skeleton: PTR  $\leftarrow$  500, CTR  $\leftarrow$  negative count, AC  $\leftarrow$  0, then loop STA PTR I / ISZ PTR / ISZ CTR / BUN LOP (store AC = 0 at the location PTR points to). **Trap:** the slide loads -255; 500 to 5FF is 256 locations, so use DEC -256.

## 3.5 Arithmetic programs - double-precision addition

Two 32-bit numbers A and B are stored as (AH, AL) and (BH, BL). Add the low halves first; the carry ends up in E and must be added to the high halves:

| Loc / Label | Instruction | What it does                                       |
|-------------|-------------|----------------------------------------------------|
| 100         | LDA AL      | Load A low                                         |
| 101         | ADD BL      | Add B low, carry goes to E                         |
| 102         | STA CL      | Store C low                                        |
| 103         | CLA         | Clear AC                                           |
| 104         | CIL         | Circulate left: brings E (the carry) into AC bit 0 |
| 105         | ADD AH      | Add A high and carry                               |
| 106         | ADD BH      | Add B high                                         |
| 107         | STA CH      | Store C high                                       |
| 108         | HLT         | Halt                                               |

## 3.6 Subroutines

A **subroutine** is a set of common instructions (a self-contained sequence for one task) that can be used many times in a program. Each time it is needed the main program **branches to its beginning**, and after it finishes a branch is made **back** to the main program.

- **Advantage 1 - code reuse / saves memory:** write once, use many times; no need to repeat the same instructions.
- **Advantage 2 - modularity:** the program is divided into small, tested pieces; easier to write, debug, maintain and change.

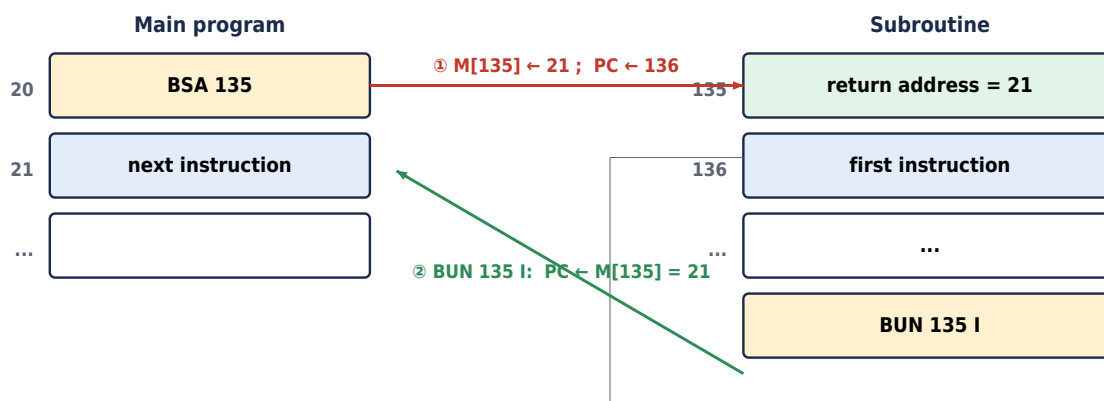


Fig 3.1 BSA (call) stores the return address; BUN ... I (return) reads it back

### How a subroutine is called and returned in the basic computer (Q10, Q27)

1. **Call** -  $BSA\ SUB: M[AR] \leftarrow PC$  (return address = address of the next instruction is stored in the **first word of the subroutine**) and  $PC \leftarrow AR + 1$  (execution starts at the second word).
2. The subroutine runs.
3. **Return** -  $BUN\ SUB\ I$ : an **indirect** branch. The address part points to the first word (which holds the return address), so  $PC \leftarrow M[SUB] =$  the saved return address. The main program continues.

### Example from the slides: SH4 shifts AC left 4 times (using X and Y)

| Loc / Label | Instruction    | What it does                                 |
|-------------|----------------|----------------------------------------------|
| 100         | LDA X          | Load X = 1234                                |
| 101         | BSA SH4        | Call subroutine                              |
| 102         | STA X          | Store result                                 |
| 103         | LDA Y          | Load Y = 4321                                |
| 104         | BSA SH4        | Call again                                   |
| 105         | STA Y          | Store result                                 |
| 106         | HLT            |                                              |
| 107         | X, HEX 1234    |                                              |
| 108         | Y, HEX 4321    |                                              |
| 109         | SH4, HEX 0     | Return address is stored here by BSA         |
| 10A – 10D   | CIL $\times 4$ | Circulate left AC and E four times           |
| 10E         | AND MSK        | Clear the 4 low bits (they hold junk from E) |
| 10F         | BUN SH4 I      | Return                                       |
| 110         | MSK, HEX FFF0  | Mask                                         |

Result:  $X = 1234 \rightarrow 2340$  and  $Y = 4321 \rightarrow 3210$  (each hex digit moves one place left). The same 6 instructions serve both calls – that is the point of a subroutine.

**TRAP** Because BSA saves the return address **inside the subroutine's own first word**, a subroutine cannot call itself (no recursion) and a second call overwrites the first return address. Real CPUs use a **stack** for this.

**DEV BRAIN** BSA = call; BUN ... I = return. The saved word is a one-slot "call stack".

## 3.7 Input-output programming

These programs **poll** the flag: SKI skips the next instruction if  $FGI = 1$ , so BUN CIF keeps looping until a character is ready.

| Input a character (keyboard → memory)                                                                                                                                                  | Output a character (memory → printer)                                                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>ORG 100 CIF, SKI / check input flag BUN CIF / flag = 0, check again INP / flag = 1, AC(0-7) ← INPR OUT / echo the character STA CHR / store it HLT CHR, HEX 0 / storage END</pre> | <pre>ORG 100 LDA CHR / load character into AC COF, SK0 / check output flag BUN COF / flag = 0, check again OUT / flag = 1, print HLT CHR, HEX 0057 / the letter "W" END</pre> |

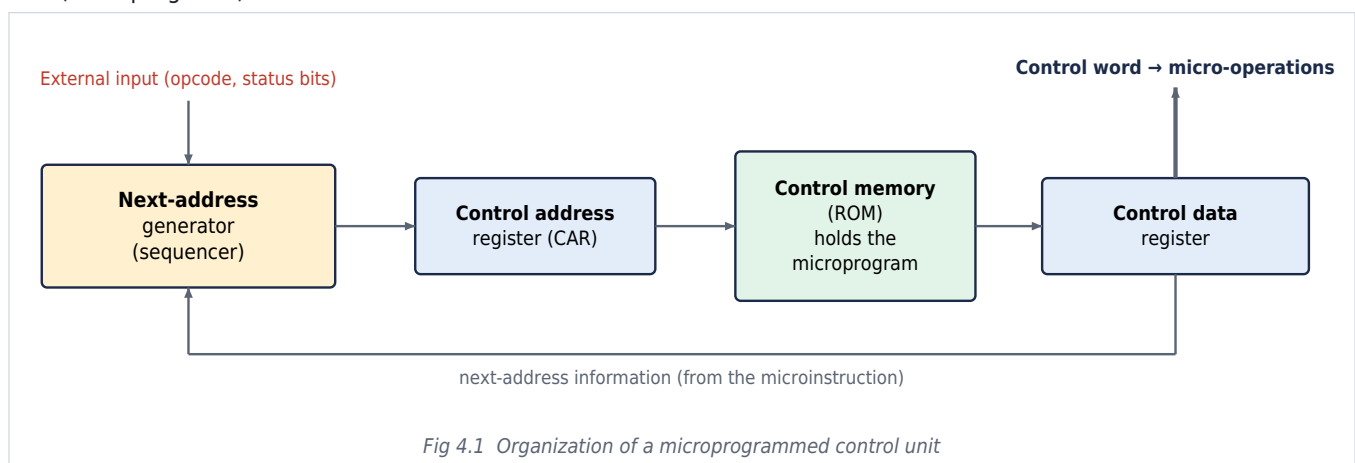
# UNIT 4 - Microprogrammed Control

control memory, address sequencing, microinstruction format

## 4.1 Control memory and the microprogrammed control unit

In a **microprogrammed** control unit the control signals are not made by gates; they are **stored as words in a special memory** (the control memory). Reading a word from this memory produces the control signals for one step.

- **Micro-operation** - one elementary register-level operation.
- **Microinstruction** - a control word stored in control memory; it specifies one or more micro-operations for the components of the computer (plus how to find the next microinstruction).
- **Microprogram** - a sequence of microinstructions (a *routine*) for one machine instruction, such as fetch or ADD.
- **Control memory** - the memory (usually a **ROM**) that holds the microprogram. It is fixed and cannot be altered by the ordinary user.
- The computer has **two memories**: the **main memory** (instructions and data) and the **control memory** (microprograms).



| Component                                 | Function                                                                                                                                      |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Control memory (ROM)</b>               | Stores the microprogram: microinstructions to fetch an instruction, evaluate the effective address, execute the operation and return to fetch |
| <b>Control address register (CAR)</b>     | Holds the <b>address of the next microinstruction</b> to be read from control memory                                                          |
| <b>Control data register</b>              | <b>Pipeline register:</b> holds the microinstruction being executed, so the next one can be read while this one executes                      |
| <b>Next-address generator (sequencer)</b> | Computes the next value of CAR from the current microinstruction and external inputs (opcode, status bits)                                    |

- **Working:** CAR gives an address → control memory outputs a microinstruction → it is held in the control data register → its **micro-operation fields** go to the datapath as control signals, and its **next-address information** goes to the sequencer, which loads CAR with the next address. The cycle repeats every clock.

### Why control memory is required (Q11) and what CAR and control memory do (Q12)

- It stores the **control words** that generate all micro-operations, so **no complicated random logic** is needed.
- It makes the design **systematic and flexible**: changing or adding an instruction only means changing the microprogram.
- **CAR** = pointer into control memory (like a program counter for microinstructions). **Control memory** = holds the microprogram and outputs the microinstruction addressed by CAR.

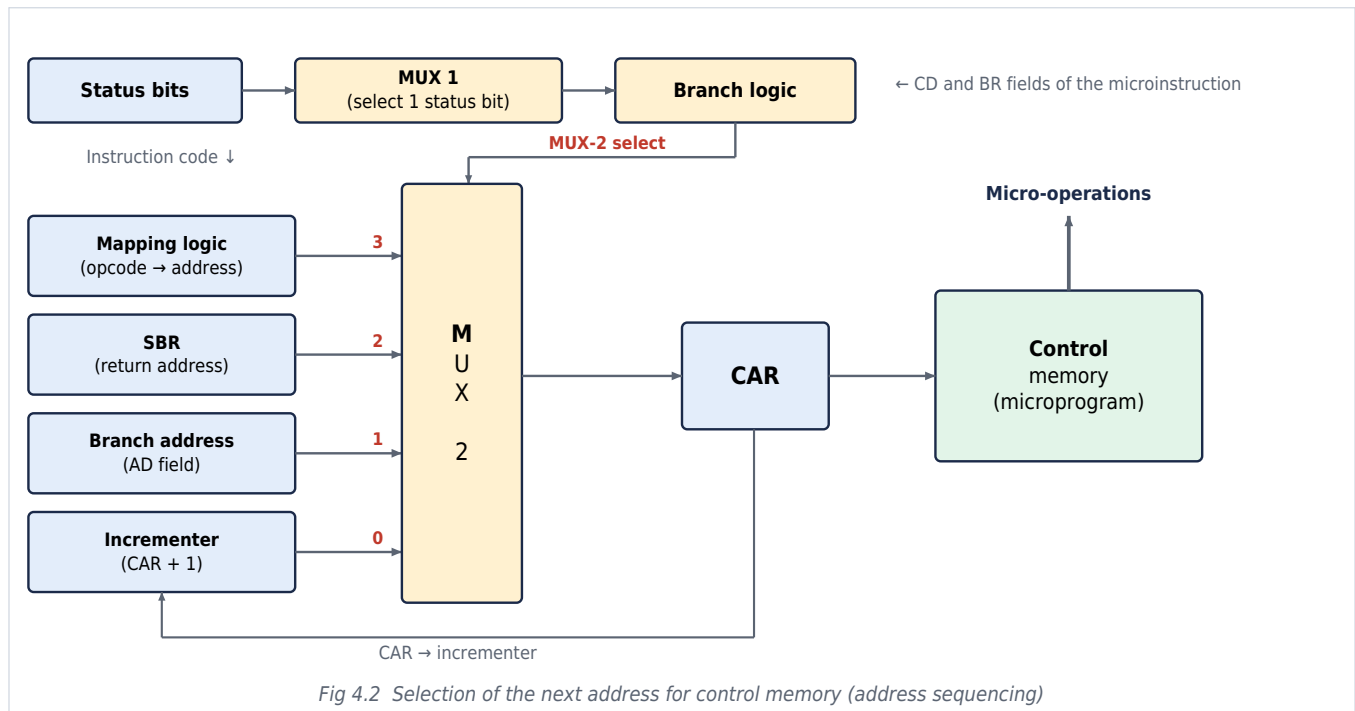
### Hardwired vs microprogrammed control (Q13)

| Point                 | Hardwired                      | Microprogrammed                       |
|-----------------------|--------------------------------|---------------------------------------|
| <b>Implementation</b> | Gates, flip-flops, decoders    | Control memory (ROM) + sequencer      |
| <b>Speed</b>          | Faster                         | Slower (memory access per step)       |
| <b>Flexibility</b>    | Hard to modify - <b>rewire</b> | Easy - <b>change the microprogram</b> |

| Point                    | Hardwired                        | Microprogrammed                                 |
|--------------------------|----------------------------------|-------------------------------------------------|
| <b>Design complexity</b> | Complex for big instruction sets | Systematic, easier for complex instruction sets |
| <b>Cost / use</b>        | Used in RISC processors          | Used in CISC processors                         |

## 4.2 Address sequencing

Microinstructions are stored in control memory in **groups**, each group = one routine (fetch, ADD, STA...). To execute the program the control must move from one microinstruction to the next, jump, and enter the right routine for each machine instruction. This is **address sequencing**.



Four capabilities are needed (this is also the answer to "methods of generating the next address", Q29 and Q41):

| Method                             | How CAR gets its next value                                                                                                                                                                          | When used                                                                 |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <b>1. Sequential increment</b>     | $CAR \leftarrow CAR + 1$                                                                                                                                                                             | Normal case: next microinstruction follows the current one                |
| <b>2. Branching</b>                | $CAR \leftarrow AD$ (address field) if the condition is true, otherwise $CAR \leftarrow CAR + 1$ . A MUX selects one <b>status bit</b> ; branch logic decides.                                       | Unconditional (U) or conditional (I, S, Z bits) jump inside a routine     |
| <b>3. Opcode mapping</b>           | Opcode bits of the machine instruction are mapped to the starting address of its routine. In the slides: <b>MAP: CAR(2-5) ← DR(11-14), CAR(0,1,6) ← 0</b> , so each routine gets a block of 4 words. | After fetch, to enter the routine for the fetched instruction             |
| <b>4. Subroutine call / return</b> | <b>CALL:</b> $CAR \leftarrow AD$ and $SBR \leftarrow CAR + 1$ . <b>RET:</b> $CAR \leftarrow SBR$ .                                                                                                   | A microroutine used by many routines (e.g. effective-address calculation) |

**Also:** at power-on / reset an **initial address** (the first fetch microinstruction) is loaded into CAR.

**DEV BRAIN** CAR is the "program counter of microcode". Increment = next line; branching = if/goto; mapping = switch(opcode) to a routine; call/return through SBR = a function call with a one-level return register.

## 4.3 Microinstruction code format

**Microinstruction = 20 bits (3 + 3 + 3 + 2 + 2 + 7)**

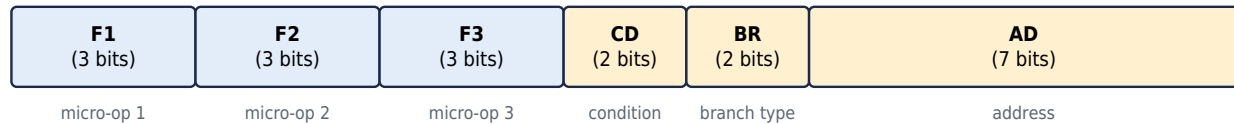


Fig 4.3 Microinstruction code format

| Field             | Bits      | Meaning                                                                                                                              |
|-------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>F1, F2, F3</b> | 3 + 3 + 3 | <b>Micro-operation fields:</b> each selects one micro-operation, so up to 3 micro-operations run in parallel in one microinstruction |
| <b>CD</b>         | 2         | <b>Condition</b> for branching: which status bit to test                                                                             |
| <b>BR</b>         | 2         | <b>Branch field:</b> type of branch (JMP, CALL, RET, MAP)                                                                            |
| <b>AD</b>         | 7         | <b>Address field:</b> branch address (control memory has 128 words)                                                                  |

| F1  | Micro-op                 | Symbol |  | F2  | Micro-op                     | Symbol |  | F3  | Micro-op                       | Symbol |
|-----|--------------------------|--------|--|-----|------------------------------|--------|--|-----|--------------------------------|--------|
| 000 | None                     | NOP    |  | 000 | None                         | NOP    |  | 000 | None                           | NOP    |
| 001 | $AC \leftarrow AC + DR$  | ADD    |  | 001 | $AC \leftarrow AC - DR$      | SUB    |  | 001 | $AC \leftarrow AC \oplus DR$   | XOR    |
| 010 | $AC \leftarrow 0$        | CLRAC  |  | 010 | $AC \leftarrow AC \vee DR$   | OR     |  | 010 | $AC \leftarrow AC'$            | COM    |
| 011 | $AC \leftarrow AC + 1$   | INCAC  |  | 011 | $AC \leftarrow AC \wedge DR$ | AND    |  | 011 | $AC \leftarrow \text{shl } AC$ | SHL    |
| 100 | $AC \leftarrow DR$       | DRTAC  |  | 100 | $DR \leftarrow M[AR]$        | READ   |  | 100 | $AC \leftarrow \text{shr } AC$ | SHR    |
| 101 | $AR \leftarrow DR(0-10)$ | DRTAR  |  | 101 | $DR \leftarrow AC$           | ACTDR  |  | 101 | $PC \leftarrow PC + 1$         | INCPC  |
| 110 | $AR \leftarrow PC$       | PCTAR  |  | 110 | $DR \leftarrow DR + 1$       | INCDR  |  | 110 | $PC \leftarrow AR$             | ARTPC  |
| 111 | $M[AR] \leftarrow DR$    | WRITE  |  | 111 | $DR(0-10) \leftarrow PC$     | PCTDR  |  | 111 | reserved                       | -      |

In the slide export a few operator symbols (OR, AND, XOR) were lost; the rows above use the standard operations for those codes.

| CD | Symbol | Condition                |  | BR | Symbol | Function                                                                                       |
|----|--------|--------------------------|--|----|--------|------------------------------------------------------------------------------------------------|
| 00 | U      | Always 1 (unconditional) |  | 00 | JMP    | $CAR \leftarrow AD$ if condition = 1, else $CAR \leftarrow CAR + 1$                            |
| 01 | I      | Indirect bit DR(15)      |  | 01 | CALL   | $CAR \leftarrow AD$ , $SBR \leftarrow CAR + 1$ if condition = 1, else $CAR \leftarrow CAR + 1$ |
| 10 | S      | Sign bit AC(15)          |  | 10 | RET    | $CAR \leftarrow SBR$ (return from subroutine)                                                  |
| 11 | Z      | $AC = 0$ (zero)          |  | 11 | MAP    | $CAR(2-5) \leftarrow DR(11-14)$ , $CAR(0,1,6) \leftarrow 0$                                    |

### Bonus: the fetch routine as microinstructions (makes everything click)

| Address | Label        | F1    | F2   | F3    | CD | BR  | AD | What happens                                                              |
|---------|--------------|-------|------|-------|----|-----|----|---------------------------------------------------------------------------|
| 64      | <b>FETCH</b> | PCTAR | -    | -     | U  | JMP | 65 | $AR \leftarrow PC$                                                        |
| 65      |              | -     | READ | INCPC | U  | JMP | 66 | $DR \leftarrow M[AR]$ , $PC \leftarrow PC + 1$ (two micro-ops together)   |
| 66      |              | DRTAR | -    | -     | U  | MAP | -  | $AR \leftarrow DR(0-10)$ ; $CAR \leftarrow$ routine address of the opcode |

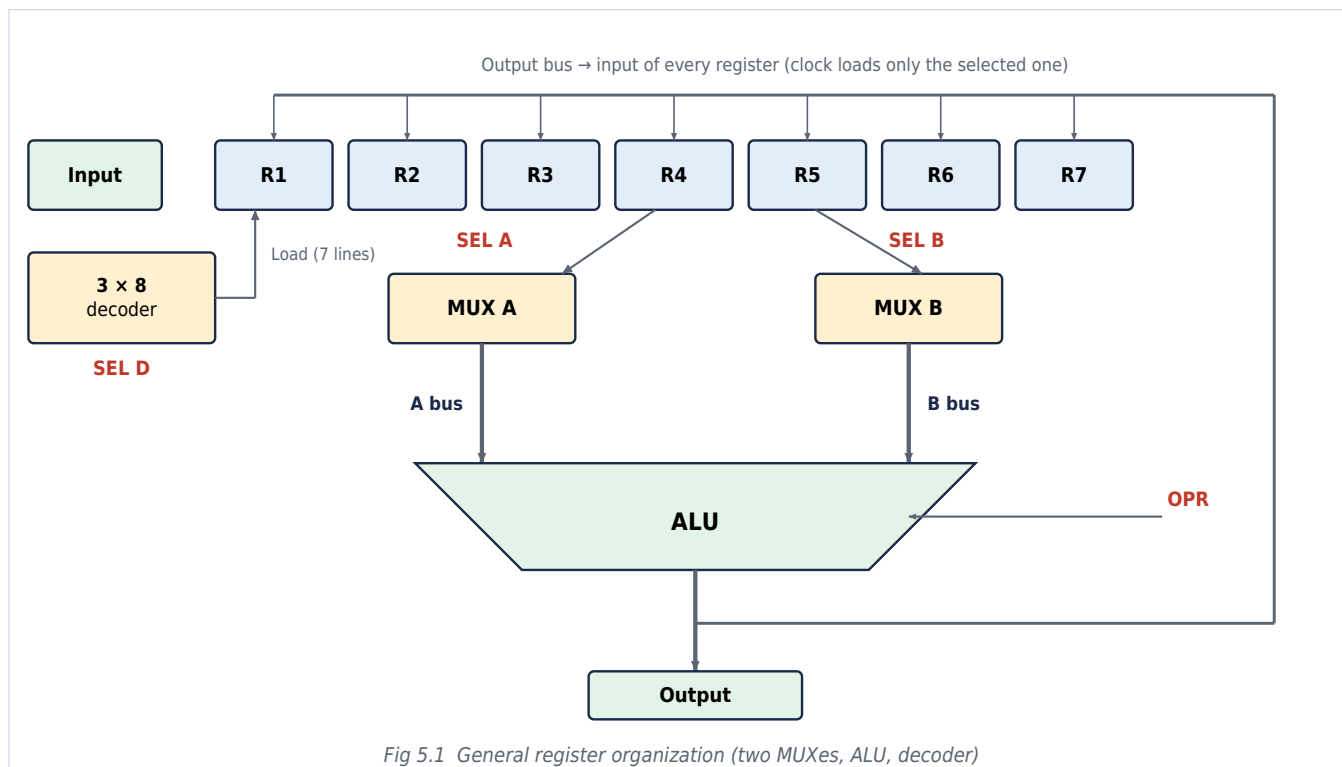
# UNIT 5 - Central Processing Unit

general registers, stack, instruction formats, addressing modes, program control, interrupts, RISC vs CISC

**This unit is 31% of the question bank.** It is also the most "list and table" unit: learn the tables and you score.

## 5.1 General register organization

A real CPU has many registers so that intermediate results stay inside the CPU instead of going to memory. In the textbook design there are **7 registers (R1-R7)** connected through **two multiplexers** to an **ALU**, and the result is loaded back through a **3×8 decoder**.



- **MUX A** and **MUX B** pick the two ALU operands (any register or the external Input) and put them on the **A bus** and **B bus**. Selected by **SEL A** and **SEL B**.
- The **ALU** performs the operation chosen by **OPR**; its result goes on the output bus, which feeds the inputs of all registers.
- The **decoder (SEL D)** enables the **Load** line of exactly one destination register. The clock loads it.
- All of it happens in **one clock cycle**: read two registers, compute, write one register.

**Control word = SELA (3) | SELB (3) | SELD (3) | OPR (5) = 14 bits**

| Binary | SELA  | SELB  | SELD |
|--------|-------|-------|------|
| 000    | Input | Input | None |
| 001    | R1    | R1    | R1   |
| 010    | R2    | R2    | R2   |
| 011    | R3    | R3    | R3   |
| 100    | R4    | R4    | R4   |
| 101    | R5    | R5    | R5   |
| 110    | R6    | R6    | R6   |
| 111    | R7    | R7    | R7   |

| OPR   | Operation   | Symbol | OPR   | Operation     | Symbol |
|-------|-------------|--------|-------|---------------|--------|
| 00000 | Transfer A  | TSFA   | 01010 | A OR B        | OR     |
| 00001 | Increment A | INCA   | 01100 | A XOR B       | XOR    |
| 00010 | A + B       | ADD    | 01110 | Complement A  | COMA   |
| 00101 | A – B       | SUB    | 10000 | Shift right A | SHRA   |
| 00110 | Decrement A | DECA   | 11000 | Shift left A  | SHLA   |
| 01000 | A AND B     | AND    |       |               |        |

### Worked example: $R1 \leftarrow R2 + R3$

| Field | Value       | Why                                                                                           |
|-------|-------------|-----------------------------------------------------------------------------------------------|
| SELA  | 010 (R2)    | MUX A puts the content of R2 on the A bus                                                     |
| SELB  | 011 (R3)    | MUX B puts the content of R3 on the B bus                                                     |
| OPR   | 00010 (ADD) | ALU computes A + B                                                                            |
| SELD  | 001 (R1)    | Decoder enables the load line of R1; the output bus value is loaded into R1 on the clock edge |

Control word: **010 011 001 00010** (SELA SELB SELD OPR). More examples:

| Micro-operation              | SELA  | SELB | SELD | OPR  | Control word      |
|------------------------------|-------|------|------|------|-------------------|
| $R1 \leftarrow R2 - R3$      | R2    | R3   | R1   | SUB  | 010 011 001 00101 |
| $R4 \leftarrow R4 \vee R5$   | R4    | R5   | R4   | OR   | 100 101 100 01010 |
| $R6 \leftarrow R6 + 1$       | R6    | -    | R6   | INCA | 110 000 110 00001 |
| $R7 \leftarrow R1$           | R1    | -    | R7   | TSFA | 001 000 111 00000 |
| $R5 \leftarrow \text{Input}$ | Input | -    | R5   | TSFA | 000 000 101 00000 |

**DEV BRAIN** SELA / SELB / SELD / OPR are just **function arguments**: `alu(opr, reg[sela], reg[selb]) → reg[seld]`. The 14-bit control word is the encoded call.

## 5.2 Stack organization

A **stack** stores information so that the item stored **last** is retrieved **first** (**LIFO**). The register that holds the address of the top item is the **stack pointer (SP)**. Only two operations: **PUSH** (insert on top) and **POP** (remove from top).

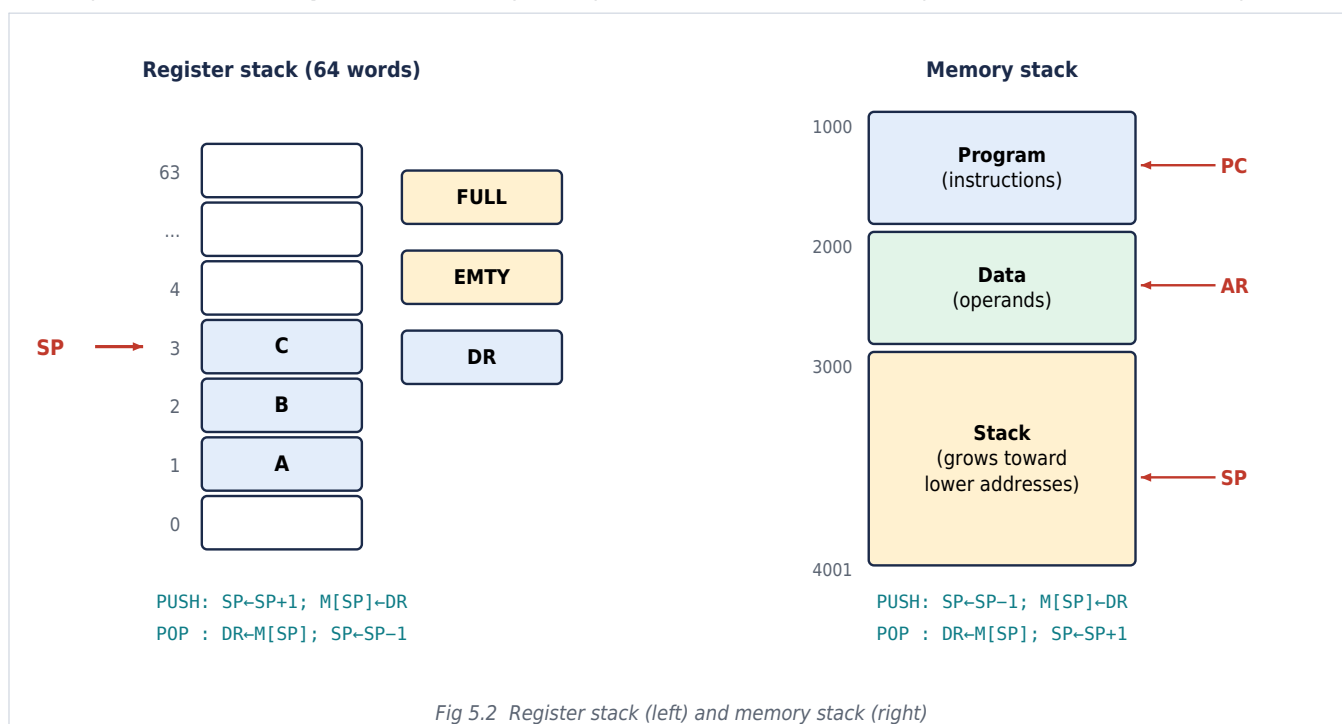


Fig 5.2 Register stack (left) and memory stack (right)

### Register stack (64 words) - hardware stack

- SP is **6 bits** ( $2^6 = 64$ ), so it can hold 0 to 63. A 1-bit **FULL** flag and a 1-bit **EMPTY** flag show stack full / empty. **DR** holds the data to push or the data popped.
- PUSH:**  $SP \leftarrow SP + 1$ ;  $M[SP] \leftarrow DR$ ; if  $(SP = 0)$  then  $FULL \leftarrow 1$ ;  $EMPTY \leftarrow 0$  (SP wraps from 63 to 0 when the stack becomes full).
- POP:**  $DR \leftarrow M[SP]$ ;  $SP \leftarrow SP - 1$ ; if  $(SP = 0)$  then  $EMPTY \leftarrow 1$ ;  $FULL \leftarrow 0$ .

### Memory stack - part of main memory used as a stack

- Memory is divided into **program** (PC points here), **data** (AR points here) and **stack** (SP points here). SP is an ordinary processor register.
- The stack usually **grows toward lower addresses** (in the slide: from 4001 downward).
- PUSH:**  $SP \leftarrow SP - 1$ ;  $M[SP] \leftarrow DR$ . **POP:**  $DR \leftarrow M[SP]$ ;  $SP \leftarrow SP + 1$ .

### Register stack vs memory stack (Q16, Q45)

| Basis          | Register stack                                                               | Memory stack                                                                                    |
|----------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Organization   | A fixed set of registers inside the CPU (e.g. 64 words) with SP, FULL, EMPTY | A portion of main memory reserved for the stack; SP is a CPU register pointing into memory      |
| Speed          | Very fast - no memory access                                                 | Slower - each push/pop is a memory access                                                       |
| Capacity       | Small and fixed by the hardware                                              | Large and flexible (limited only by the memory allotted)                                        |
| Cost           | Expensive per word (registers)                                               | Cheap per word (RAM)                                                                            |
| Implementation | Extra hardware: FULL / EMPTY flags, SP wraps; SP increments on push          | No extra storage hardware; SP decrements on push; overflow must be checked with limit registers |
| Used for       | Small hardware stacks, dedicated stack machines                              | Subroutine calls, interrupts, expression evaluation in normal CPUs                              |

### Reverse Polish Notation (RPN) and evaluating with a stack

Ordinary (infix) expressions are awkward for a computer. Łukasiewicz showed they can be written in **prefix** (operator first) or **postfix** (operator last, = Reverse Polish) form, which needs **no brackets** and is evaluated by a stack.

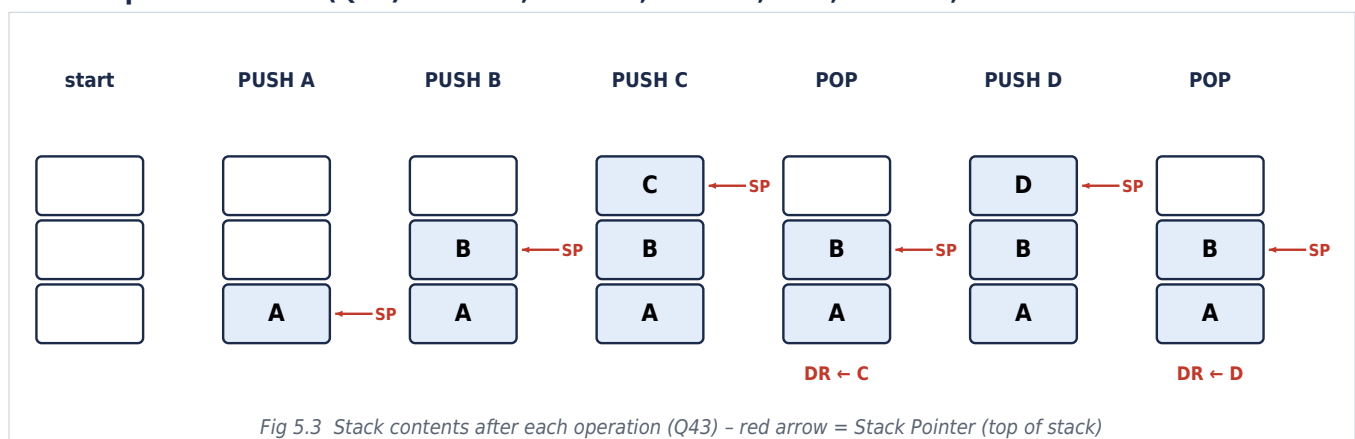
| Infix               | Prefix (Polish) | Postfix (Reverse Polish) |
|---------------------|-----------------|--------------------------|
| $A + B$             | $+ A B$         | $A B +$                  |
| $A * B + C * D$     | $+ * A B * C D$ | $A B * C D * +$          |
| $(A + B) * (C - D)$ | $* + A B - C D$ | $A B + C D - *$          |

**Rule for evaluation:** scan left to right. Operand → PUSH. Operator → POP two operands, apply, PUSH the result.

Example  $(3 \times 4) + (5 \times 6) = 3 \ 4 \ * \ 5 \ 6 \ * \ +$ :

| Symbol                     | 3 | 4   | *  | 5    | 6      | *     | +  |
|----------------------------|---|-----|----|------|--------|-------|----|
| Stack after (bottom → top) | 3 | 3 4 | 12 | 12 5 | 12 5 6 | 12 30 | 42 |

### Stack operation trace (Q43): PUSH A, PUSH B, PUSH C, POP, PUSH D, POP



| Operation | Stack (bottom → top) | SP points to  | Data register DR                |
|-----------|----------------------|---------------|---------------------------------|
| (start)   | empty                | - (EMPTY = 1) | -                               |
| PUSH A    | A                    | A             | A written                       |
| PUSH B    | A, B                 | B             | B written                       |
| PUSH C    | A, B, C              | C             | C written                       |
| POP       | A, B                 | B             | C read out                      |
| PUSH D    | A, B, D              | D             | D written (overwrites C's slot) |
| POP       | A, B                 | B             | D read out                      |

**Role of the Stack Pointer:** SP always holds the address of the **top of the stack**. PUSH moves SP to the next free slot and writes there; POP reads the top and moves SP back. A POP does **not erase** the data; it only moves SP, and the old value is overwritten by the next PUSH. SP is what enforces the LIFO order and lets the stack know when it is full or empty.

**DEV BRAIN** Stack = the **call stack** / undo history. PUSH = `arr.push(x)`, POP = `arr.pop()`, SP = `arr.length - 1`.

## 5.3 Instruction formats

Instructions are classified by the number of **address (operand) fields**. We evaluate  $X = (A + B) \times (C - D)$  in each style (the slides use  $(A + B) \times (C + D)$ ; the method is identical).

### Three-address instructions

Each address field names a register or a memory word. **Short programs**, but each instruction needs **many bits** for three addresses.

| Instruction   | Meaning                        |
|---------------|--------------------------------|
| ADD R1, A, B  | $R1 \leftarrow M[A] + M[B]$    |
| SUB R2, C, D  | $R2 \leftarrow M[C] - M[D]$    |
| MUL X, R1, R2 | $M[X] \leftarrow R1 \times R2$ |

### Two-address instructions

Most common in commercial computers. The destination is also one of the sources.

| Instruction | Meaning                      |
|-------------|------------------------------|
| MOV R1, A   | $R1 \leftarrow M[A]$         |
| ADD R1, B   | $R1 \leftarrow R1 + M[B]$    |
| MOV R2, C   | $R2 \leftarrow M[C]$         |
| SUB R2, D   | $R2 \leftarrow R2 - M[D]$    |
| MUL R1, R2  | $R1 \leftarrow R1 \times R2$ |
| MOV X, R1   | $M[X] \leftarrow R1$         |

### One-address instructions

Use an **implied accumulator (AC)** for all data manipulation. Needs a temporary memory location T.

| Instruction | Meaning                        |
|-------------|--------------------------------|
| LOAD A      | $AC \leftarrow M[A]$           |
| ADD B       | $AC \leftarrow AC + M[B]$      |
| STORE T     | $M[T] \leftarrow AC$           |
| LOAD C      | $AC \leftarrow M[C]$           |
| SUB D       | $AC \leftarrow AC - M[D]$      |
| MUL T       | $AC \leftarrow AC \times M[T]$ |
| STORE X     | $M[X] \leftarrow AC$           |

### Zero-address instructions (stack computer)

ADD, SUB, MUL use **no address field**: operands are on top of the stack. Only PUSH and POP carry an address. The expression must first be converted to RPN:  $A \ B \ + \ C \ D \ - \times$ .

| Instruction | Meaning                                         |
|-------------|-------------------------------------------------|
| PUSH A      | $TOS \leftarrow A$                              |
| PUSH B      | $TOS \leftarrow B$                              |
| ADD         | $TOS \leftarrow A + B$                          |
| PUSH C      | $TOS \leftarrow C$                              |
| PUSH D      | $TOS \leftarrow D$                              |
| SUB         | $TOS \leftarrow C - D$ (second item - top item) |
| MUL         | $TOS \leftarrow (A + B) \times (C - D)$         |
| POP X       | $M[X] \leftarrow TOS$                           |

### RISC instructions (for comparison)

Only LOAD and STORE touch memory; all arithmetic is **register to register** with three register operands.

| Instruction             | Meaning                                  |
|-------------------------|------------------------------------------|
| LOAD R1, A / LOAD R2, B | $R1 \leftarrow M[A], R2 \leftarrow M[B]$ |
| LOAD R3, C / LOAD R4, D | $R3 \leftarrow M[C], R4 \leftarrow M[D]$ |
| ADD R1, R1, R2          | $R1 \leftarrow R1 + R2$                  |
| SUB R3, R3, R4          | $R3 \leftarrow R3 - R4$                  |
| MUL R1, R1, R3          | $R1 \leftarrow R1 \times R3$             |
| STORE X, R1             | $M[X] \leftarrow R1$                     |

| Format        | Instructions for $X = (A+B) \times (C-D)$ | Address bits per instruction         | Program length               |
|---------------|-------------------------------------------|--------------------------------------|------------------------------|
| Three-address | 3                                         | Largest (3 addresses)                | Shortest                     |
| Two-address   | 6                                         | Medium                               | Short                        |
| One-address   | 7                                         | Small (1 address; AC implied)        | Longer                       |
| Zero-address  | 8 (incl. POP)                             | Smallest (address only for PUSH/POP) | Longest; needs RPN           |
| RISC          | 8                                         | Fixed length; registers only         | Long but fast (1 cycle each) |

**EXAM TIP** Trade-off to write in the exam: **more addresses per instruction = shorter program but longer instructions; fewer addresses = shorter instructions but longer program.**

## 5.4 Addressing modes

The **addressing mode** specifies a rule for interpreting or modifying the address field of an instruction before the operand is actually referenced. They exist (1) to give **programming versatility** (pointers, loop counters, indexing, program relocation) and (2) to **reduce the number of bits** in the address field.

| # | Mode                      | Where the operand is / Effective address (EA)                  | Example          | Memory refs | Why useful                          |
|---|---------------------------|----------------------------------------------------------------|------------------|-------------|-------------------------------------|
| 1 | Implied                   | Operand is implied by the opcode (AC or top of stack)          | CMA; ADD (stack) | 0           | Very short instructions             |
| 2 | Immediate                 | Operand is <b>in the instruction</b> itself                    | MOV R1, 05H      | 0           | Initialise registers with constants |
| 3 | Register                  | Operand is in a <b>CPU register</b> named in the instruction   | MOV AX, BX       | 0           | Fast; few address bits              |
| 4 | Register indirect         | EA = <b>content of a register</b>                              | MOV [R1], R2     | 1           | Pointers; short address field       |
| 5 | Autoincrement / decrement | Register indirect, then register $\pm 1$ after (or before) use | MOV (R1)+, R2    | 1           | Walk through a table / array        |
| 6 | Direct                    | EA = <b>address part</b> of the instruction                    | ADD 457          | 1           | Simple variables                    |

| #  | Mode          | Where the operand is / Effective address (EA)              | Example      | Memory refs | Why useful                                       |
|----|---------------|------------------------------------------------------------|--------------|-------------|--------------------------------------------------|
| 7  | Indirect      | EA = <b>content of the memory word</b> at the address part | ADD @300     | 2           | Pointers; jump tables                            |
| 8  | Relative      | EA = address part + <b>PC</b>                              | BR +350      | 1           | Branches; relocatable code                       |
| 9  | Indexed       | EA = address part + <b>index register</b>                  | ADD 1000(X)  | 1           | Arrays: address part = start, index = element    |
| 10 | Base register | EA = address part + <b>base register</b>                   | ADD 20(BASE) | 1           | Program relocation (address part = displacement) |

### The numericals (Q14, Q33)

- **Q14 - Indexed:** R1 = 500 (index register), displacement 250 → EA = 250 + 500 = **750**.
- **Q33 - Relative:** PC = 1200, displacement = +350 → EA = 1200 + 350 = **1550**. (Real hardware uses the PC value *after* fetch, but use the given PC as-is.)

**When relative addressing is useful:** (1) branch/jump instructions, where the target is usually near the current instruction, so a **short signed offset** is enough; (2) **position-independent (relocatable) code** - the program still works if loaded at a different memory address because only offsets are stored.

**DEV BRAIN** Immediate = literal. Direct = variable. Indirect = \*ptr. Register indirect = pointer held in a register. Indexed = a[i] (start address + index). Base register = obj.field (object base + field offset). Relative = goto +350 (offset from where you are).

## 5.5 Data transfer, manipulation and program control

**Data transfer** instructions move data without changing it (memory ↔ registers, registers ↔ I/O, register ↔ register).

**Data manipulation** instructions are of three types: arithmetic, logical / bit manipulation and shift.

| Data transfer                                                                                      | Arithmetic                                                                                                                                               | Logical and bit manipulation                                                                                                                    | Shift                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Load (LD)<br>Store (ST)<br>Move (MOV)<br>Exchange (XCH)<br>Input (IN)<br>Output (OUT)<br>Push, Pop | Increment INC<br>Decrement DEC<br>Add ADD<br>Subtract SUB<br>Multiply MUL<br>Divide DIV<br>Add with carry ADC<br>Subtract with borrow SUBB<br>Negate NEG | Clear CLR<br>Complement COM<br>AND, OR, XOR<br>Clear carry CLRC<br>Set carry SETC<br>Complement carry COMC<br>Enable / disable interrupt EI, DI | Logical shift right / left: SHR, SHL<br>Arithmetic shift right / left: SHRA, SHLA<br>Rotate right / left: ROR, ROL<br>Rotate through carry: RORC, ROLC |

### Program control instructions

They change the value of the **program counter**, which breaks the normal sequence of execution: **BUN / JMP** (branch, jump), **SKP** (skip), **CALL**, **RET**, **CMP** (compare by subtraction), **TST** (test by ANDing). Conditional branches test the **status bits** of the ALU:

| Flag     | Name     | Set to 1 when                                                        |
|----------|----------|----------------------------------------------------------------------|
| <b>C</b> | Carry    | The end carry out of the ALU is 1                                    |
| <b>S</b> | Sign     | The highest-order bit of the result is 1 (negative)                  |
| <b>Z</b> | Zero     | The result is zero                                                   |
| <b>V</b> | Overflow | XOR of the last two carries is 1 (signed overflow in 2's complement) |

### Program interrupt - types (Q8)

An interrupt is like a subroutine call with **three differences**: (1) it is started by an **internal or external signal**, not by an instruction; (2) the address of the service routine is decided by **hardware**; (3) it saves the **whole CPU state** (PC + all registers + status bits), not just the PC, so the program can resume exactly as before.

| Type                       | Source                                    | Examples                                        | Nature                                                   |
|----------------------------|-------------------------------------------|-------------------------------------------------|----------------------------------------------------------|
| <b>External (hardware)</b> | I/O devices, timers, power-supply monitor | Device ready / finished, timeout, power failure | <b>Asynchronous</b> - independent of the running program |

| Type                   | Source                                             | Examples                                                                       | Nature                                                        |
|------------------------|----------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------|
| <b>Internal (trap)</b> | Illegal or erroneous use of an instruction or data | Overflow, divide by zero, invalid opcode, stack overflow, protection violation | <b>Synchronous</b> - same place every time the program reruns |
| <b>Software</b>        | A special call instruction (supervisor call)       | System call to switch from user mode to supervisor mode for I/O                | Deliberate; behaves like an interrupt not a normal call       |

### Hardware interrupt vs software interrupt (Q8)

| Point                | Hardware interrupt                                            | Software interrupt                                                       |
|----------------------|---------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Cause</b>         | Electrical signal from an external device (I/O, timer, power) | Execution of a special instruction in the program (e.g. supervisor call) |
| <b>Timing</b>        | Asynchronous - can occur at any time                          | Synchronous - occurs exactly where the instruction is placed             |
| <b>Controlled by</b> | Hardware / device                                             | Programmer / operating system call                                       |
| <b>Purpose</b>       | Attend a device or event immediately                          | Request an OS service, switch to supervisor mode                         |
| <b>Example</b>       | Keyboard key press, timer, power failure                      | System call for I/O                                                      |

## 5.6 RISC and CISC

**RISC** (Reduced Instruction Set Computer) and **CISC** (Complex Instruction Set Computer) are two design philosophies.

| RISC characteristics                                                   | CISC characteristics                                          |
|------------------------------------------------------------------------|---------------------------------------------------------------|
| Relatively <b>few instructions</b>                                     | A <b>larger number</b> of instructions (typically 100 to 250) |
| Relatively <b>few addressing modes</b>                                 | A large variety of addressing modes (5 to 20)                 |
| Memory access <b>only by LOAD and STORE</b>                            | Instructions that <b>manipulate operands in memory</b>        |
| All operations done <b>within registers</b>                            | Some instructions do specialised tasks and are used rarely    |
| <b>Fixed-length</b> , easily decoded format                            | <b>Variable-length</b> instruction formats                    |
| <b>Single-cycle</b> execution                                          | Many instructions take several cycles                         |
| <b>Hardwired</b> control (not microprogrammed)                         | Usually <b>microprogrammed</b> control                        |
| <b>Large number of registers</b> ; overlapped register windows         | Fewer registers                                               |
| Efficient <b>instruction pipeline</b> ; compiler does much of the work | Complex decoding makes pipelining harder                      |

### RISC vs CISC - comparison for Q18 and Q34

| Basis                          | RISC                                                 | CISC                                                 |
|--------------------------------|------------------------------------------------------|------------------------------------------------------|
| <b>Instruction set</b>         | Small, simple, fixed                                 | Large, complex, powerful                             |
| <b>Instruction format</b>      | Fixed length (e.g. 32 bit), easy to decode           | Variable length, harder to decode                    |
| <b>Addressing modes</b>        | Few (simple)                                         | Many (5 - 20)                                        |
| <b>Memory access</b>           | Only LOAD / STORE; operands in registers             | Many instructions can use memory operands directly   |
| <b>Control unit</b>            | Hardwired                                            | Microprogrammed                                      |
| <b>Execution</b>               | One instruction per clock cycle (pipelined)          | Multiple cycles per instruction                      |
| <b>Registers</b>               | Large register file, register windows                | Small register set                                   |
| <b>Program size / compiler</b> | Longer programs; simple instructions, smart compiler | Shorter programs; complex hardware, simpler compiler |
| <b>Examples</b>                | ARM, MIPS, SPARC, RISC-V                             | x86 (Intel / AMD), VAX                               |

### Overlapped register windows (RISC)

Each procedure call gets a new **window** of registers from a big register file. Neighbouring windows **overlap**, so the registers common to caller and callee are used to **pass parameters without copying or saving/restoring** to memory - procedure call and return become fast.

- **L** = local registers per window, **C** = registers common to two adjacent windows, **G** = global registers, **W** = number of windows.
- Window size = **L + 2C + G**. Register file = **(L + C) × W + G**.

- Slide example:  $G = 10$ ,  $L = 10$ ,  $C = 6$ ,  $W = 4 \rightarrow \text{window} = 10 + 12 + 10 = 32$  registers; register file =  $(10 + 6) \times 4 + 10 = \mathbf{74}$  registers (R0 to R73).

# UNIT 6 - Pipeline and Vector Processing

appendix – no question from this unit in the bank; read last

## 6.1 Flynn's classification and parallel processing

**Parallel processing** = techniques that perform data-processing tasks simultaneously to increase computational speed and **throughput** (amount of processing done in a given time). Flynn classified computers by the number of **instruction streams** and **data streams**:

|                                     | Single data stream                                                                                      | Multiple data streams                                                                                                                         |
|-------------------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Single instruction stream</b>    | <b>SISD</b> – one control unit, one processor, one memory; sequential execution (ordinary uniprocessor) | <b>SIMD</b> – many processing units under one control unit; all get the <b>same instruction</b> on different data (array / vector processors) |
| <b>Multiple instruction streams</b> | <b>MISD</b> – only theoretical; no practical machine                                                    | <b>MIMD</b> – several programs at once; multiprocessors and multicomputers                                                                    |

## 6.2 Pipelining

A **pipeline** decomposes a sequential process into **sub-operations**, each executed in a dedicated **segment** that works **concurrently** with the others. Information flows through the segments; **registers between segments** isolate them. It suits tasks repeated many times on different data.



Clock drives every register R1..R4 together; registers isolate the segments

Fig 6.1 General structure of a four-segment pipeline

**Speedup** of a  $k$ -segment pipeline for  $n$  tasks (segment time  $t_p$ , non-pipelined time  $t_n = k \cdot t_p$ ):

$$S = (n \cdot t_n) / ((k + n - 1) \cdot t_p) \rightarrow k \text{ as } n \rightarrow \infty$$

Example:  $k = 4$ ,  $n = 100$ ,  $t_p = 20$  ns,  $t_n = 80$  ns  $\rightarrow S = (100 \times 80) / ((4 + 99) \times 20) = 8000 / 2060 \approx 3.88$  (close to the maximum  $k = 4$ ).

| Clock $\rightarrow$ | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  |
|---------------------|----|----|----|----|----|----|----|----|----|
| <b>Task 1</b>       | S1 | S2 | S3 | S4 |    |    |    |    |    |
| <b>Task 2</b>       |    | S1 | S2 | S3 | S4 |    |    |    |    |
| <b>Task 3</b>       |    |    | S1 | S2 | S3 | S4 |    |    |    |
| <b>Task 4</b>       |    |    |    | S1 | S2 | S3 | S4 |    |    |
| <b>Task 5</b>       |    |    |    |    | S1 | S2 | S3 | S4 |    |
| <b>Task 6</b>       |    |    |    |    |    | S1 | S2 | S3 | S4 |

Space-time diagram: 6 tasks take  $k + n - 1 = 9$  clock cycles instead of  $6 \times 4 = 24$  without pipelining.

## 6.3 Arithmetic pipeline (floating-point addition)

Four segments: (1) **compare exponents**, (2) **align mantissas**, (3) **add/subtract mantissas**, (4) **normalise the result**. Example:  $X = 0.9504 \times 10^3$ ,  $Y = 0.8200 \times 10^2$ :

1. Larger exponent (3) is chosen for the result.
2. Align:  $Y = 0.0820 \times 10^3$ .
3. Add mantissas:  $Z = 1.0324 \times 10^3$ .
4. Normalise:  $Z = 0.10324 \times 10^4$ .

## 6.4 Instruction pipeline

Six steps in general: fetch, decode, calculate effective address, fetch operands, execute, store result. Combine them into **four segments**: **FI** (fetch instruction), **DA** (decode + effective address), **FO** (fetch operand), **EX** (execute + store). Design is best when segments take equal time.

| Conflict                 | Cause                                                   | Remedies                                                                              |
|--------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Resource conflict</b> | Two segments access memory at the same time             | Separate instruction memory and data memory                                           |
| <b>Data dependency</b>   | An instruction needs a result that is not ready yet     | Hardware interlocks, operand forwarding, delayed load                                 |
| <b>Branch difficulty</b> | Branch changes PC, so prefetched instructions are wrong | Prefetch target, branch target buffer, loop buffer, branch prediction, delayed branch |

## 6.5 RISC pipeline

Fixed-length instructions and register-only operands allow a **3-segment pipeline**: **I** (instruction fetch), **A** (ALU operation), **E** (execute: write to register or memory). Data conflicts are removed by **delayed load** (compiler inserts a no-operation) and branch penalty by **delayed branch** (compiler moves useful instructions after the branch, or inserts no-ops).

## 6.6 Vector processing and array processors

- **Vector processing** handles vectors and matrices (weather forecasting, seismic analysis, genome mapping, image processing). Matrix multiplication is the classic heavy workload.
- **Memory interleaving**: memory is split into several modules, each with its own address and data registers, so several words can be accessed in parallel.
- **Array processor** = processor for large arrays. **Attached array processor**: an auxiliary processor attached to a general-purpose computer. **SIMD array processor**: many processing elements (PE1 ... PEn), each with its own local memory, under one master control unit.

# PART 2 - Model answers to all 46 questions

in bank order · written the way you should write them in the paper

Every answer below is **ready to copy in your own words**. The full explanation is in Part 1; here you get the exam version. Where a diagram is needed the answer tells you which figure to redraw (tap the blue link to jump to it).

| Marks   | How much to write              | What the examiner wants                                            |
|---------|--------------------------------|--------------------------------------------------------------------|
| 3 marks | 5 to 8 lines, or 1 small table | Definition + 2 or 3 clear points (or one example)                  |
| 5 marks | About 1 page                   | Definition, <b>neat diagram</b> , 4 to 6 points, one small example |
| 7 marks | 1.5 to 2 pages                 | All of the above + a worked example or a table + a closing line    |

**EXAM TIP** Numericals: **show the steps** even if the final answer is wrong; marks are given per step. Write results in the same number of bits as the question (8 bits stays 8 bits).

## Section A - 3-mark questions (Q1 to Q18)

**Q1**  
3 marks

**Define Register Transfer Language (RTL). Explain any two RTL statements with examples.**

**RTL** is a symbolic notation used to describe, precisely and briefly, the **micro-operations** of a digital system, i.e. which register gets which data and under which condition. A micro-operation is an elementary operation (transfer, add, shift, clear...) done on data in registers during one clock pulse. Registers are written in capital letters (AR, PC, R1) and the arrow  $\leftarrow$  means transfer (destination on the left, source on the right).

1.  $R2 \leftarrow R1$  - simple transfer: the content of R1 is copied into R2 at the next clock pulse. R1 keeps its value.
2.  $P: R2 \leftarrow R1$  - conditional transfer: the transfer takes place only if the control function  $P = 1$ . In hardware P drives the LOAD input of R2.

Other forms:  $DR \leftarrow M[AR]$  (memory read) and  $T: R2 \leftarrow R1, R1 \leftarrow R2$  (two transfers on the same clock edge = swap).

**Q2**  
3 marks

**Differentiate between register transfer, arithmetic micro-operation, and logic micro-operation with one example each.**

| Point                | Register transfer                                        | Arithmetic                                                    | Logic                                                        |
|----------------------|----------------------------------------------------------|---------------------------------------------------------------|--------------------------------------------------------------|
| <b>What it does</b>  | Copies data from one register to another; no calculation | Numeric calculation on the data (add, subtract, increment...) | Bit-by-bit Boolean operation on the data (AND, OR, XOR, NOT) |
| <b>Hardware</b>      | Bus / multiplexer + load control                         | Binary adder, adder-subtractor, counter                       | One gate per bit (no carry between bits)                     |
| <b>Data changed?</b> | No, only moved                                           | Yes, new numeric value                                        | Yes, new bit pattern                                         |
| <b>Example</b>       | $R2 \leftarrow R1$                                       | $R3 \leftarrow R1 + R2$                                       | $R3 \leftarrow R1 \wedge R2$                                 |

**Q3**  
3 marks

**A 4-bit register contains 1011. Find the result after: (a) Logical left shift (b) Logical right shift (c) Arithmetic right shift.**

| Shift                       | Rule                                                                              | Before | After       |
|-----------------------------|-----------------------------------------------------------------------------------|--------|-------------|
| <b>(a) Logical left</b>     | Every bit moves one place left, MSB (1) is lost, 0 enters the LSB                 | 1011   | <b>0110</b> |
| <b>(b) Logical right</b>    | Every bit moves one place right, LSB (1) is lost, 0 enters the MSB                | 1011   | <b>0101</b> |
| <b>(c) Arithmetic right</b> | Every bit moves right, the <b>sign bit (MSB = 1) is kept</b> and copied, LSB lost | 1011   | <b>1101</b> |

Note the difference in (b) and (c): the logical shift turned a negative number into a positive one, the arithmetic shift kept it negative ( $1011 = -5$  in 2's complement,  $1101 = -3 = -5 \div 2$  rounded down).

**Q4**

3 marks

**Perform 10110110 AND 01101101 and 10110110 XOR 01101101. Show the results.**

| Bit position   | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|----------------|---|---|---|---|---|---|---|---|
| <b>A</b>       | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 |
| <b>B</b>       | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| <b>A AND B</b> | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| <b>A XOR B</b> | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |

**AND result = 00100100** (1 only where both bits are 1). **XOR result = 11011011** (1 where the bits differ).

**Q5**

3 marks

**Explain the functions of AR, PC, IR, DR, and AC in a basic computer.**

| Register                         | Bits | Function                                                                                                      |
|----------------------------------|------|---------------------------------------------------------------------------------------------------------------|
| <b>AR - Address Register</b>     | 12   | Holds the <b>address of the memory location</b> to be read or written. Memory is always addressed through AR. |
| <b>PC - Program Counter</b>      | 12   | Holds the <b>address of the next instruction</b> ; incremented after each fetch.                              |
| <b>IR - Instruction Register</b> | 16   | Holds the <b>instruction</b> that was fetched; its opcode goes to the control unit.                           |
| <b>DR - Data Register</b>        | 16   | Holds the <b>operand read from memory</b> (or to be written); one input to the ALU.                           |
| <b>AC - Accumulator</b>          | 16   | The main <b>processing register</b> ; ALU result is stored in it.                                             |

**DRAW THIS** Draw [Fig 2.3](#) (tap to jump).

**Q6**

3 marks

**Explain the three major phases of an instruction cycle.**

1. **Fetch** - the instruction is read from memory.  $T_0: AR \leftarrow PC; T_1: IR \leftarrow M[AR], PC \leftarrow PC + 1$ .
2. **Decode** - the opcode in IR is decoded by a  $3 \times 8$  decoder; address bits go to AR and the mode bit to I.  $T_2: D_0...D_7 \leftarrow \text{decode } IR(12-14), AR \leftarrow IR(0-11), I \leftarrow IR(15)$ .
3. **Execute** - the operation is carried out. If the instruction is memory-reference and indirect ( $I = 1$ ) the effective address is first read:  $T_3: AR \leftarrow M[AR]$ . At the end  $SC \leftarrow 0$  and the cycle starts again.

**DRAW THIS** Draw [Fig 2.7](#) (tap to jump).

**Q7**

3 marks

**Explain memory-reference instructions with any three suitable examples.**

A **memory-reference instruction** has opcode 000 to 110 (bits 14-12) and uses a 12-bit address; bit 15 (I) selects direct (0) or indirect (1). It works on a memory word and the accumulator.

| Instruction | Hex (I = 0) | RTL                                                                    | Example                                                                      |
|-------------|-------------|------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>LDA</b>  | 2xxx        | $DR \leftarrow M[AR]; AC \leftarrow DR$                                | LDA 004 with $M[004] = 0053 \rightarrow AC = 0053$                           |
| <b>STA</b>  | 3xxx        | $M[AR] \leftarrow AC$                                                  | STA 006 $\rightarrow$ memory word 006 gets the content of AC                 |
| <b>ADD</b>  | 1xxx        | $DR \leftarrow M[AR]; AC \leftarrow AC + DR, E \leftarrow \text{Cout}$ | ADD 005 with $AC = 0053, M[005] = FFE9 \rightarrow AC = 003C, E = 1$ (carry) |

**Q8**

3 marks

**Differentiate between hardware interrupt and software interrupt.**

| Point                | Hardware interrupt                                                         | Software interrupt                                                  |
|----------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Cause</b>         | Electrical signal from an external device (keyboard, timer, power monitor) | Execution of a special instruction in the program (supervisor call) |
| <b>Timing</b>        | <b>Asynchronous</b> - can occur at any time                                | <b>Synchronous</b> - occurs exactly where the instruction is placed |
| <b>Controlled by</b> | Hardware / device                                                          | Programmer / operating system                                       |
| <b>Purpose</b>       | Attend an I/O device or event immediately                                  | Request an OS service, switch to supervisor mode                    |
| <b>Example</b>       | Key press, printer ready, power failure                                    | System call for input/output                                        |

**Q9**

3 marks

**Differentiate between machine language and assembly language.**

| Point               | Machine language                    | Assembly language                                       |
|---------------------|-------------------------------------|---------------------------------------------------------|
| <b>Form</b>         | Binary / hex numbers (e.g. 2004)    | Mnemonics and labels (e.g. LDA SUB)                     |
| <b>Readability</b>  | Very hard for humans                | Easy to read, write and debug                           |
| <b>Translator</b>   | None - runs directly on the CPU     | Needs an <b>assembler</b> to convert it to machine code |
| <b>Addresses</b>    | Programmer gives absolute addresses | Symbolic labels; the assembler finds the addresses      |
| <b>Error chance</b> | High                                | Lower                                                   |

**Q10**

3 marks

**What is a subroutine? Explain its two major advantages in assembly language programming.**

A **subroutine** is a self-contained sequence of instructions that does one task and can be used many times in a program. The main program branches to its beginning (**BSA**) and, when it finishes, control returns to the instruction after the call (**BUN SUB I**).

1. **Reuse / saves memory** - the code is written once and called from many places instead of being repeated each time.
2. **Modularity** - a large program is broken into small tested pieces, so it is easier to write, debug, maintain and modify.

**Q11**

3 marks

**What is control memory? Why is it required in a microprogrammed control unit?**

**Control memory** is the (usually read-only) memory inside the control unit that stores the **microprogram**: the control words (microinstructions) that generate all micro-operations of the computer.

**Why it is required:**

- Control signals are **read from memory instead of being produced by complicated random logic**, so the design is systematic and simple.
- To change or add an instruction only the microprogram is changed, no rewiring, so the design is **flexible**.
- It keeps the microprograms for fetch, effective-address calculation and every instruction routine in one place, addressed by the control address register.

**Q12**

3 marks

**Explain the functions of CAR and control memory in a microprogrammed control unit.**

| Component                             | Function                                                                                                                                                                                      |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CAR (Control Address Register)</b> | Holds the <b>address of the next microinstruction</b> to be read. It is loaded by the next-address generator (increment, branch, map, return). It is the program counter of the microprogram. |
| <b>Control memory (ROM)</b>           | Stores the <b>microprogram</b> . For the address in CAR it outputs one microinstruction, whose fields give the control signals for the datapath and the information for the next address.     |

**DRAW THIS** Draw [Fig 4.1](#) (tap to jump).**Q13**

3 marks

**Differentiate between hardwired and microprogrammed control in any three points.**

| Point                 | Hardwired control                           | Microprogrammed control                          |
|-----------------------|---------------------------------------------|--------------------------------------------------|
| <b>Implementation</b> | Gates, flip-flops, decoders (fixed circuit) | Control memory (ROM) + sequencer                 |
| <b>Speed</b>          | Faster                                      | Slower (a memory read per microinstruction)      |
| <b>Flexibility</b>    | Hard to change - must <b>rewire</b>         | Easy to change - <b>rewrite the microprogram</b> |
| <b>Used in</b>        | RISC processors                             | CISC processors                                  |

**Q14**

3 marks

**Define effective address. Find the effective address if R1 = 500 and the displacement is 250 in indexed addressing.**

**Effective address (EA)** is the address in memory where the operand actually resides, i.e. the address obtained after the address field of the instruction has been interpreted or modified by the addressing mode.

**Indexed addressing:** EA = address part (displacement) + content of the index register.

EA = 250 + R1 = 250 + 500 = **750**. The operand is at memory location 750.

**Q15**

3 marks

**Explain immediate, direct and indirect addressing modes with examples.**

Take an instruction whose address field is **300**, with  $M[300] = 1350$  and  $M[1350] = 45$ .

| Mode             | Meaning                                                          | Effective address | Operand         | Memory accesses |
|------------------|------------------------------------------------------------------|-------------------|-----------------|-----------------|
| <b>Immediate</b> | The address field <b>is</b> the operand                          | none              | 300             | 0               |
| <b>Direct</b>    | Address field = address of the operand                           | 300               | $M[300] = 1350$ | 1               |
| <b>Indirect</b>  | Address field = address of a word that holds the operand address | $M[300] = 1350$   | $M[1350] = 45$  | 2               |

**Q16**

3 marks

**Differentiate between register stack and memory stack.**

| Point           | Register stack                          | Memory stack                                               |
|-----------------|-----------------------------------------|------------------------------------------------------------|
| <b>Where</b>    | A fixed set of registers inside the CPU | A part of main memory reserved for the stack               |
| <b>Speed</b>    | Very fast (no memory access)            | Slower (memory access per push/pop)                        |
| <b>Capacity</b> | Small, fixed (e.g. 64 words)            | Large, flexible                                            |
| <b>Cost</b>     | Expensive per word                      | Cheap per word                                             |
| <b>Pointer</b>  | SP plus FULL and EMPT flags             | SP is a CPU register pointing into memory (grows downward) |

**DRAW THIS** Draw [Fig 5.2](#) (tap to jump).

**Q17**

3 marks

**Explain zero-address and one-address instruction formats with examples.**

|                                       | Zero-address (stack)                             | One-address (accumulator)                            |
|---------------------------------------|--------------------------------------------------|------------------------------------------------------|
| <b>Format</b>                         | Opcode only (PUSH/POP carry one address)         | Opcode   one address                                 |
| <b>Operands</b>                       | Implied: <b>top of stack</b>                     | One in memory, the other is the <b>implied AC</b>    |
| <b>Needs</b>                          | Expression in postfix (RPN)                      | A temporary memory location for intermediate results |
| <b>Example <math>X = A + B</math></b> | PUSH A; PUSH B; ADD; POP X                       | LOAD A; ADD B; STORE X                               |
| <b>Meaning</b>                        | ADD: $TOS \leftarrow \text{top two items added}$ | ADD B: $AC \leftarrow AC + M[B]$                     |

**Q18**

3 marks

**State any three important differences between RISC and CISC architectures.**

| Basis                               | RISC                                                     | CISC                                               |
|-------------------------------------|----------------------------------------------------------|----------------------------------------------------|
| <b>Instruction set</b>              | Small, simple instructions                               | Large, complex instructions (100 to 250)           |
| <b>Memory access</b>                | Only LOAD and STORE; all operations register to register | Many instructions work directly on memory operands |
| <b>Instruction format / control</b> | Fixed length, hardwired control                          | Variable length, microprogrammed control           |
| <b>Execution</b>                    | One instruction per clock cycle (pipelined)              | Several cycles per instruction                     |

## Section B - 5-mark questions (Q19 to Q34)

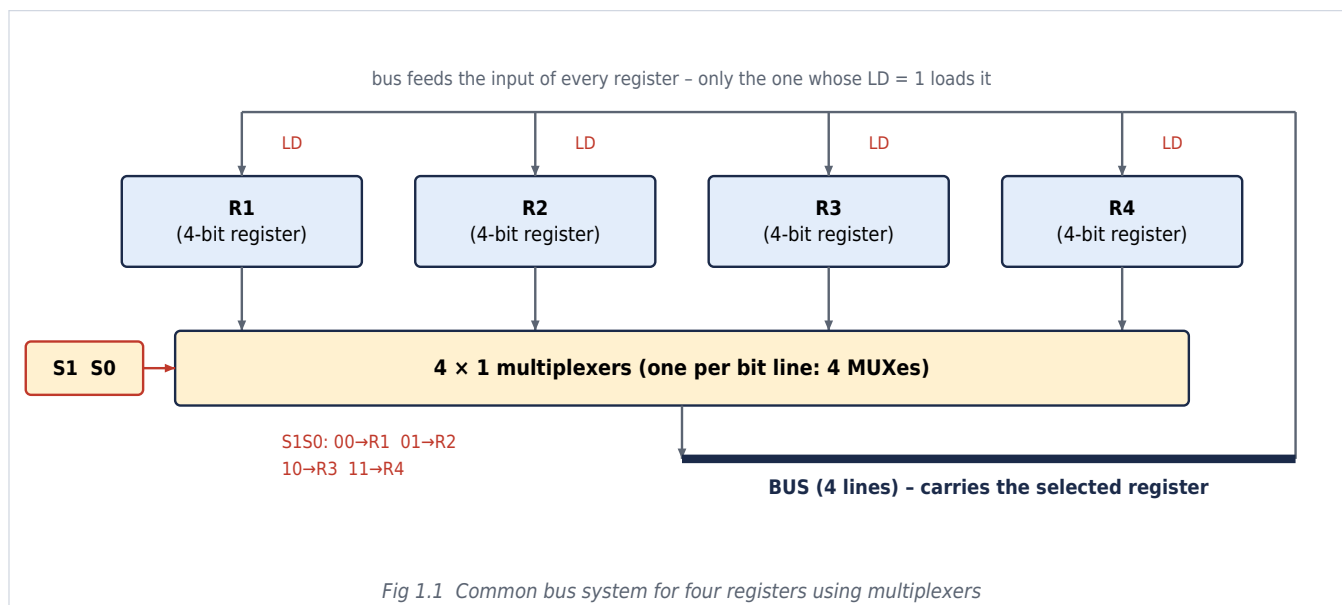
**Q19**

5 marks

**Explain Register Transfer Language and common bus system with a suitable diagram and RTL examples.**

**RTL** is the symbolic notation for micro-operations: capital letters name registers,  $\leftarrow$  is transfer, a colon separates a control condition from the operation ( $P: R2 \leftarrow R1$ ), a comma joins operations that happen on the same clock edge,  $[$  ] selects a memory word ( $DR \leftarrow M[AR]$ ).

**Common bus system:** instead of wiring every register to every other, all registers share one set of lines called the **bus**. Only one register may put data on the bus at a time. A **multiplexer** per bit line (4×1 for four registers) selects the source register using the select lines S1 S0. The destination register loads the bus value when its **LD** input = 1 at the clock edge.



| RTL                | Select S1 S0 | Load         | Steps                                               |
|--------------------|--------------|--------------|-----------------------------------------------------|
| $R1 \leftarrow R2$ | 01           | $LD(R1) = 1$ | $BUS \leftarrow R2$ (MUX), then $R1 \leftarrow BUS$ |
| $R3 \leftarrow R1$ | 00           | $LD(R3) = 1$ | $BUS \leftarrow R1$ , then $R3 \leftarrow BUS$      |
| $R4 \leftarrow R3$ | 10           | $LD(R4) = 1$ | $BUS \leftarrow R3$ , then $R4 \leftarrow BUS$      |

- n registers need  $\log_2 n$  select lines; k-bit registers need **k multiplexers**.
- Alternative to multiplexers: **tri-state buffers** with a decoder enabling exactly one buffer.

**Q20**

5 marks

**Explain arithmetic, logic and shift micro-operations with suitable examples. Differentiate between logical and arithmetic right shifts.**

### Arithmetic micro-operations

| RTL                             | Operation             | Note                                  |
|---------------------------------|-----------------------|---------------------------------------|
| $R3 \leftarrow R1 + R2$         | Add                   | Binary adder (full adders in a chain) |
| $R3 \leftarrow R1 - R2$         | Subtract              | $R1 + R2' + 1$ (2's complement)       |
| $R2 \leftarrow R2' + 1$         | Negate                | Invert all bits, add 1                |
| $R1 \leftarrow R1 + 1 / R1 - 1$ | Increment / decrement | Up / down counter                     |

### Logic micro-operations (A = 1100, B = 1010)

| Operation | RTL                          | Result | Use                  |
|-----------|------------------------------|--------|----------------------|
| AND       | $R3 \leftarrow R1 \wedge R2$ | 1000   | Masking, clear bits  |
| OR        | $R3 \leftarrow R1 \vee R2$   | 1110   | Selective set        |
| XOR       | $R3 \leftarrow R1 \oplus R2$ | 0110   | Selective complement |
| NOT       | $R1 \leftarrow R1'$          | 0011   | Complement           |

**Shift micro-operations (register 1011)**

| Shift                          | Rule                                                     | Result          |
|--------------------------------|----------------------------------------------------------|-----------------|
| Logical left / right           | 0 enters the empty end; the bit at the other end is lost | 0110 / 0101     |
| Circular left / right (rotate) | The bit shifted out re-enters at the other end           | 0111 / 1101     |
| Arithmetic right               | Sign bit is preserved and copied                         | 1101            |
| Arithmetic left                | 0 enters LSB; overflow if the sign bit changes           | 0110 (overflow) |

**Logical vs arithmetic right shift**

|                      | Logical right shift             | Arithmetic right shift        |
|----------------------|---------------------------------|-------------------------------|
| Bit entering the MSB | Always 0                        | Copy of the sign bit          |
| Used for             | Unsigned numbers / bit patterns | Signed 2's complement numbers |
| Effect               | Unsigned $\div 2$               | Signed $\div 2$ , sign kept   |
| 1011 $\rightarrow$   | 0101                            | 1101                          |

**Q21**

5 marks

Given R1 = 10110101 and R2 = 00111011, perform: (a) R1 + R2 (b) R1 - R2 using 2's complement (c) R1 AND R2 (d) R1 XOR R2.

**(a) R1 + R2**

```

R1  = 1011 0101   (181)
R2  = 0011 1011   ( 59)
-----
Sum = 1111 0000   (240)    carry out = 0

```

**(b) R1 - R2 using 2's complement**

```

R2      = 0011 1011
1's compl. = 1100 0100      (invert every bit)
2's compl. = 1100 0101      (add 1) = -R2

R1      = 1011 0101
+ (-R2) = 1100 0101
-----
carry 1 | 0111 1010      discard the carry
R1 - R2 = 0111 1010   (122) (carry 1 = no borrow, result positive)

```

**(c) R1 AND R2 and (d) R1 XOR R2**

| Bit       | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | Result           |
|-----------|---|---|---|---|---|---|---|---|------------------|
| R1        | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |                  |
| R2        | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |                  |
| R1 AND R2 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | <b>0011 0001</b> |
| R1 XOR R2 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | <b>1000 1110</b> |

**EXAM TIP**

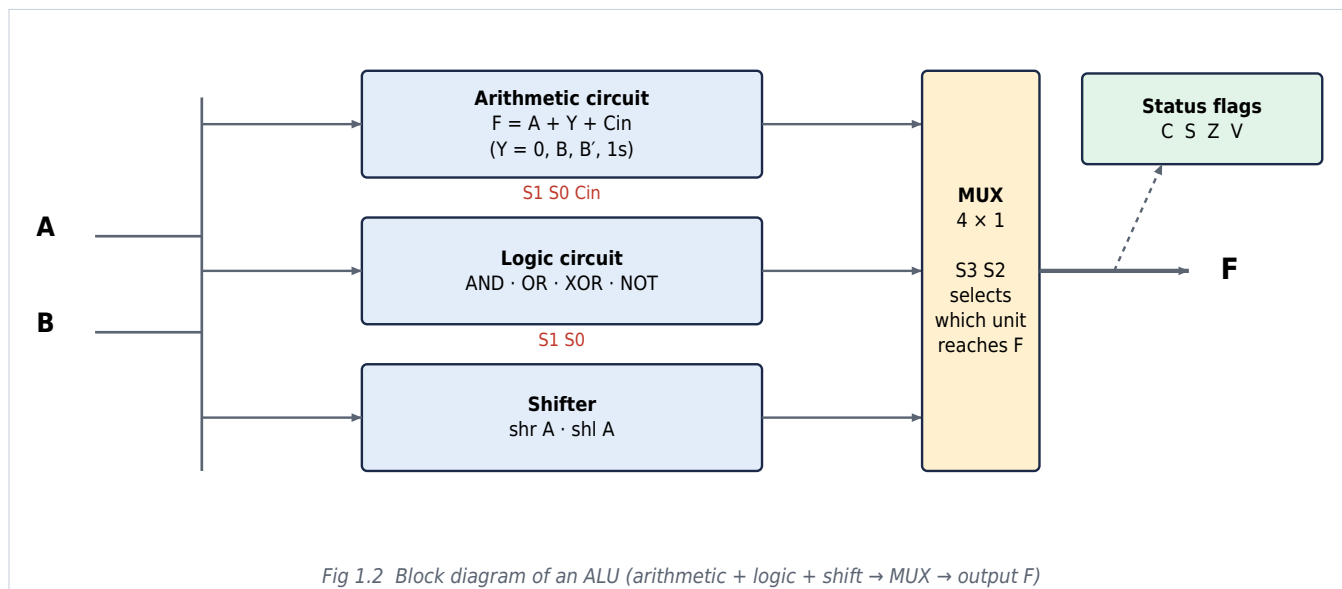
Answers: (a) **11110000** (b) **01111010** (c) **00110001** (d) **10001110**. Extra mark: if R1 and R2 are read as signed numbers, R1 = -75, and (b) gives  $-75 - 59 = -134$ , which does not fit in 8 bits - signed overflow ( $V = 1$ ). Read as unsigned,  $181 - 59 = 122$  is correct.

**Q22**

5 marks

**Explain the organization and working of an Arithmetic Logic Unit (ALU) with a neat block diagram.**

The **ALU** is the combinational circuit of the CPU that performs arithmetic, logic and shift micro-operations on operands (from AC and DR) and produces a result and status flags. The operation is selected by control (select) lines; the result returns to a register.



- **Arithmetic unit** – a 4-bit parallel adder. A multiplexer on the B input picks  $Y = 0, B, B', 1s$ ; output  $F = A + Y + C_{in}$ . This gives transfer, increment, add, subtract, decrement.
- **Logic unit** – one AND, OR, XOR and NOT gate per bit, chosen by a  $4 \times 1$  MUX.
- **Shifter** – shr A or shl A.
- **Output MUX (S3 S2)** chooses which unit reaches F: 00 arithmetic, 01 logic, 10 shift right, 11 shift left.
- **Status flags:** C (carry out), S (sign bit), Z (all bits zero), V (overflow = carry into MSB XOR carry out of MSB).

| S3 S2   | S1 S0 | $C_{in}$ | Function                    |
|---------|-------|----------|-----------------------------|
| 00      | 01    | 0        | $F = A + B$ (add)           |
| 00      | 10    | 1        | $F = A + B' + 1$ (subtract) |
| 00      | 00    | 1        | $F = A + 1$ (increment)     |
| 01      | 00    | x        | $F = A \wedge B$ (AND)      |
| 01      | 10    | x        | $F = A \oplus B$ (XOR)      |
| 10 / 11 | x x   | x        | shr A / shl A               |

**Q23**

5 marks

**Explain the complete fetch and decode cycle of a basic computer**

Every instruction begins with **fetch** and **decode**. They are driven by the sequence counter SC (timing signals T0, T1, T2) and only run when the interrupt flip-flop R = 0 (RT0, RT1, RT2).

| Timing | RTL                                                                                            | What happens (bus and control)                                                                                                           |
|--------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| T0     | $AR \leftarrow PC$                                                                             | PC drives the bus ( $S2S1S0 = 010$ ) and AR loads it. AR holds the address of the instruction.                                           |
| T1     | $IR \leftarrow M[AR], PC \leftarrow PC + 1$                                                    | Memory is read ( $S2S1S0 = 111$ ), IR loads the instruction. In the same pulse PC is incremented and now points to the next instruction. |
| T2     | $D0...D7 \leftarrow \text{decode } IR(12-14)$<br>$AR \leftarrow IR(0-11), I \leftarrow IR(15)$ | The $3 \times 8$ decoder decodes the opcode into D0 to D7. The 12 address bits are copied to AR and the mode bit to the I flip-flop.     |

**DRAW THIS** Draw Fig 2.7, Fig 2.6 (tap to jump).

**After decode (T3)** the path depends on D7 and I:

| D7 | I | Type                       | Action at T3                                       |
|----|---|----------------------------|----------------------------------------------------|
| 0  | 1 | Memory-reference, indirect | $AR \leftarrow M[AR]$ (read the effective address) |
| 0  | 0 | Memory-reference, direct   | Nothing; AR already holds the effective address    |
| 1  | 0 | Register-reference         | Executed now ( $rB_i$ ), then $SC \leftarrow 0$    |
| 1  | 1 | Input-output               | Executed now ( $pB_i$ ), then $SC \leftarrow 0$    |

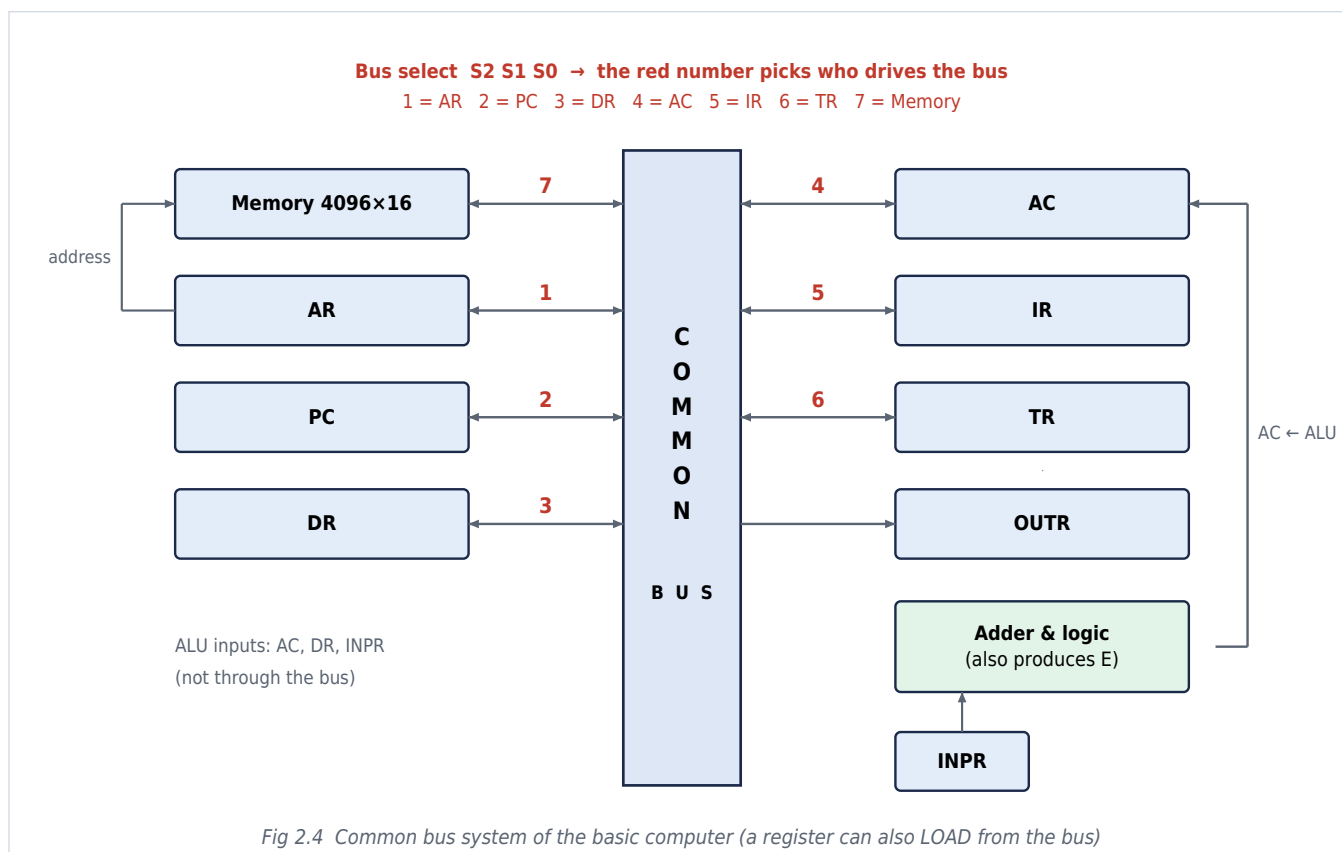
For memory-reference instructions execution continues at T4 onward ( $D_iT4, D_iT5...$ ). At the end  $SC \leftarrow 0$  so T0 comes next and the next instruction is fetched.

**Q24**

5 marks

**Explain the functions of the major registers in a basic computer and describe how they participate during common bus system.**

| Register    | Bits    | Function                                                          | Bus select |
|-------------|---------|-------------------------------------------------------------------|------------|
| AR          | 12      | Address for memory (memory address always comes directly from AR) | 001        |
| PC          | 12      | Address of the next instruction                                   | 010        |
| DR          | 16      | Operand read from / to be written to memory; ALU input            | 011        |
| AC          | 16      | Accumulator, main processing register                             | 100        |
| IR          | 16      | Instruction just fetched                                          | 101        |
| TR          | 16      | Temporary data (e.g. saves PC in an interrupt)                    | 110        |
| Memory      | 4096×16 | Program and data                                                  | 111        |
| INPR / OUTR | 8       | Input / output character; talk to AC                              | not on bus |



**How they participate:** the three select lines S2 S1 S0 choose **who drives the 16-bit bus** (code in the last column). Any register can **load** from the bus when its LD input is 1, can **increment** with INR and **clear** with CLR. AC gets its input from the adder-and-logic circuit (inputs AC, DR, INPR), not directly from the bus.

| Micro-operation | On the bus       | Control      |
|-----------------|------------------|--------------|
| AR ← PC         | PC (S = 010)     | LD(AR)       |
| IR ← M[AR]      | Memory (S = 111) | Read, LD(IR) |
| PC ← PC + 1     | -                | INR(PC)      |
| M[AR] ← AC      | AC (S = 100)     | Write        |

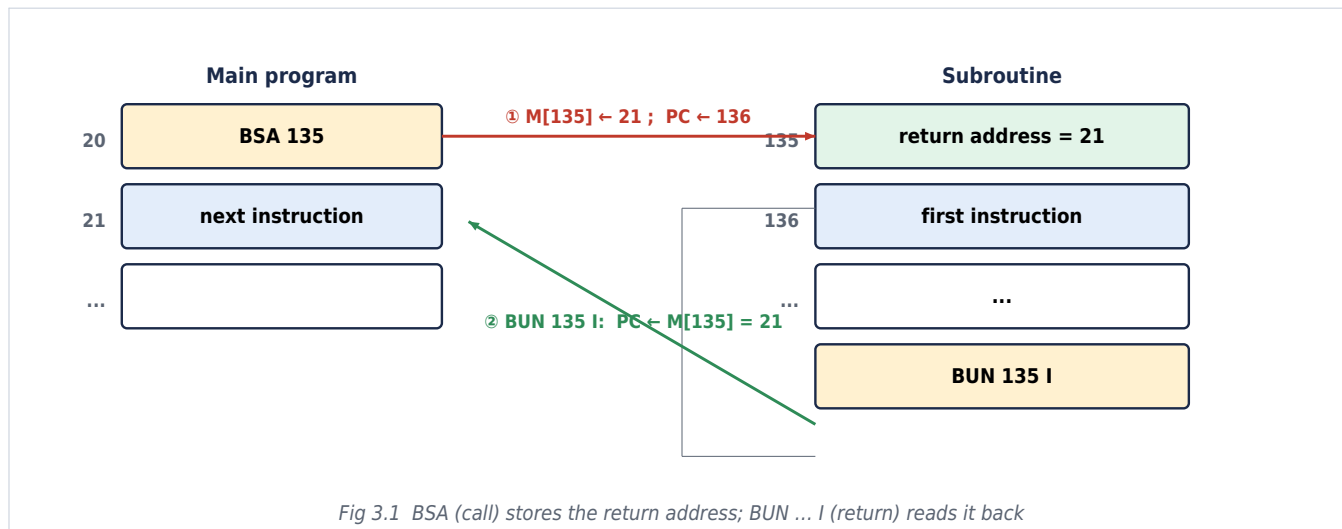
**Q25**

5 marks

**Explain the following memory-reference instructions with suitable examples: BSA, NBR, LDA, STA, BUN.**

| Instr. | Hex (I = 0) | RTL (execution)                           | Meaning and example                                                          |
|--------|-------------|-------------------------------------------|------------------------------------------------------------------------------|
| LDA    | 2xxx        | D2T4: DR ← M[AR]<br>D2T5: AC ← DR, SC ← 0 | Load memory word into AC. <b>LDA 004</b> with M[004] = 0053 gives AC = 0053. |

| Instr.     | Hex (I = 0) | RTL (execution)                                                                                | Meaning and example                                                                                                                                                                                     |
|------------|-------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>STA</b> | 3xxx        | D3T4: $M[AR] \leftarrow AC, SC \leftarrow 0$                                                   | Store AC in memory. <b>STA 006</b> writes the content of AC to location 006.                                                                                                                            |
| <b>BUN</b> | 4xxx        | D4T4: $PC \leftarrow AR, SC \leftarrow 0$                                                      | Branch unconditionally (jump). <b>BUN 010</b> : next instruction is fetched from 010.                                                                                                                   |
| <b>BSA</b> | 5xxx        | D5T4: $M[AR] \leftarrow PC, AR \leftarrow AR + 1$<br>D5T5: $PC \leftarrow AR, SC \leftarrow 0$ | Branch and Save return Address: stores the return address in the first word of the subroutine and starts at the next word. <b>BSA 020</b> at location 100: $M[020] \leftarrow 101, PC \leftarrow 021$ . |



**Return:** BUN 020 I (indirect) gives  $PC \leftarrow M[020] = 101$ , so the main program continues after the call.

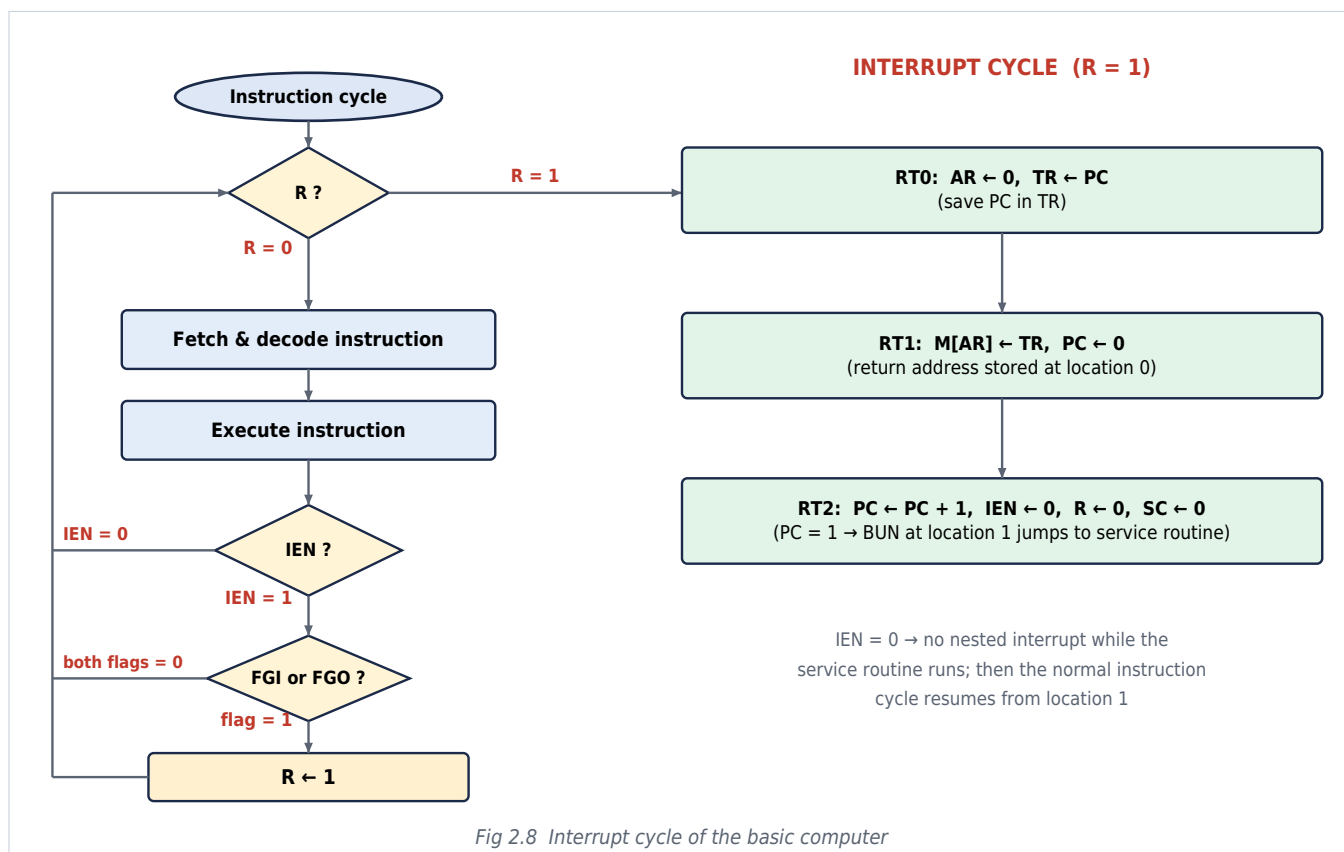
**EXAM TIP** **NBR** is not an instruction of the basic computer. In the “add 100 numbers” program it is a **label** for a memory word holding the constant  $-100$  (NBR, DEC  $-100 = FF9C$ ). It is loaded into the counter (LDA NBR; STA CTR) so that ISZ CTR reaches 0 after 100 passes. If your paper means **ISZ** (Increment and Skip if Zero):  $M[AR] \leftarrow M[AR] + 1$ ; if the result = 0 then  $PC \leftarrow PC + 1$ .

**Q26**

5 marks

**Explain the interrupt cycle of a basic computer. Why is an interrupt mechanism necessary?**

**Why necessary:** I/O devices are much slower than the CPU. Without interrupts the CPU must keep testing the flags (programmed I/O / polling) and wastes time. With interrupts the CPU runs the program and the device **signals when it is ready**; the CPU serves it and returns.



- A flip-flop **R** is added: **R = 0** normal instruction cycle, **R = 1** interrupt cycle. **IEN** (set by ION, cleared by IOF) tells whether interrupts are allowed.
- At the end of an instruction, if **IEN = 1** and (**FGI = 1** or **FGO = 1**), **R** is set to 1.  $T_0' T_1' T_2' (IEN) (FGI + FGO) : R \leftarrow 1$

| Timing     | RTL                                                                       | Effect                                                            |
|------------|---------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>RT0</b> | $AR \leftarrow 0, TR \leftarrow PC$                                       | $AR = 0$ ; return address saved in TR                             |
| <b>RT1</b> | $M[AR] \leftarrow TR, PC \leftarrow 0$                                    | Return address stored in location 0                               |
| <b>RT2</b> | $PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$ | $PC = 1$ (service routine entry); interrupts disabled; cycle ends |

Location 1 contains a **BUN** to the interrupt service routine. The routine saves AC and E, finds which flag is set, services the device, restores AC and E, executes **ION** and returns with **BUN 0 I**.

**Q27**

5 marks

**Explain the concept of subroutine and describe how a subroutine can be called and returned from.**

A **subroutine** is a set of common instructions that does one task and can be used many times. The main program branches to its start and, when done, control goes back to the instruction after the call.

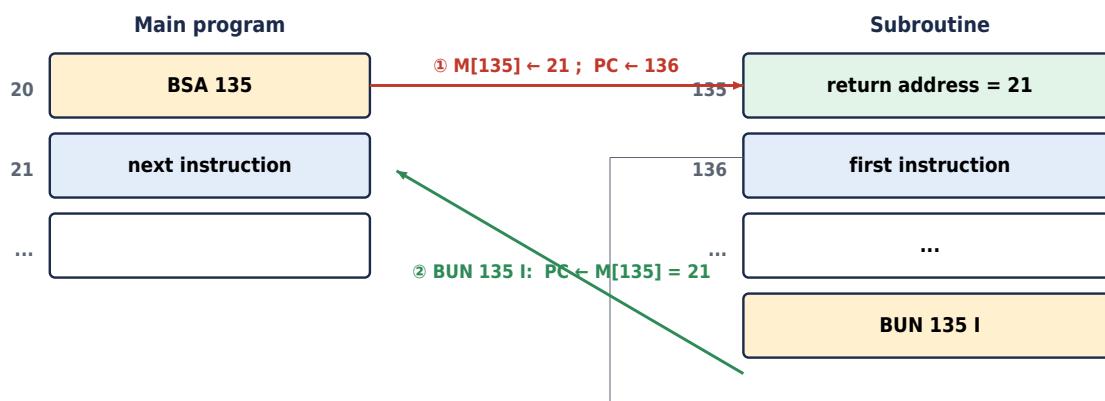


Fig 3.1 BSA (call) stores the return address; BUN ... I (return) reads it back

1. **Call - BSA SUB:**  $M[AR] \leftarrow PC$  stores the **return address** in the first word of the subroutine;  $PC \leftarrow AR + 1$  starts execution from the second word.
2. The subroutine body runs.
3. **Return - BUN SUB I:** indirect branch. The address part points to the first word, which holds the return address, so  $PC \leftarrow M[SUB]$  and the main program resumes.

**Example (SH4, shift AC left 4 times):** LDA X; BSA SH4; STA X, later LDA Y; BSA SH4; STA Y. The one subroutine (4 × CIL, AND with mask FFF0, BUN SH4 I) turns X = 1234 into 2340 and Y = 4321 into 3210.

**Advantages:** code reuse (saves memory) and modularity (easier debug and maintenance). **Limitation:** the return address is stored inside the subroutine, so it cannot call itself (no recursion).

**Q28**

5 marks

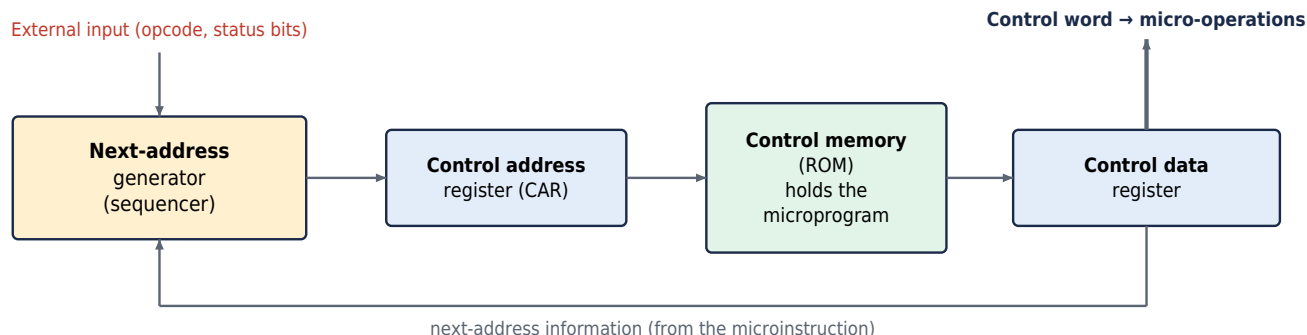
**Explain the organization of a microprogrammed control unit with a neat block diagram.**

Fig 4.1 Organization of a microprogrammed control unit

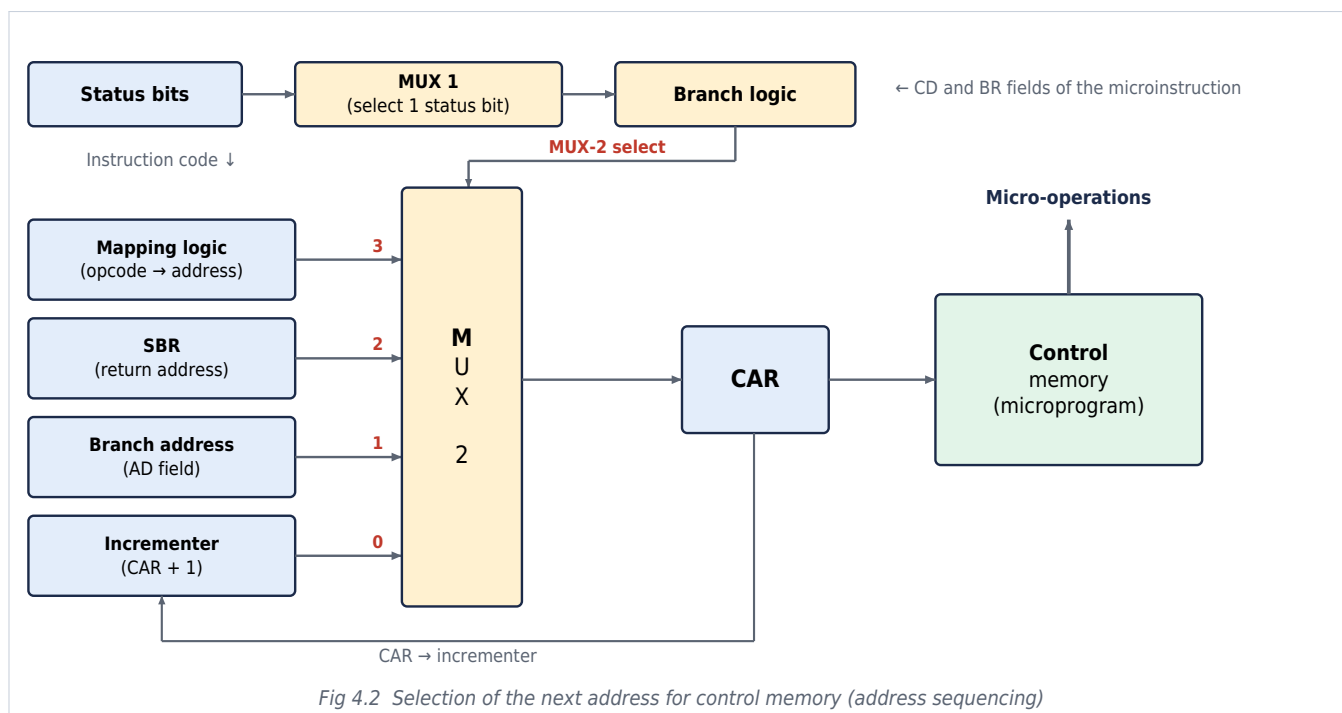
| Block                                            | Function                                                                              |
|--------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Control memory (ROM)</b>                      | Holds the microprogram (all microinstructions)                                        |
| <b>CAR</b>                                       | Holds the address of the next microinstruction to read                                |
| <b>Control data register (pipeline register)</b> | Holds the microinstruction being executed so the next can be read at the same time    |
| <b>Next-address generator (sequencer)</b>        | Computes the next CAR value from the current microinstruction, opcode and status bits |
| <b>Datapath / external inputs</b>                | Receive control signals; give back status bits and the opcode                         |

**Working:** CAR supplies an address → control memory outputs a microinstruction → it is latched in the control data register → its **micro-operation fields** drive the datapath, its **next-address fields** go to the sequencer → the sequencer loads CAR with the next address. This repeats every clock. The routine for an instruction is found by mapping its opcode to a starting address.

**Q29**  
5 marks

**Explain address sequencing in a microprogrammed control unit and discuss different methods of generating the next microinstruction address.**

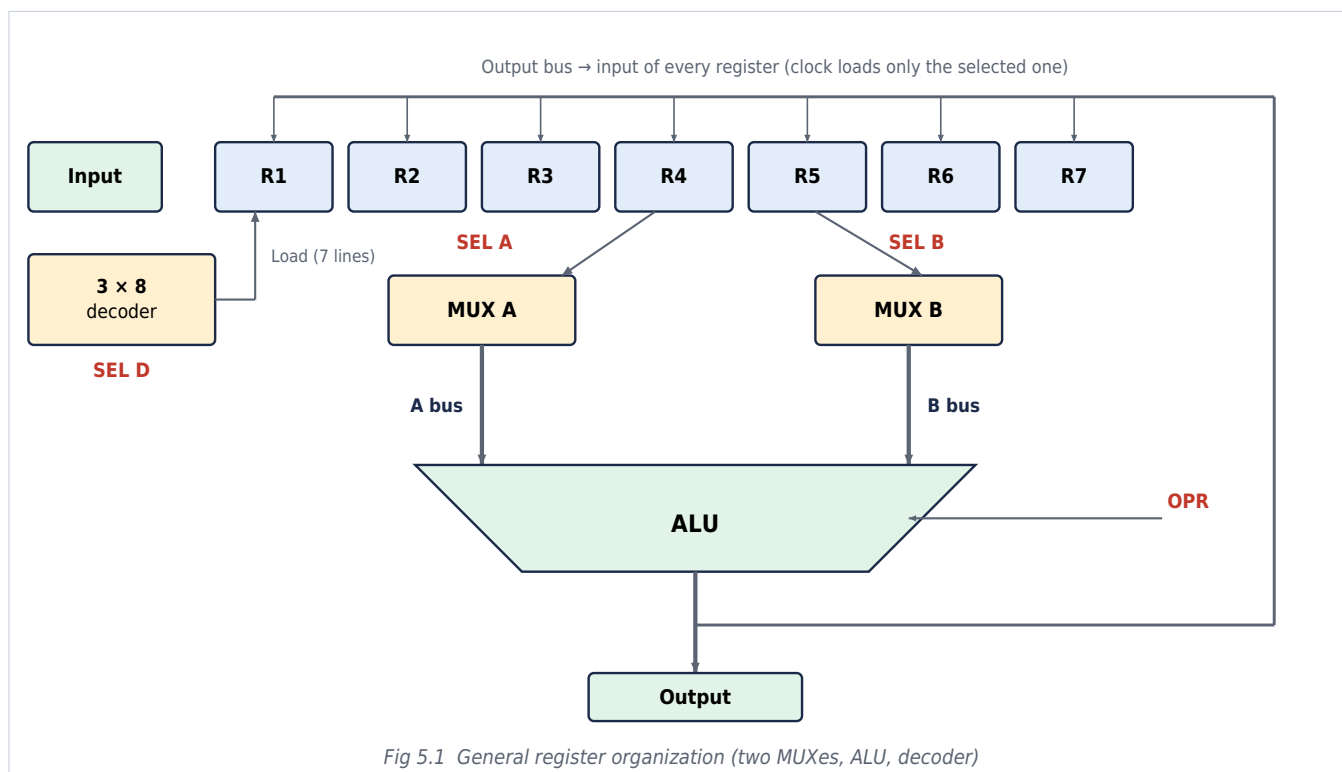
Microinstructions are stored in **groups (routines)** in control memory. **Address sequencing** is the process of choosing the address of the next microinstruction: move on, jump, enter the routine of the fetched instruction, call and return. At power-on an **initial address** (start of the fetch routine) is loaded into CAR.



| Method                      | How the next address is formed                                                                                                             |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Incrementing             | $CAR \leftarrow CAR + 1$ (normal case)                                                                                                     |
| 2. Branching                | Conditional or unconditional: $CAR \leftarrow AD$ (address field) if the selected status bit is 1, else $CAR \leftarrow CAR + 1$           |
| 3. Mapping                  | Opcode bits of the instruction are mapped to the start address of its routine: $CAR(2-5) \leftarrow DR(11-14)$ , $CAR(0,1,6) \leftarrow 0$ |
| 4. Subroutine call / return | CALL: $CAR \leftarrow AD$ , $SBR \leftarrow CAR + 1$ . RET: $CAR \leftarrow SBR$ (a microroutine shared by many routines)                  |

**Q30**  
5 marks

**Explain General Register Organization with a suitable block diagram. Show how  $R3 \leftarrow R1 + R2$  can be performed.**



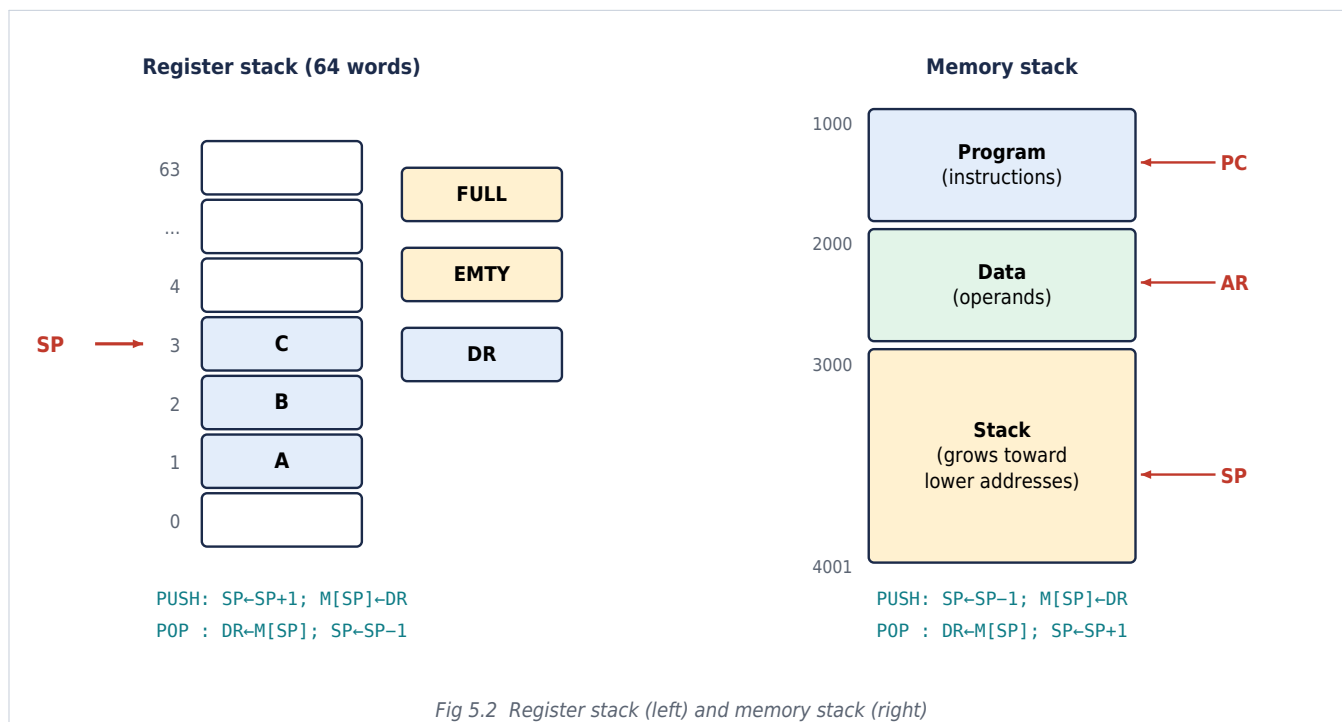
- Seven registers R1-R7 feed two multiplexers, **MUX A** and **MUX B**, which put the chosen operands on the A bus and B bus (select lines SEL A, SEL B).
- The **ALU** operates on A and B as selected by **OPR**; the result goes to the output bus, which is connected to the inputs of all registers.
- A **3×8 decoder** (SEL D) enables the load line of exactly one destination register. All in **one clock cycle**.

**R3 ← R1 + R2:** SEL A = 001 (R1), SEL B = 010 (R2), SEL D = 011 (R3), OPR = 00010 (ADD). Control word = 001 010 011 00010 (14 bits).

**Q31**  
5 marks

**Explain stack organization and the operations PUSH and POP with a suitable example.**

A **stack** is a LIFO storage: the item stored last is retrieved first. The register that holds the address of the top item is the **stack pointer (SP)**. Only two operations exist: **PUSH** (insert on top) and **POP** (remove from top).



|              | Register stack (64 words)                                                                                    | Memory stack                                   |
|--------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| <b>PUSH</b>  | $SP \leftarrow SP + 1$ ; $M[SP] \leftarrow DR$ ; if $SP = 0$ then $FULL \leftarrow 1$ ; $EMPTY \leftarrow 0$ | $SP \leftarrow SP - 1$ ; $M[SP] \leftarrow DR$ |
| <b>POP</b>   | $DR \leftarrow M[SP]$ ; $SP \leftarrow SP - 1$ ; if $SP = 0$ then $EMPTY \leftarrow 1$ ; $FULL \leftarrow 0$ | $DR \leftarrow M[SP]$ ; $SP \leftarrow SP + 1$ |
| <b>Flags</b> | $FULL$ and $EMPTY$                                                                                           | Limit registers check overflow                 |

**Example:** PUSH 10  $\rightarrow$  stack [10]; PUSH 20  $\rightarrow$  [10, 20] (SP points to 20); POP  $\rightarrow$  20 comes out into DR, stack [10] and SP points to 10 again.

**Q32**

5 marks

**Explain immediate, direct, indirect, register, register-indirect and indexed addressing modes with examples.**

Assume address field = 300,  $R1 = 200$ , index register  $XR = 100$ ,  $M[300] = 200$ ,  $M[200] = 45$ ,  $M[400] = 77$ .

| Mode                     | Effective address                                  | Instruction  | Operand loaded      | Mem. refs |
|--------------------------|----------------------------------------------------|--------------|---------------------|-----------|
| <b>Immediate</b>         | none (operand is in the instruction)               | LOAD #300    | $AC = 300$          | 0         |
| <b>Direct</b>            | $EA = \text{address field} = 300$                  | LOAD 300     | $AC = M[300] = 200$ | 1         |
| <b>Indirect</b>          | $EA = M[300] = 200$                                | LOAD @300    | $AC = M[200] = 45$  | 2         |
| <b>Register</b>          | operand is in the register                         | LOAD R1      | $AC = R1 = 200$     | 0         |
| <b>Register indirect</b> | $EA = \text{content of } R1 = 200$                 | LOAD (R1)    | $AC = M[200] = 45$  | 1         |
| <b>Indexed</b>           | $EA = \text{address field} + XR = 300 + 100 = 400$ | LOAD 300(XR) | $AC = M[400] = 77$  | 1         |

**Uses:** immediate = constants; direct = simple variables; indirect and register-indirect = pointers; register = fastest, few address bits; indexed = arrays (address field = start of array,  $XR$  = element number).

**Q33**

5 marks

**Given PC = 1200 and displacement = +350, calculate the effective address using relative addressing. Also explain when relative addressing is useful.**

**Relative addressing:**  $EA = \text{address part (displacement)} + \text{content of the program counter}$ .

$EA = PC + \text{displacement} = 1200 + 350 = 1550$ .

**When it is useful:**

- **Branch and jump instructions:** the target is usually close to the current instruction, so a short signed offset is enough and the address field can be small.
- **Position-independent (relocatable) code:** only offsets are stored, so the program still works if it is loaded at a different memory address.
- It supports jumping forward and backward (offset can be negative).

**Q34**

5 marks

**Compare RISC and CISC based on instruction set, instruction format, addressing modes, memory access, control unit and execution.**

| Basis                     | RISC                                                     | CISC                                                 |
|---------------------------|----------------------------------------------------------|------------------------------------------------------|
| <b>Instruction set</b>    | Small, simple, few instructions                          | Large, complex, 100 to 250 instructions              |
| <b>Instruction format</b> | Fixed length, easy to decode                             | Variable length, harder to decode                    |
| <b>Addressing modes</b>   | Few, simple                                              | Many (5 to 20)                                       |
| <b>Memory access</b>      | Only LOAD and STORE; operations are register to register | Many instructions work directly on memory operands   |
| <b>Control unit</b>       | Hardwired (fast)                                         | Microprogrammed (flexible)                           |
| <b>Execution</b>          | One instruction per clock cycle; pipeline is efficient   | Several cycles per instruction; pipelining is harder |

**Extra points if you have time:** RISC has a large register file with overlapped register windows and depends on a smart compiler (longer programs); CISC has few registers, shorter programs and simpler compilers. Examples: RISC = ARM, MIPS, RISC-V; CISC = x86.

## Section C - 7-mark questions (Q35 to Q46)

**Q35**  
7 marks

Given  $R1 = 10101101$  and  $R2 = 01010111$ , perform: (a)  $R3 \leftarrow R1 + R2$  (b)  $R4 \leftarrow R1 - R2$  using 2's complement (c)  $R5 \leftarrow R1 \text{ AND } R2$  (d) Logical left shift of  $R1$  by 2 positions (e) Arithmetic right shift of  $R2$  by 2 positions. Identify the type of micro-operation used in each case.

### (a) $R3 \leftarrow R1 + R2$ (arithmetic)

```
R1  = 1010 1101 (173)
R2  = 0101 0111 ( 87)
-----
Sum = 1 0000 0100 (260)
R3  = 0000 0100 (8-bit result; the carry-out 1 goes to the carry flag / E)
```

### (b) $R4 \leftarrow R1 - R2$ using 2's complement (arithmetic)

```
R2      = 0101 0111
1's compl. = 1010 1000 (invert every bit)
2's compl. = 1010 1001 (add 1) = -R2

R1      = 1010 1101
+ (-R2) = 1010 1001
-----
carry 1 | 0101 0110 discard the carry
R4      = 0101 0110 (86) (173 - 87 = 86, carry 1 = no borrow)
```

### (c) $R5 \leftarrow R1 \text{ AND } R2$ (logic)

| Bit       | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | Result    |
|-----------|---|---|---|---|---|---|---|---|-----------|
| R1        | 1 | 0 | 1 | 0 | 1 | 1 | 0 | 1 |           |
| R2        | 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 |           |
| R1 AND R2 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0000 0101 |

### (d) Logical shift left of $R1$ by 2 and (e) Arithmetic shift right of $R2$ by 2 (both shift)

(d)  $R1 = 1010 1101$  shl 2 → the two leftmost bits (10) are lost,  
two 0s enter on the right → 1011 0100

(e)  $R2 = 0101 0111$  ashr 2 → the two rightmost bits (11) are lost,  
the sign bit (0) is copied in twice → 0001 0101

| Part | Result              | Type of micro-operation                  |
|------|---------------------|------------------------------------------|
| (a)  | 0000 0100 (carry 1) | Arithmetic (add)                         |
| (b)  | 0101 0110           | Arithmetic (subtract via 2's complement) |
| (c)  | 0000 0101           | Logic (AND)                              |
| (d)  | 1011 0100           | Shift (logical left)                     |
| (e)  | 0001 0101           | Shift (arithmetic right)                 |

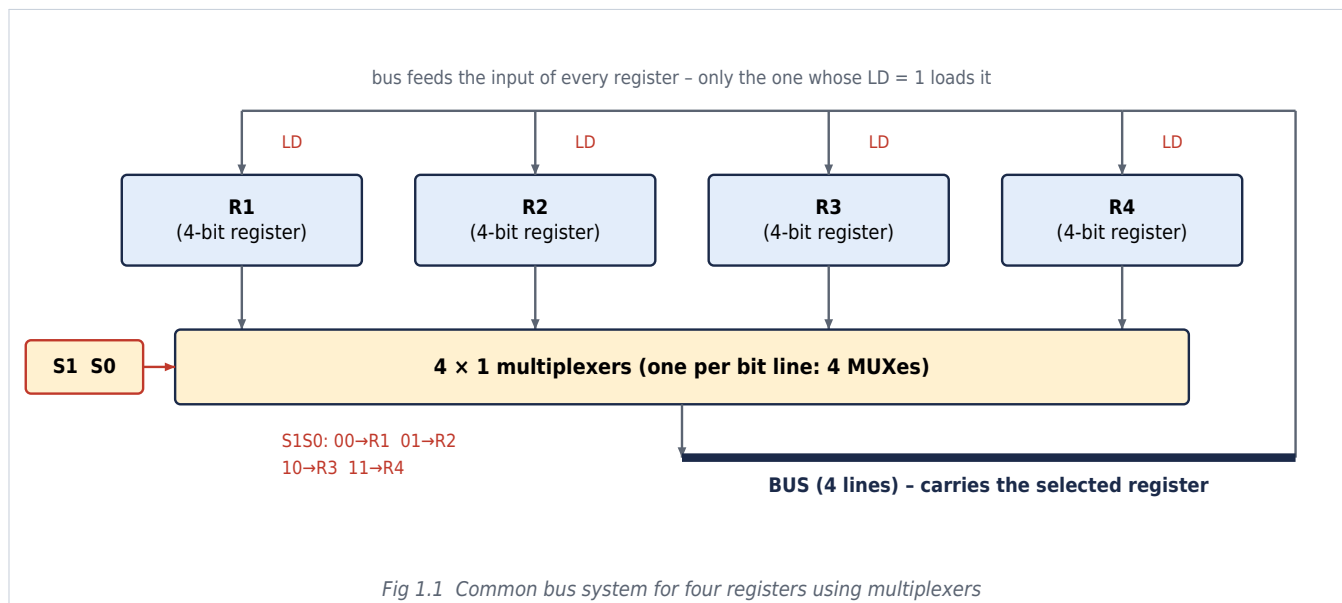
#### EXAM TIP

Extra insight if you have time: read as **signed** numbers  $R1 = -83$  and  $R2 = +87$ . (a)  $-83 + 87 = +4$  is correct with no overflow. (b)  $-83 - 87 = -170$  does not fit in 8 bits, so signed overflow occurs ( $V = 1$ ); as unsigned numbers  $173 - 87 = 86$  is correct.

**Q36**  
7 marks

Explain the organization of a common bus system using multiplexers. Show how data can be transferred between multiple registers through the common bus and explain the required selection signals.

**Idea:** connecting every register to every other needs too many wires. Instead all registers use **one common set of lines (the bus)**. Only **one register drives the bus at a time**; a multiplexer chooses which one.



### Organization

- **Four registers R1-R4, 4 bits each** → four 4×1 multiplexers, one per bit position. Input 0 of every MUX comes from R1, input 1 from R2, input 2 from R3, input 3 from R4.
- The **select lines S1 S0** are common to all four multiplexers, so they all pick the same register; the 4 MUX outputs together form the 4-bit bus.
- The bus goes to the inputs of all registers. Each register has its own **LD (load) control**. Only the register whose LD = 1 copies the bus at the clock edge.
- General rule:  $n$  registers of  $k$  bits need  **$k$  multiplexers of size  $n \times 1$**  and  **$\log_2 n$  select lines**.

### Selection signals

| S1 S0 | Register placed on the bus |
|-------|----------------------------|
| 00    | R1                         |
| 01    | R2                         |
| 10    | R3                         |
| 11    | R4                         |

### Transfers between several registers (one transfer per clock, because there is only one bus)

| Clock | Micro-operation    | S1 S0 (source) | Load enabled | Bus carries |
|-------|--------------------|----------------|--------------|-------------|
| 1     | $R4 \leftarrow R1$ | 00             | LD(R4)       | R1          |
| 2     | $R2 \leftarrow R3$ | 10             | LD(R2)       | R3          |
| 3     | $R1 \leftarrow R2$ | 01             | LD(R1)       | R2          |

**Swap R1 and R2 using R3 as a temporary:**  $R3 \leftarrow R1$  ( $S = 00$ , LD R3), then  $R1 \leftarrow R2$  ( $S = 01$ , LD R1), then  $R2 \leftarrow R3$  ( $S = 10$ , LD R2): three clock pulses, since the bus can carry only one value at a time.

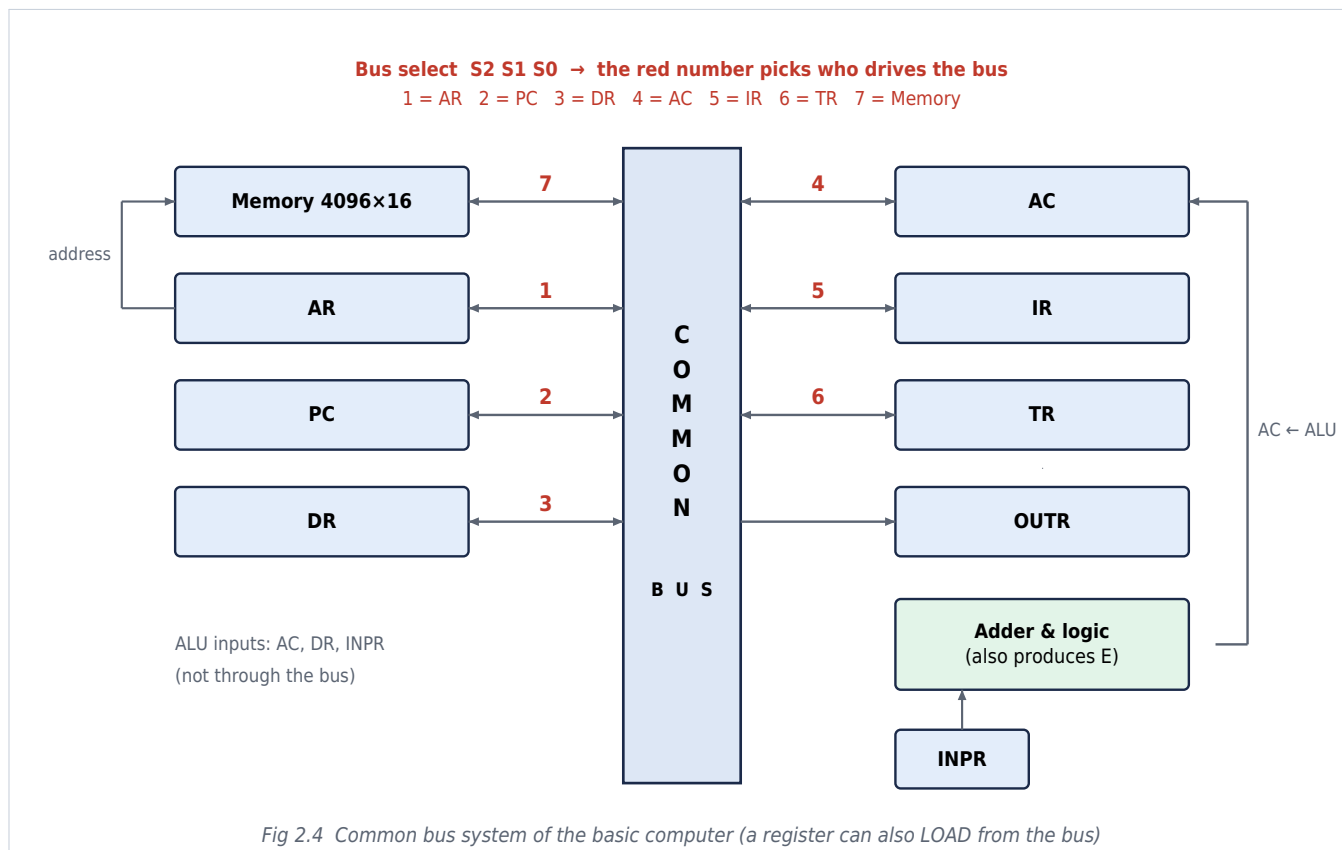
**Alternative:** tri-state buffers on every register output; a 2×4 decoder enables one buffer while the others stay in high impedance. If two registers drove the bus at once (bus contention) the data would be garbage.

**Q37**

7 marks

**Explain the complete organization of a basic computer with a neat block diagram. Discuss the functions of Memory, ALU, AC, PC, IR, AR, DR, Control Unit and I/O registers.**

The basic computer has a **4096 × 16 memory**, **eight registers**, a **16-bit common bus**, an adder-and-logic circuit (ALU), a hardwired control unit and a terminal interface. All the units talk through the common bus; the select lines S2 S1 S0 decide which unit drives it.



| Block                                | Function                                                                                                                                                                             |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Memory (4096 × 16)</b>            | Stores both instructions and data (stored-program concept). Address always comes from AR. Read and Write control lines.                                                              |
| <b>ALU (add and logic)</b>           | Performs add, AND, complement, shift on AC, DR and INPR; its output goes to AC (and to the bus).                                                                                     |
| <b>AC (accumulator)</b>              | Main processing register; holds one operand and the result. E is its 1-bit extension (carry).                                                                                        |
| <b>PC (program counter, 12)</b>      | Address of the next instruction; incremented after every fetch; loaded by branches.                                                                                                  |
| <b>IR (instruction register, 16)</b> | Holds the fetched instruction. Bits 14-12 go to the decoder, bit 15 to the I flip-flop, bits 11-0 are the address / operation bits.                                                  |
| <b>AR (address register, 12)</b>     | Supplies the memory address for every read or write.                                                                                                                                 |
| <b>DR (data register, 16)</b>        | Holds the operand read from memory; second input of the ALU.                                                                                                                         |
| <b>TR (temporary register, 16)</b>   | Temporary storage (e.g. PC during an interrupt).                                                                                                                                     |
| <b>Control unit</b>                  | Instruction decoder (3×8), sequence counter with 4×16 decoder giving T0-T15, and control logic gates. Produces LD, INR, CLR, bus select, memory read/write and ALU controls.         |
| <b>I/O: INPR, OUTR, FGI, FGO</b>     | INPR (8) receives a character from the keyboard, OUTR (8) sends one to the printer/display. Flags FGI and FGO synchronise the slow device with the CPU; IEN and R handle interrupts. |

**DRAW THIS** Draw [Fig 2.3](#), [Fig 2.6](#) (tap to jump).

**Q38**  
7 marks

**Explain the working of AND, ADD, LDA, STA, BUN, BSA and ISZ instructions of a basic computer. Show suitable RTL operations for any four instructions.**

After fetch and decode (T0-T2) and the indirect step (T3) the effective address is in AR. Then each instruction runs its own micro-operations, selected by  $D_i \cdot T_k$ . Every instruction ends with  $SC \leftarrow 0$  (return to T0).

| Instr.     | D  | RTL (execution)                                                                                  | Working                                                   |
|------------|----|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <b>AND</b> | D0 | D0T4: $DR \leftarrow M[AR]$<br>D0T5: $AC \leftarrow AC \wedge DR, SC \leftarrow 0$               | Read the operand into DR, then AND it bit by bit with AC. |
| <b>ADD</b> | D1 | D1T4: $DR \leftarrow M[AR]$<br>D1T5: $AC \leftarrow AC + DR, E \leftarrow Cout, SC \leftarrow 0$ | Read the operand, add to AC; the carry-out goes to E.     |

| Instr.     | D  | RTL (execution)                                                                                                                                               | Working                                                                                          |
|------------|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>LDA</b> | D2 | D2T4: $DR \leftarrow M[AR]$<br>D2T5: $AC \leftarrow DR, SC \leftarrow 0$                                                                                      | Read the operand into DR, copy DR to AC (AC has no direct path from memory).                     |
| <b>STA</b> | D3 | D3T4: $M[AR] \leftarrow AC, SC \leftarrow 0$                                                                                                                  | Write AC to the memory word addressed by AR (one step).                                          |
| <b>BUN</b> | D4 | D4T4: $PC \leftarrow AR, SC \leftarrow 0$                                                                                                                     | Jump: the effective address becomes the next instruction address.                                |
| <b>BSA</b> | D5 | D5T4: $M[AR] \leftarrow PC, AR \leftarrow AR + 1$<br>D5T5: $PC \leftarrow AR, SC \leftarrow 0$                                                                | Save the return address (PC) in the word at AR, continue at AR + 1: subroutine call.             |
| <b>ISZ</b> | D6 | D6T4: $DR \leftarrow M[AR]$<br>D6T5: $DR \leftarrow DR + 1$<br>D6T6: $M[AR] \leftarrow DR$ , if $(DR = 0)$ then $PC \leftarrow PC + 1$ ,<br>$SC \leftarrow 0$ | Increment the word in memory; if it becomes 0 skip the next instruction. Used as a loop counter. |

**MEMORIZE** Opcodes: **AND 000, ADD 001, LDA 010, STA 011, BUN 100, BSA 101, ISZ 110** (hex with I = 0: 0xxx to 6xxx; with I = 1: 8xxx to Exxx). Example: ISZ on a word holding FFFF becomes 0000, so the next instruction is skipped.

**Q39**

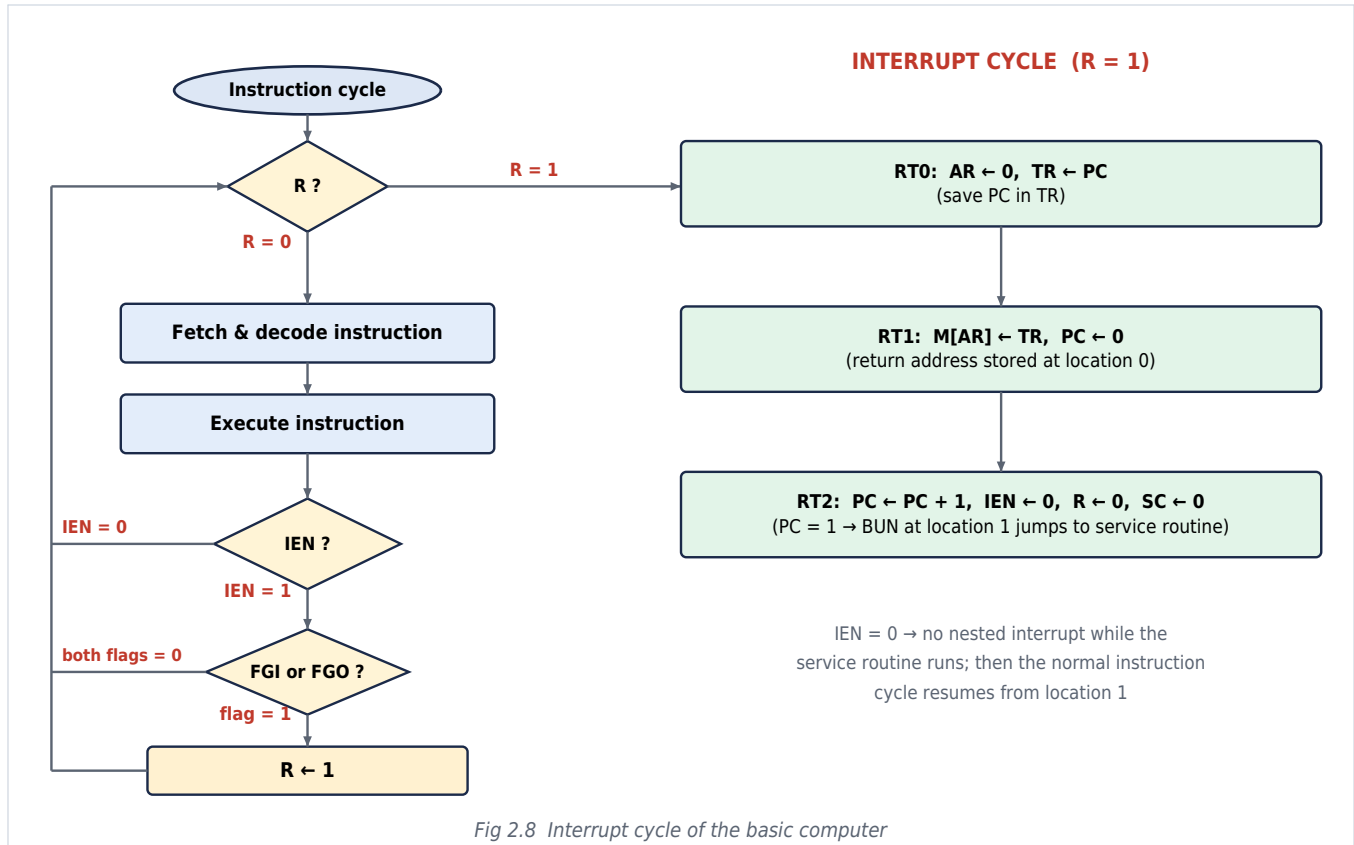
7 marks

**Explain the interrupt mechanism and interrupt cycle of a basic computer with a suitable flow diagram. Discuss what happens to the current program when an interrupt occurs.**

### Mechanism

The CPU works much faster than I/O devices. With **programmed I/O** it wastes time testing the flags FGI/FGO. With **interrupt-initiated I/O** the device sets its flag and the CPU is interrupted only when a device needs service; meanwhile the CPU executes the main program.

- **IEN** (interrupt enable flip-flop): set by **ION**, cleared by **IOF**. If  $IEN = 0$  interrupts are ignored.
- **R** (interrupt flip-flop):  $R = 0$  normal instruction cycle,  $R = 1$  interrupt cycle.
- At the end of the execute phase (never during T0-T2): if  $IEN = 1$  and  $(FGI = 1 \text{ or } FGO = 1)$  then  $R \leftarrow 1$ .  $T_0' T_1' T_2'$  (**IEN**) ( $FGI + FGO$ ):  $R \leftarrow 1$



### Interrupt cycle in RTL

| Timing     | RTL                                                                       | Effect                                                                                                          |
|------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>RT0</b> | $AR \leftarrow 0, TR \leftarrow PC$                                       | AR = address 0; PC (return address) saved in TR                                                                 |
| <b>RT1</b> | $M[AR] \leftarrow TR, PC \leftarrow 0$                                    | Return address stored in memory location 0                                                                      |
| <b>RT2</b> | $PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$ | PC = 1 (location 1 has a BUN to the service routine); IEN cleared so no new interrupt; back to the normal cycle |

### What happens to the current program

1. The instruction being executed **finishes first**; the interrupt is taken between instructions.
2. The program is **suspended, not lost**: the address of its next instruction is saved in location 0.
3. Control jumps to location 1 → **BUN** to the interrupt service routine (ISR). The ISR saves AC and E, checks FGI/FGO to find the device, transfers the data (INP / OUT), clears the flag, restores AC and E.
4. The ISR executes **ION** (interrupts allowed again) and returns with **BUN 0 I** (indirect through location 0). The main program resumes exactly where it stopped.

**Q40**

7 marks

**Write an assembly language program/algorithm to add N numbers stored in consecutive memory locations. Explain initialization, memory addressing, loop counter, accumulation, loop termination and result storage.**

### Algorithm

1.  $PTR \leftarrow$  address of the first number;  $CTR \leftarrow -N$ ;  $AC \leftarrow 0$ .
2. Repeat:  $AC \leftarrow AC + M[PTR]$ ;  $PTR \leftarrow PTR + 1$ ;  $CTR \leftarrow CTR + 1$ .
3. If  $CTR \neq 0$  go back to step 2; if  $CTR = 0$  leave the loop.
4.  $SUM \leftarrow AC$ ; halt.

### Program (N = 100 numbers stored from hex 150; for another N use DEC -N)

| Loc / Label | Instruction    | What it does                                                             |
|-------------|----------------|--------------------------------------------------------------------------|
|             | ORG 100        | Program starts at hex 100                                                |
| 100         | LDA ADS        | $AC \leftarrow$ address of the first operand (150)                       |
| 101         | STA PTR        | $PTR \leftarrow 150$ (pointer to the array)                              |
| 102         | LDA NBR        | $AC \leftarrow -100$                                                     |
| 103         | STA CTR        | $CTR \leftarrow -100$ (loop counter)                                     |
| 104         | CLA            | $AC \leftarrow 0$ (running sum)                                          |
| 105         | LOP, ADD PTR I | $AC \leftarrow AC + M[M[PTR]]$ (indirect: PTR holds the operand address) |
| 106         | ISZ PTR        | $PTR \leftarrow PTR + 1$ : next number                                   |
| 107         | ISZ CTR        | $CTR \leftarrow CTR + 1$ ; skip next instruction when $CTR = 0$          |
| 108         | BUN LOP        | Counter not zero: repeat the loop                                        |
| 109         | STA SUM        | Loop finished: store the sum                                             |
| 10A         | HLT            | Halt                                                                     |
| 10B         | ADS, HEX 150   | Address of the first number                                              |
| 10C         | PTR, HEX 0     | Pointer (variable)                                                       |
| 10D         | NBR, DEC -100  | Constant -N                                                              |
| 10E         | CTR, HEX 0     | Counter (variable)                                                       |
| 10F         | SUM, HEX 0     | Result                                                                   |
|             | ORG 150        | Data starts at hex 150                                                   |
| 150         | DEC 75         | first number                                                             |
| ...         | ...            | ...                                                                      |
| 1B3         | DEC 23         | hundredth number                                                         |
|             | END            |                                                                          |

| Part asked            | Where in the program | Explanation                                                                                          |
|-----------------------|----------------------|------------------------------------------------------------------------------------------------------|
| <b>Initialization</b> | 100 - 104            | $PTR =$ start address, $CTR = -N$ , $AC = 0$ . PTR and CTR are loaded from memory words ADS and NBR. |

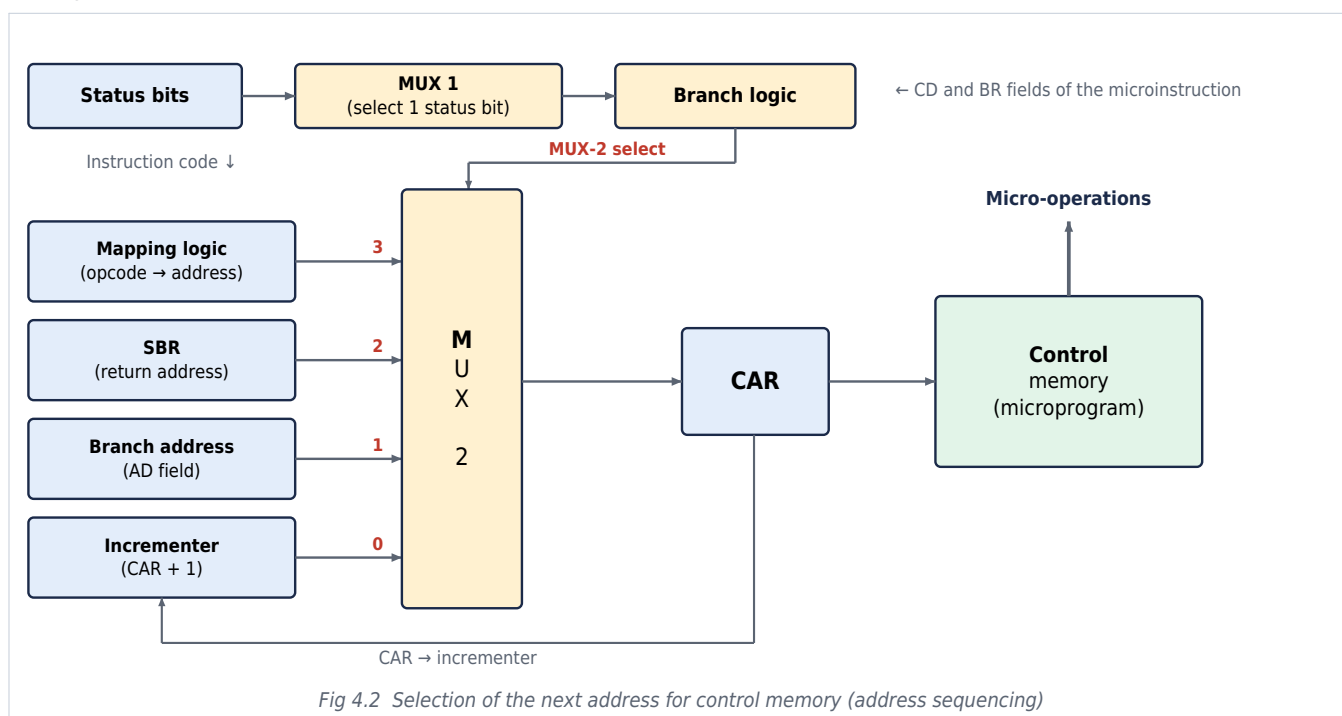
| Part asked               | Where in the program | Explanation                                                                                                                        |
|--------------------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Memory addressing</b> | ADD PTR I, ISZ PTR   | <b>Indirect addressing</b> reads the operand whose address is stored in PTR; incrementing PTR walks through consecutive locations. |
| <b>Loop counter</b>      | ISZ CTR              | Counts from $-N$ up to 0; the basic computer has no loop instruction, so ISZ is the counter.                                       |
| <b>Accumulation</b>      | ADD PTR I            | Each pass adds one number to AC.                                                                                                   |
| <b>Loop termination</b>  | ISZ CTR / BUN LOP    | While $CTR \neq 0$ the BUN executes and repeats; when CTR becomes 0 ISZ skips the BUN and control falls to STA SUM.                |
| <b>Result storage</b>    | STA SUM              | Final AC is written to SUM.                                                                                                        |

**Trace for N = 3 with numbers 5, 7, 9:** start PTR = 150, CTR = -3, AC = 0. Pass 1: AC = 5, PTR = 151, CTR = -2. Pass 2: AC = 12, PTR = 152, CTR = -1. Pass 3: AC = 21, PTR = 153, CTR = 0 → skip BUN → SUM = 21.

**Q41**  
7 marks

**Explain address sequencing in microprogrammed control. Describe how the next microinstruction address can be obtained through: (1) Sequential increment (2) Branching (3) Opcode mapping (4) Subroutine/return mechanism.**

Microinstructions are stored in control memory in **routines** (fetch, ADD, STA...). **Address sequencing** is the logic that decides the next value of CAR so that microinstructions run in the right order and the right routine is entered for every machine instruction. At reset an initial address (start of fetch) is loaded into CAR.



- Sequential increment:**  $CAR \leftarrow CAR + 1$ . Used when the next microinstruction is the following word, e.g. fetch routine  $64 \rightarrow 65 \rightarrow 66$ .
- Branching:** the microinstruction has a branch field (BR = JMP), a condition field (CD = U, I, S or Z) and an address field AD. A multiplexer selects the tested status bit; if the condition is 1 then  $CAR \leftarrow AD$  else  $CAR \leftarrow CAR + 1$ . Example: CD = Z, BR = JMP, AD = 80 means "if AC = 0 go to 80".
- Opcode mapping:** after fetch, the opcode bits of the instruction in DR are converted into the start address of its routine:  $CAR(2-5) \leftarrow DR(11-14)$ ,  $CAR(0,1,6) \leftarrow 0$ . Each routine gets a block of 4 words, so opcode 0010 maps to  $CAR = 0001000 = 8$ .
- Subroutine / return:** a microroutine shared by many instructions (such as effective-address calculation) is entered with **CALL**:  $CAR \leftarrow AD$ ,  $SBR \leftarrow CAR + 1$  and left with **RET**:  $CAR \leftarrow SBR$ . SBR is the subroutine register that holds the return address.

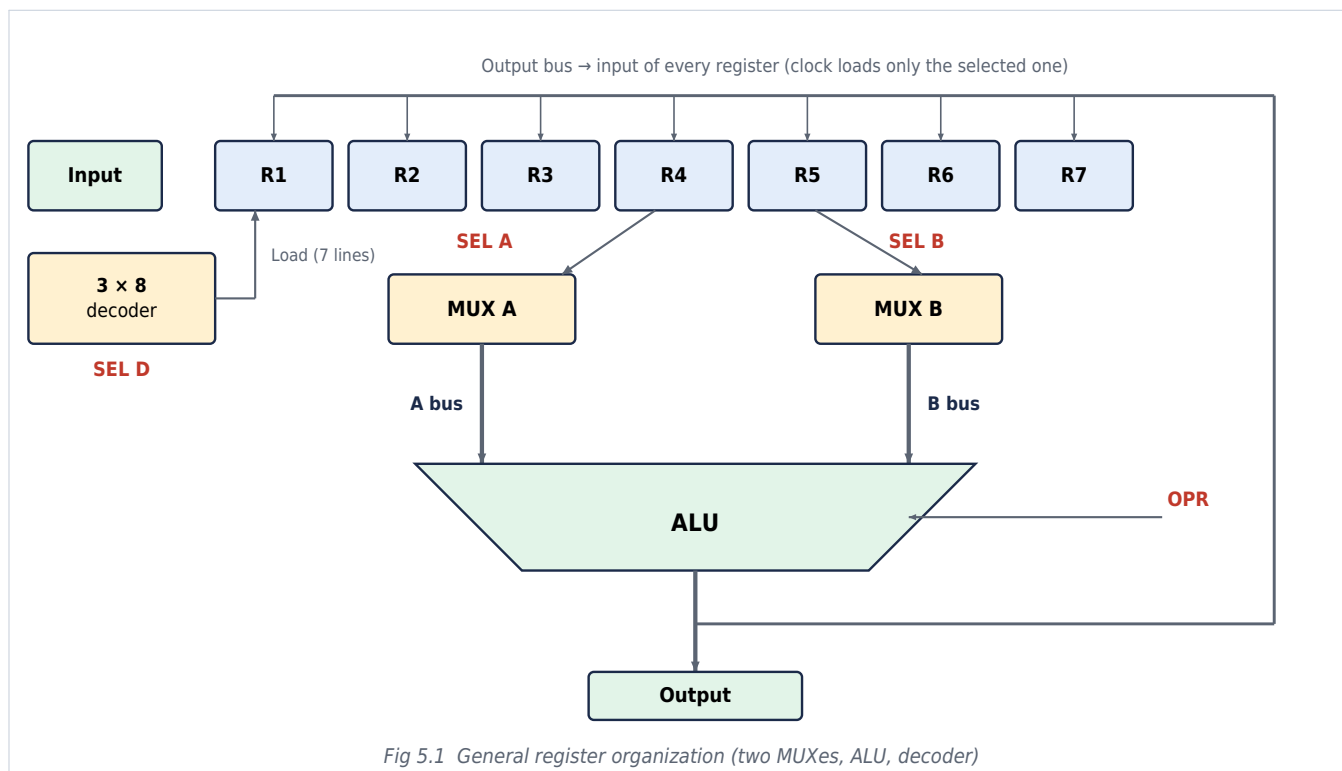
| BR | Symbol | Next address                                                                    |
|----|--------|---------------------------------------------------------------------------------|
| 00 | JMP    | $CAR \leftarrow AD$ if condition = 1, else $CAR \leftarrow CAR + 1$             |
| 01 | CALL   | $CAR \leftarrow AD$ , $SBR \leftarrow CAR + 1$ if condition = 1, else $CAR + 1$ |
| 10 | RET    | $CAR \leftarrow SBR$                                                            |

| BR | Symbol | Next address                                             |
|----|--------|----------------------------------------------------------|
| 11 | MAP    | $CAR(2-5) \leftarrow DR(11-14), CAR(0,1,6) \leftarrow 0$ |

**Q42**

7 marks

**Explain General Register Organization with a suitable diagram. Show the sequence of control operations required to execute  $R3 \leftarrow R1 + R2$ . Clearly identify source registers, destination register, bus and ALU operations.**



**Organization:** seven registers R1-R7 (plus an external input) are connected to two 7+1-input multiplexers. **MUX A** (select SEL A) drives the **A bus** and **MUX B** (select SEL B) drives the **B bus**. Both buses go to the **ALU**, whose operation is chosen by **OPR**. The ALU output (output bus) is connected to the inputs of all registers; a **3x8 decoder** (SEL D) enables the load line of one register. The 14-bit control word is **SELA (3) | SELB (3) | SELD (3) | OPR (5)**.

#### Sequence of control operations for $R3 \leftarrow R1 + R2$

| Step | Control field | Value | What happens                                                                                                                |
|------|---------------|-------|-----------------------------------------------------------------------------------------------------------------------------|
| 1    | SEL A         | 001   | <b>Source register 1 = R1:</b> MUX A puts R1 on the <b>A bus</b>                                                            |
| 2    | SEL B         | 010   | <b>Source register 2 = R2:</b> MUX B puts R2 on the <b>B bus</b>                                                            |
| 3    | OPR           | 00010 | <b>ALU operation = ADD:</b> the ALU computes $A + B$ and puts it on the <b>output bus</b>                                   |
| 4    | SEL D         | 011   | <b>Destination = R3:</b> decoder enables the load line of R3; the output bus value is loaded into R3 at the next clock edge |

Control word = 001 010 011 00010. Steps 1-4 all happen within **one clock cycle**: the two operands are read, the ALU works and the result is written back at the clock edge.

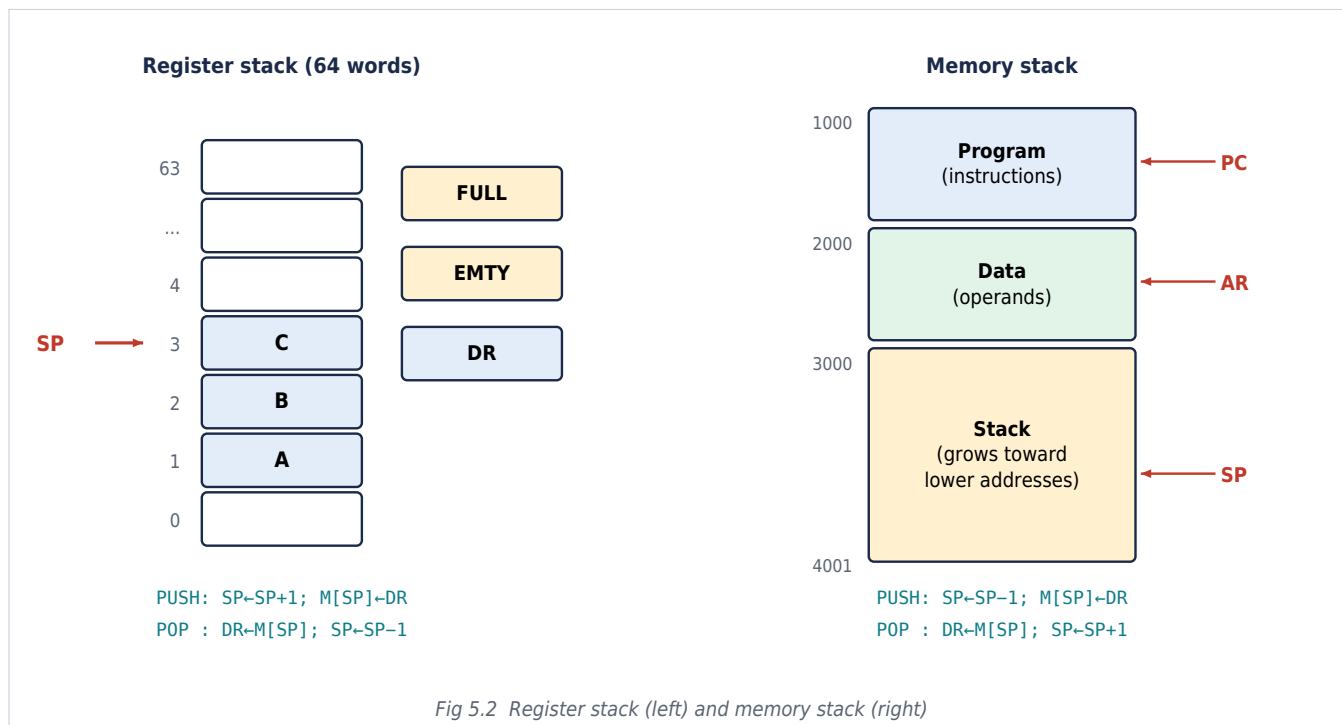
Other control words in the same format:  $R1 \leftarrow R2 - R3 = 010\ 011\ 001\ 00101$ ,  $R7 \leftarrow R1 = 001\ 000\ 111\ 00000$ .

**Q43**

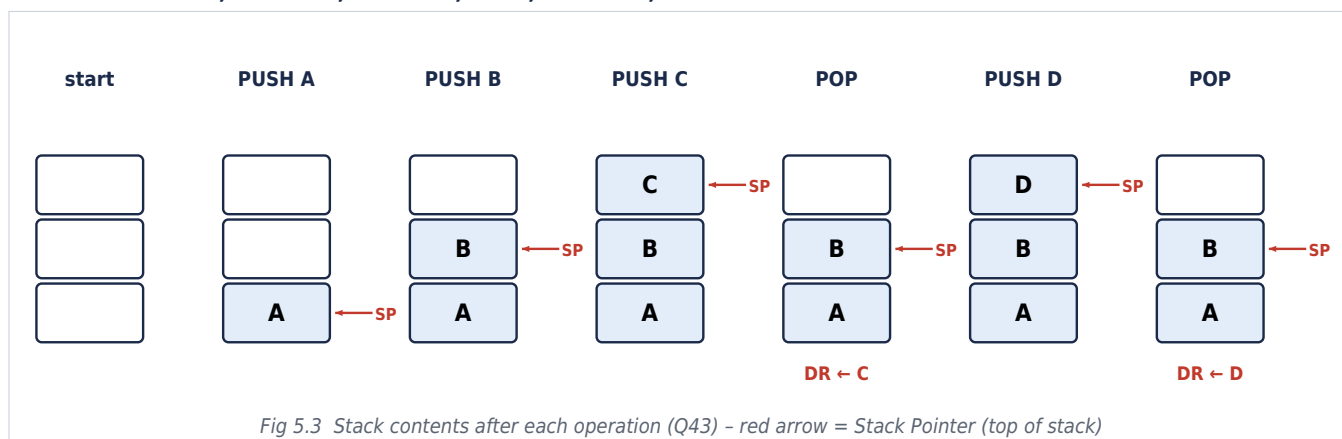
7 marks

**Explain stack organization with a neat diagram. Consider: PUSH A, PUSH B, PUSH C, POP, PUSH D, POP. Show the stack contents after each operation and explain the role of the Stack Pointer.**

A **stack** is a LIFO memory: the item stored last is retrieved first. Only the **top** is accessible. The **stack pointer (SP)** holds the address of the top item. In a **register stack** SP is a 6-bit register with FULL/EMPTY flags; in a **memory stack** a part of main memory is used and SP is an ordinary CPU register (the stack usually grows toward lower addresses).



### Trace: PUSH A, PUSH B, PUSH C, POP, PUSH D, POP



| Operation | Stack (bottom → top) | SP points to  | Data moved                       |
|-----------|----------------------|---------------|----------------------------------|
| (start)   | empty                | - (EMPTY = 1) | -                                |
| PUSH A    | A                    | A             | A written                        |
| PUSH B    | A, B                 | B             | B written                        |
| PUSH C    | A, B, C              | C             | C written                        |
| POP       | A, B                 | B             | C read out                       |
| PUSH D    | A, B, D              | D             | D written (into the slot C used) |
| POP       | A, B                 | B             | D read out                       |

**Final stack: A, B** (top = B). Items popped in order: C, then D (last in, first out).

### Role of the stack pointer

- SP always holds the address of the **top of the stack**, so no address is needed in a PUSH or POP-type stack instruction (zero-address).
- PUSH:** SP moves to the next free slot and the data is written there. **POP:** the top is read and SP moves back. (Register stack:  $SP \leftarrow SP + 1$ ;  $M[SP] \leftarrow DR$  /  $DR \leftarrow M[SP]$ ;  $SP \leftarrow SP - 1$ ; memory stack: SP decrements on PUSH and increments on POP.)
- SP is what enforces LIFO order and tells the hardware when the stack is full (overflow) or empty (underflow).
- A POP does not erase the data; it only moves SP, and the old value is overwritten by the next PUSH.

**Q44**  
7 marks

**Explain zero-address, one-address, two-address and three-address instruction formats with suitable examples. Then explain how  $X = (A + B) \times (C - D)$  could be evaluated using different instruction formats.**

| Format               | Layout                      | Working                                                                              | Example                                      |
|----------------------|-----------------------------|--------------------------------------------------------------------------------------|----------------------------------------------|
| <b>Three-address</b> | Opcode   Dest   Src1   Src2 | Result address given explicitly; shortest programs, longest instructions             | ADD R1, A, B ( $R1 \leftarrow M[A] + M[B]$ ) |
| <b>Two-address</b>   | Opcode   Dest/Src1   Src2   | Destination is also one source (overwritten); most common                            | ADD R1, B ( $R1 \leftarrow R1 + M[B]$ )      |
| <b>One-address</b>   | Opcode   Address            | Other operand and result in the implied <b>accumulator</b>                           | ADD B ( $AC \leftarrow AC + M[B]$ )          |
| <b>Zero-address</b>  | Opcode only                 | Operands are the top two stack items; needs postfix (RPN); PUSH/POP carry an address | ADD ( $TOS \leftarrow TOS + \text{next}$ )   |

**$X = (A + B) \times (C - D)$**

| Three-address (3)                                  | Two-address (6)                                                             | One-address (7)                                                   | Zero-address (8)                                                   |
|----------------------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------|
| ADD R1, A, B<br>SUB R2, C, D<br>MUL X, R1, R2      | MOV R1, A<br>ADD R1, B<br>MOV R2, C<br>SUB R2, D<br>MUL R1, R2<br>MOV X, R1 | LOAD A<br>ADD B<br>STORE T<br>LOAD C<br>SUB D<br>MUL T<br>STORE X | PUSH A<br>PUSH B<br>ADD<br>PUSH C<br>PUSH D<br>SUB<br>MUL<br>POP X |
| $R1 = A + B$<br>$R2 = C - D$<br>$X = R1 \times R2$ | $R1 = A + B$<br>$R2 = C - D$<br>$R1 = R1 \times R2$<br>$X = R1$             | $T = A + B$ (temp)<br>$AC = (C - D) \times T$                     | Postfix: A B + C D - $\times$<br>uses stack only                   |

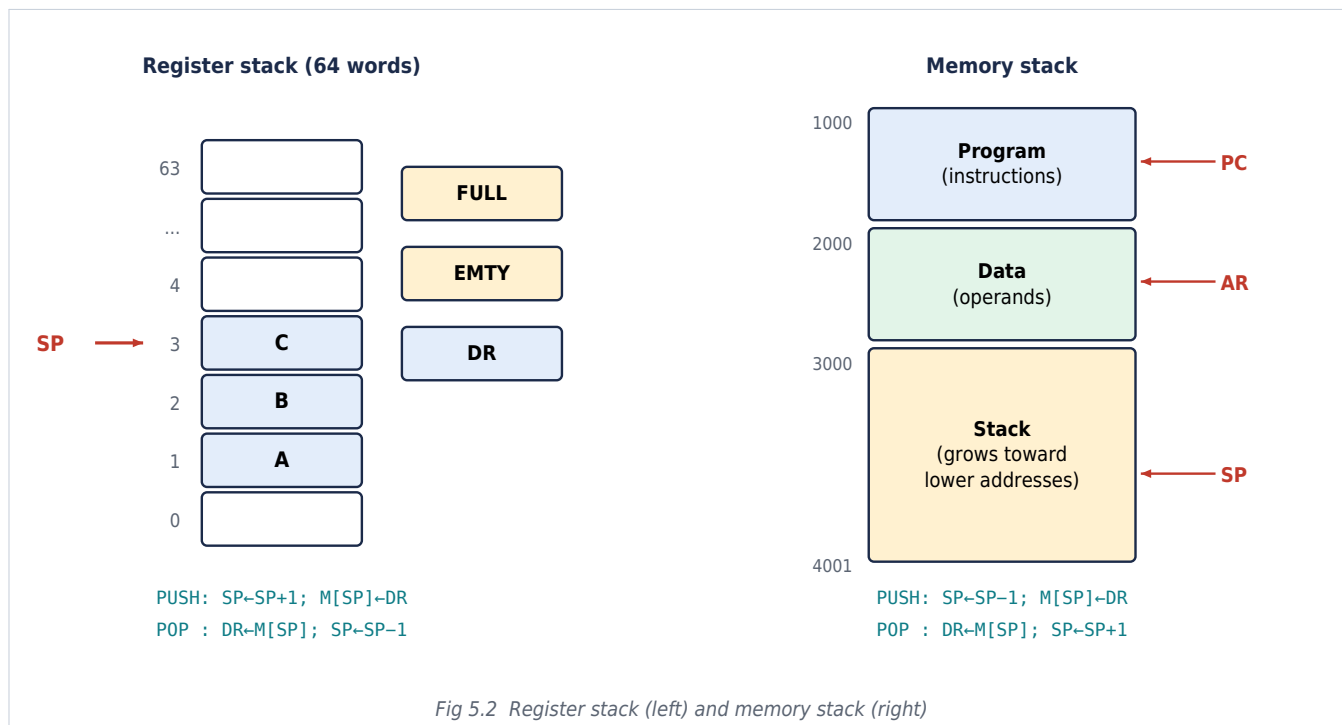
**EXAM TIP**

Closing line for full marks: **more addresses per instruction = fewer instructions but longer instruction words; fewer addresses = shorter instructions but longer programs** (three-address 3 instructions, zero-address 8).

**Q45**  
7 marks

**Explain stack organization in detail. Compare register stack and memory stack based on organization, speed, capacity, cost and implementation.**

A **stack** stores data so that the last item stored is the first retrieved (**LIFO**). The **stack pointer SP** holds the address of the top item. Operations: **PUSH** (insert) and **POP** (delete). Two ways to build it:



- **Register stack (64 words):** registers in the CPU; SP is 6 bits; **FULL** and **EMTY** flags; DR holds the data. PUSH:  $SP \leftarrow SP + 1$ ;  $M[SP] \leftarrow DR$ ; if  $SP = 0$  then  $FULL \leftarrow 1$ ;  $EMTY \leftarrow 0$ . POP:  $DR \leftarrow M[SP]$ ;  $SP \leftarrow SP - 1$ ; if  $SP = 0$  then  $EMTY \leftarrow 1$ ;  $FULL \leftarrow 0$ .
- **Memory stack:** a portion of main memory (memory is split into program, data and stack areas; PC, AR and SP point into them). PUSH:  $SP \leftarrow SP - 1$ ;  $M[SP] \leftarrow DR$ . POP:  $DR \leftarrow M[SP]$ ;  $SP \leftarrow SP + 1$ .
- **Uses:** evaluation of arithmetic expressions in **Reverse Polish (postfix) notation**, subroutine call and return, interrupts (saving state), recursion, nested calls.

**Example, RPN:**  $(3 \times 4) + (5 \times 6) = 3 \ 4 \ * \ 5 \ 6 \ * \ +$ . Scan left to right: operand → PUSH; operator → POP two, compute, PUSH result. Stack:  $3 \rightarrow 3 \ 4 \rightarrow 12 \rightarrow 12 \ 5 \rightarrow 12 \ 5 \ 6 \rightarrow 12 \ 30 \rightarrow 42$ .

### Register stack vs memory stack

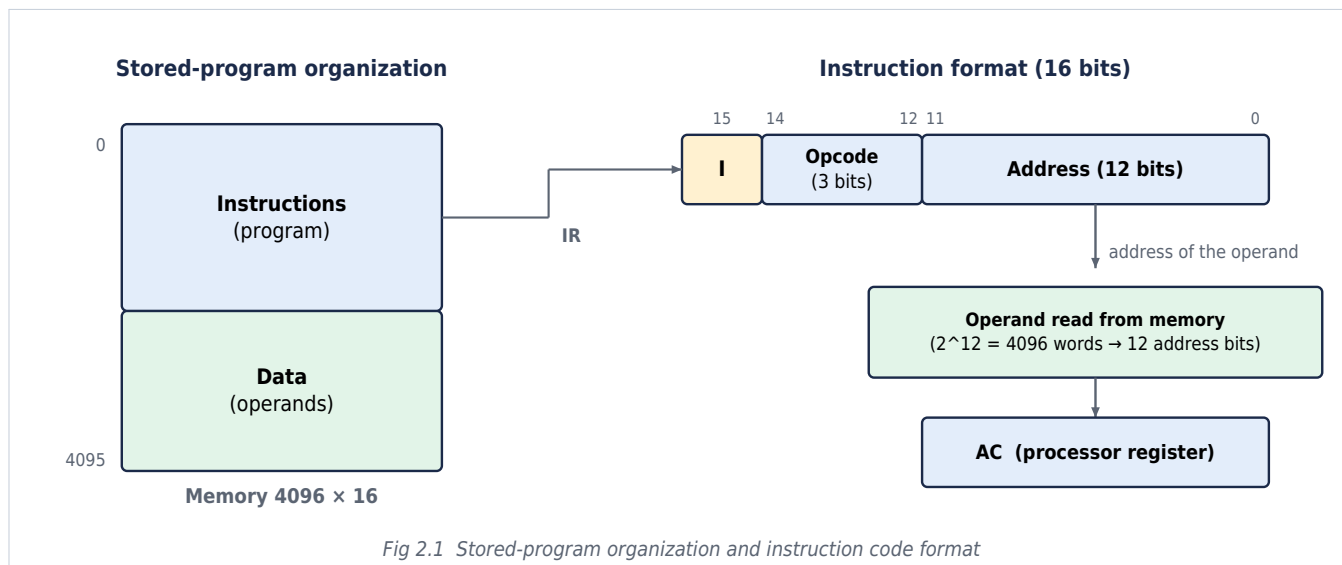
| Basis                 | Register stack                                                            | Memory stack                                                                            |
|-----------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Organization</b>   | Fixed set of registers inside the CPU (e.g. 64 words) with SP, FULL, EMTY | Part of main memory allotted to the stack; SP is a CPU register pointing into memory    |
| <b>Speed</b>          | Very fast: no memory access                                               | Slower: each push/pop is a memory access                                                |
| <b>Capacity</b>       | Small and fixed by hardware                                               | Large and flexible (limited only by memory allotted)                                    |
| <b>Cost</b>           | Expensive per word                                                        | Cheap per word                                                                          |
| <b>Implementation</b> | Extra hardware (flags, wrap-around SP); SP increments on PUSH             | No extra storage hardware; SP decrements on PUSH; overflow checked with limit registers |

**Q46**

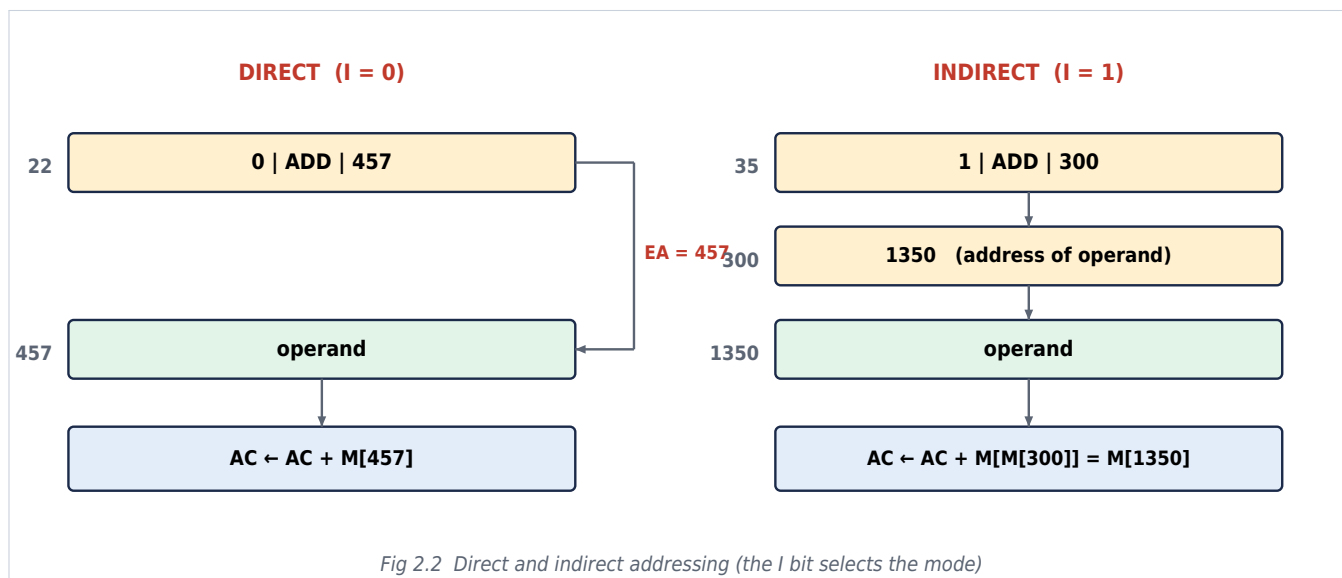
7 marks

**Explain the storage program organization in detail. Explain opcode, instruction, address and memory with diagram**

**Stored-program concept:** the program (instructions) and its data (operands) are stored **together in the same memory**, in different sections. The control unit reads the instructions one by one from the program section (address from PC), decodes them and executes them on the data section. Changing the program only means changing memory contents.



| Term                          | Meaning                                                                                         | In the basic computer                                                            |
|-------------------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Memory</b>                 | Storage of the computer; holds instructions and data as binary words                            | 4096 words × 16 bits (2 <sup>12</sup> = 4096 locations)                          |
| <b>Instruction</b>            | A binary code that specifies a sequence of micro-operations; a program is a set of instructions | 16 bits = I (1) + opcode (3) + address (12)                                      |
| <b>Opcode</b>                 | The part that says <b>what</b> operation to do; n bits give up to 2 <sup>n</sup> operations     | 3 bits (bits 14-12): 000 to 110 memory-reference; 111 = register-reference / I/O |
| <b>Address (operand part)</b> | The part that says <b>where</b> the operand is (or the operand itself in immediate mode)        | 12 bits (bits 11-0); I = 0 direct, I = 1 indirect                                |



**Example: ADD 457** (address in hex) is stored as I = 0, opcode = 001, address = 0100 0101 0111 → 0001 0100 0101 0111 = 1457h. Fetch: IR ← M[PC]. Decode: opcode 001 = ADD. Execute: DR ← M[457], AC ← AC + DR. If I were 1, the word at 457 would hold the **address** of the operand (indirect) and one more memory read is needed.

**DRAW THIS** Draw [Fig 2.5](#) (tap to jump).

# PART 3 - Cheat sheet

last-night revision · opcodes, RTL, addressing modes, formulas

## Opcodes of the basic computer (memory-reference, I = 0 / I = 1)

| Symbol     | I=0  | I=1  | Meaning                                              |
|------------|------|------|------------------------------------------------------|
| <b>AND</b> | 0xxx | 8xxx | $AC \leftarrow AC \wedge M[AR]$                      |
| <b>ADD</b> | 1xxx | 9xxx | $AC \leftarrow AC + M[AR], E \leftarrow \text{Cout}$ |
| <b>LDA</b> | 2xxx | Axxx | $AC \leftarrow M[AR]$                                |
| <b>STA</b> | 3xxx | Bxxx | $M[AR] \leftarrow AC$                                |
| <b>BUN</b> | 4xxx | Cxxx | $PC \leftarrow AR$                                   |
| <b>BSA</b> | 5xxx | Dxxx | $M[AR] \leftarrow PC, PC \leftarrow AR + 1$          |
| <b>ISZ</b> | 6xxx | Exxx | $M[AR] \leftarrow M[AR] + 1, \text{skip if } 0$      |

| Register-reference (7xxx) | Hex                       |  | Input-output (Fxxx)     | Hex  |
|---------------------------|---------------------------|--|-------------------------|------|
| CLA                       | 7800                      |  | INP                     | F800 |
| CLE                       | 7400                      |  | OUT                     | F400 |
| CMA                       | 7200                      |  | SKI                     | F200 |
| CME                       | 7100                      |  | SKO                     | F100 |
| CIR                       | 7080                      |  | ION                     | F080 |
| CIL                       | 7040                      |  | IOF                     | F040 |
| INC                       | 7020                      |  | HLT (also register-ref) | 7001 |
| SPA / SNA / SZA / SZE     | 7010 / 7008 / 7004 / 7002 |  |                         |      |

## Fetch, decode, indirect - memorise verbatim

| T0                     | AR $\leftarrow$ PC                                                                        |
|------------------------|-------------------------------------------------------------------------------------------|
| T1                     | $IR \leftarrow M[AR], PC \leftarrow PC + 1$                                               |
| T2                     | $D0..D7 \leftarrow \text{decode } IR(12-14), AR \leftarrow IR(0-11), I \leftarrow IR(15)$ |
| T3 (indirect, D7'I)    | $AR \leftarrow M[AR]$                                                                     |
| T3 (reg-ref / I-O, D7) | execute now, $SC \leftarrow 0$                                                            |

## Interrupt cycle

| RT0 | AR $\leftarrow$ 0, TR $\leftarrow$ PC                                   |
|-----|-------------------------------------------------------------------------|
| RT1 | $M[AR] \leftarrow TR, PC \leftarrow 0$                                  |
| RT2 | $PC \leftarrow PC+1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$ |

Condition to set R:  $T0' T1' T2' (IEN) (FGI+FG0) : R \leftarrow 1$ . Return from ISR:  $BUN \ 0 \ I$ .

## Addressing modes - one-line effective address (EA)

| Mode                     | EA                                    |
|--------------------------|---------------------------------------|
| <b>Immediate</b>         | operand is in the instruction (no EA) |
| <b>Direct</b>            | EA = address field                    |
| <b>Indirect</b>          | EA = $M[\text{address field}]$        |
| <b>Register</b>          | operand is in the register (no EA)    |
| <b>Register indirect</b> | EA = content of the register          |
| <b>Indexed</b>           | EA = address field + index register   |
| <b>Relative</b>          | EA = address field + PC               |

| Mode            | EA                                     |
|-----------------|----------------------------------------|
| Base register   | EA = address field + base register     |
| Autoincr./decr. | EA = register (then register $\pm 1$ ) |

## 2's complement subtraction (5 steps)

Same bit-width  $\rightarrow$  invert the number being subtracted  $\rightarrow$  add 1  $\rightarrow$  add to the first number  $\rightarrow$  **discard final carry**.

## ALU select lines (S3 S2 pick the unit, S1 S0 pick Y, Cin = extra +1)

| S3S2 | Unit                                 | S1S0 = 10, Cin = 1 |
|------|--------------------------------------|--------------------|
| 00   | Arithmetic ( $Y = 0 / B / B' / 1s$ ) | <b>subtract</b>    |
| 01   | Logic (AND/OR/XOR/NOT)               | -                  |
| 10   | Shift right                          | -                  |
| 11   | Shift left                           | -                  |

## General register organization - control word

SELA (3) | SELB (3) | SELD (3) | OPR (5) e.g.  $R1 \leftarrow R2 + R3 = 010\ 011\ 001\ 00010$ .

## Register windows formula

Window =  $L + 2C + G$  Register file =  $(L + C) \times W + G$  (G=global, L=local, C=common, W=windows).

## Microinstruction fields

F1 | F2 | F3 (3+3+3) | CD (2) | BR (2) | AD (7) BR: 00 JMP, 01 CALL, 10 RET, 11 MAP. CD: 00 U, 01 I, 10 S, 11 Z.

## Pipeline speedup

$S = (k \cdot n) / (k + n - 1)$  for k segments, n tasks; as  $n \rightarrow \infty$ ,  $S \rightarrow k$ .

## RPN quick reference

Infix  $(A+B) \times (C-D) \rightarrow$  Postfix  $A\ B\ +\ C\ D\ -\ \times$ . Scan left $\rightarrow$ right: operand = PUSH, operator = POP two / compute / PUSH.

## One-paragraph answers for the two most awkward Qs

**MEMORIZE** **Q25 NBR** = label for the constant  $-N$  (e.g.  $-100$ ) used to initialise the loop counter; not an instruction. **Q46 stored-program** = instructions + data share memory; instruction = I (1) + opcode (3) + address (12); opcode selects the operation, address/operand tells where.

## Numbers to double-check on the day (from Q4, Q21, Q35)

| Question  | Key numbers                                                                      |
|-----------|----------------------------------------------------------------------------------|
| Q4        | AND = 00100100, XOR = 11011011                                                   |
| Q21       | $R1+R2=11110000$ , $R1-R2=01111010$ , AND=00110001, XOR=10001110                 |
| Q35       | sum=00000100(carry1), diff=01010110, AND=00000101, shl2=10110100, ashr2=00010101 |
| Q14 / Q33 | EA = 750 (indexed) / EA = 1550 (relative)                                        |